

尚硅谷嵌入式技术之 C 语言（高级）

（作者：尚硅谷研究院）

版本：V1.6.0

第 10 章 数组

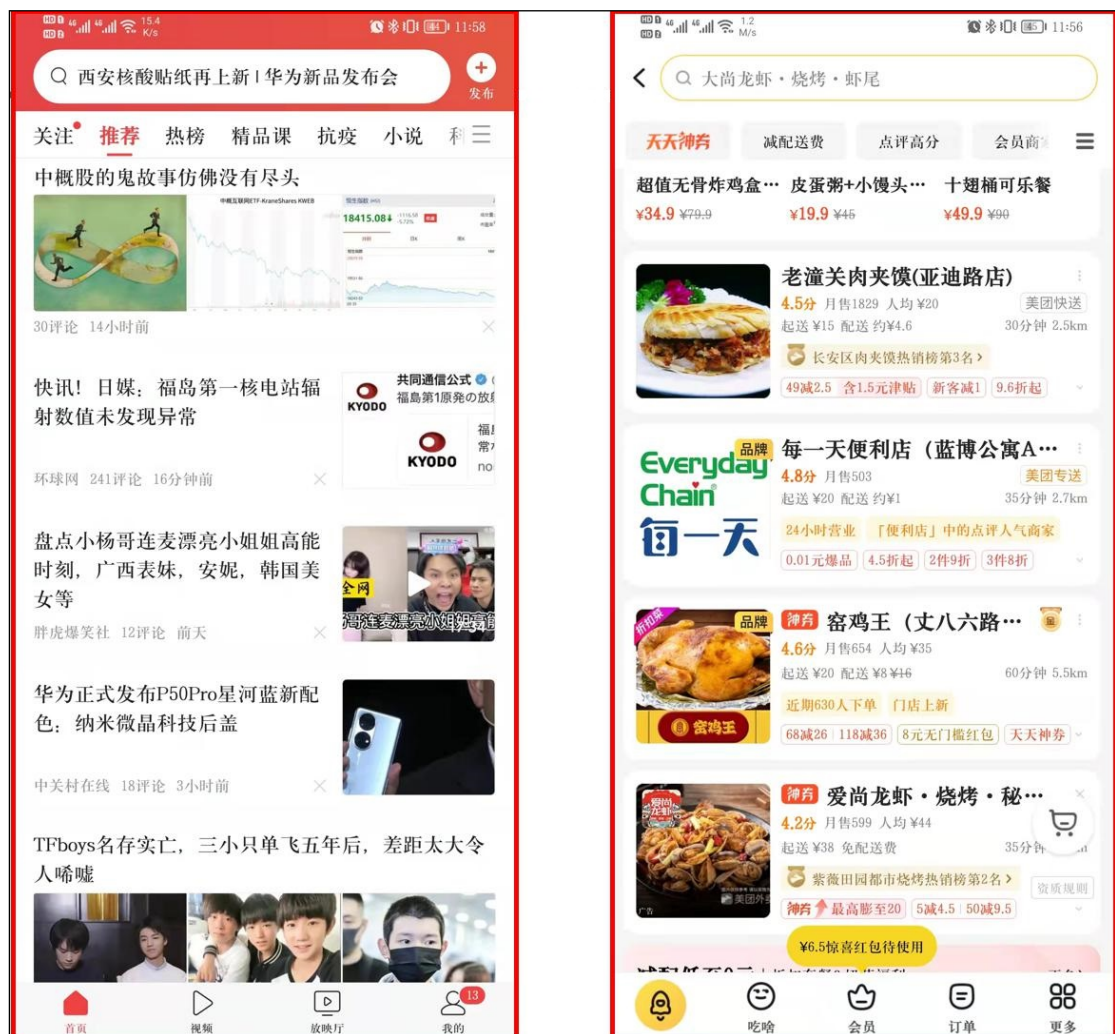
10.1 数组的概念

10.1.1 为什么需要数组

1) 需求分析 1:

需要统计某公司 50 个员工的工资情况，例如计算平均工资、找到最高工资等。用之前知识，首先需要声明 50 个变量来分别记录每位员工的工资，这样会很麻烦。因此我们可以将所有的数据全部存储到一个容器中统一管理，并使用容器进行计算。

2) 需求分析 2:



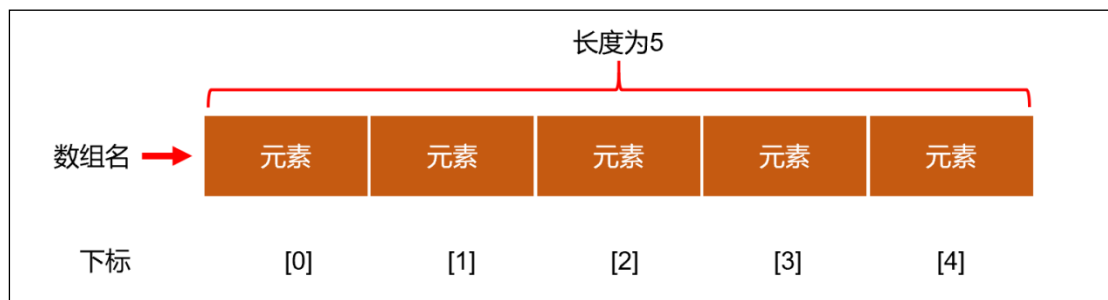
左边图例中有许多条新闻信息，如果用变量来表示一条新闻信息，我们将需要非常多的变量，如果将多条新闻信息存储到一个容器中统一管理将会非常方便。右边图例中有许多条外卖商家数据，也是同样的道理，将多条外卖商家数据存储到一个容器中统一管理将会非常方便。

3) 容器的概念:

- 生活中的容器：水杯（装水等液体），衣柜（装衣服等物品），集装箱（装货物等）。
- 程序中的容器：将多个数据存储到一起，每个数据称为该容器的元素。

10.1.2 什么是数组

数组（Array），是多个相同类型数据按一定顺序排列的集合，并使用一个标识符命名，并通过编号（索引，亦称为下标或角标）的方式对这些数据进行统一管理。



10.1.3 数组相关概念

- (1) **数组名**：本质上是一个标识符常量，命名需要符合标识符规范。
- (2) **元素**：同一个数组中的元素必须是相同的数据类型。
- (3) **下标（索引、角标）**：从 0 开始的连续数字。
- (4) **数组的长度**：表示元素的个数。

10.1.4 数组的特点

- (1) 创建数组时会在内存中开辟一整块连续的空间，占据的空间的大小，取决于数组的长度和数组中元素的类型。
- (2) 数组中的元素在内存中是依次紧密排列的且有序的。
- (3) 数组一旦初始化完成，其长度就是确定的，数组的长度一旦确定，就不能修改。
- (4) 我们可以直接通过索引（下标）获取指定位置的元素，速度很快。

10.2 数组操作

10.2.1 数组的定义

1) 方式一：先指定元素的个数和类型，再进行初始化

```
// 定义数组，数组名字是 arr1，元素类型是 int，元素个数是 3 个
int arr1[3];

// 定义完成后再给元素赋值
arr1[0] = 100;
arr1[1] = 200;
arr1[2] = 300;
```

2) 方式二：指定元素的类型和个数并同时进行初始化

```
// 定义完数组直接进行初始化
int arr2[3] = {4,5,6};
```

3) 方式三：指定元素的类型，不指定元素个数，同时进行初始化

```
// 没有指定元素个数，系统会自动计算
```

```
int arr3[] = {7,8,9,10};
```

10.2.2 访问数组元素

通过“数组名[下标]”可以访问数组中的元素，案例如下

```
#include <stdio.h>

int main()
{
    // 定义 4 个元素的数组
    int nums[4] = {10, 20, 30, 40};

    // 修改第二个元素的值
    nums[1] += 100;

    // 读取元素的值
    printf("第一个元素的值: %d\n", nums[0]); // 第一个元素的值: 10
    printf("第二个元素的值: %d\n", nums[1]); // 第一个元素的值: 120
    printf("第三个元素的值: %d\n", nums[2]); // 第一个元素的值: 30
    printf("第四个元素的值: %d\n", nums[3]); // 第一个元素的值: 40

    return 0;
}
```

输出结果：

```
第一个元素的值: 10
第二个元素的值: 120
第三个元素的值: 30
第四个元素的值: 40
```

10.2.3 数组越界

数组下标必须在指定范围内使用，超出范围视为越界。



代码示例：

```
#include <stdio.h>
```

```
int main() {

    // 定义数组 没有指定长度
    int nums[5] = {10,20,30,40,50};

    printf("下标是 0 的元素: %d\n", nums[0]); // 10
    printf("下标是 4 的元素: %d\n", nums[4]); // 50
    printf("下标是 -1 的元素: %d\n", nums[-1]); // 得到的是不确定结果
    printf("下标是 5 的元素: %d\n", nums[5]); // 得到的是不确定结果

    return 0;
}
```

输出结果:

```
下标是 0 的元素: 10
下标是 4 的元素: 50
下标是 -1 的元素: 0
下标是 5 的元素: 0
```

10.2.4 计算数组长度

数组长度（元素个数）是在数组定义时明确指定且固定的，我们不能在运行时直接获取数组长度，但是，我们可以通过 `sizeof` 运算符间接计算出数组长度，计算步骤如下：

- （1）使用 `sizeof` 运算符计算出整个数组的字节长度。
- （2）由于数组成员是同一类型，每个元素字节长度相等，用整个数组的字节长度除以单个元素的字节长度就可以得到数组的长度。



代码示例:

```
#include <stdio.h>

int main()
{
    // 定义数组 没有指定长度
    int nums[] = {10, 20, 30, 40, 50, 60, 70};

    // 计算数组总的字节长度
    int arrByteLen = sizeof nums;

    // 用总字节长度除以单个元素的字节长度
```

```
int arrLen = arrByteLen / sizeof nums[0];

printf("数组的长度: %d", arrLen); // 数组的长度: 7

return 0;
}
```

输出结果:

数组的长度: 7

10.2.5 遍历数组

遍历数组是指按顺序访问数组中的每个元素，以便读取或修改它们，编程中一般使用循环结构对数组进行遍历。

代码示例:

```
#include <stdio.h>

int main()
{
    // 定义数组
    int arr[10] = {12, 2, 31, 24, 15, 36, 67, 108, 29, 51};

    // 计算数组长度
    int len = sizeof arr / sizeof arr[0];

    // 遍历数组中的元素
    printf("遍历数组中的元素: \n");
    for (int i = 0; i < len; i++)
    {
        printf("%d: %d \n", i, arr[i]);
    }

    return 0;
}
```

输出结果:

遍历数组中的元素:

0: 12
1: 2
2: 31
3: 24
4: 15
5: 36
6: 67
7: 108
8: 29
9: 51

我们还可以通过数组遍历完成数组的初始化赋值，看下面案例。

创建长度为 10 的数组，元素依次赋值为 0，1，2，3，4，5，6，7，8，9，并按逆序输出每个元素，代码如下：

```
#include <stdio.h>

int main()
{
    // 声明数组
    int arr[10];

    // 计算数组长度
    int len = sizeof arr / sizeof arr[0];

    // 遍历数组进行初始化赋值,依次赋值为 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
    for (int i = 0; i < len; i++)
    {
        arr[i] = i;
    }

    // 通过遍历逆序输出数组元素
    for (int i = len-1; i >= 0; i--)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

输出结果：

9 8 7 6 5 4 3 2 1 0

10.2.6 数组应用案例

1) 案例 1

计算数组中所有元素的和以及平均数。

```
#include <stdio.h>

int main()
{
    // 定义数组
    int arr[10] = {12, 2, 31, 24, 15, 36, 67, 108, 29, 51};

    // 计算数组长度
```

```
int len = sizeof arr / sizeof arr[0];

// 定义变量用于存储数组的和
int sum = 0;

// 遍历数组计算所有元素的和
for (int i = 0; i < len; i++)
{
    sum += arr[i];
}

// 计算平均值
int avg = sum / len;

// 输出结果
printf("数组元素之和: %d\n", sum);
printf("数组元素平均值: %d", avg);

return 0;
}
```

输出结果:

数组元素之和: 375
数组元素平均值: 37

2) 案例 2

取出数组中之值最大的元素。

思路分析:

max

12

arr

12

2

31

24

15

36

67

108

29

51

代码：

```
#include <stdio.h>

int main()
{
    // 定义数组
    int arr[10] = {12, 2, 31, 24, 15, 36, 67, 108, 29, 51};

    // 计算数组长度
    int len = sizeof arr / sizeof arr[0];

    // 定义变量用于存储最大元素值,初始值为第一个元素的值
    int max = arr[0];

    // 遍历数组比较最大值
    for (int i = 0; i < len; i++)
    {
        // 如果当前遍历到的元素比 max 大,就将当前元素的值赋值给 max
        if (arr[i] > max) {
            max = arr[i];
        }
    }

    // 输出结果
    printf("最大的元素值: %d", max);

    return 0;
}
```

输出结果：

最大的元素值: 108

10.3 字符数组

10.3.1 字符数组（字符串）介绍

用来存放字符的数组称为字符数组，也可以称为字符串。字符串的输入输出格式占位符是 %s。

H	e	l	l	o		W	o	r	l	d	!	\0
---	---	---	---	---	--	---	---	---	---	---	---	----

结束字符

字符串结尾，会自动添加一个 \0 作为字符串结束的标志，所以字符数组最后一个元素

必须是 `\0`。

`\0` 是 ASCII 码表中的第 0 个字符，用 NUL 表示，称为空字符，该字符既不能显示，也不是控制字符，输出该字符不会有任何效果，它在 C 语言中仅作为字符串的结束标志。

10.3.2 字符数组（字符串）的定义

1) 方式一：最后一个元素设置成 `\0`

在给某个字符数组赋值时，赋值的元素个数小于字符数组的长度，则会自动在后面加 `\0`，表示字符串结束；赋值的元素的个数等于该数组的长度（或不指定数组长度），则不会自动添加 `\0`。

```
#include <stdio.h>

int main()
{
    char str1[12] = {'H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd', '\0'}; // 显式地设值 \0
    char str2[4] = {'t', 'o', 'm'}; // 后面自动添加 \0
    char str3[] = {'j', 'a', 'c', 'k'}; // 不会自动添加 \0

    printf("str1=%s \n", str1);
    printf("str2=%s \n", str2);
    printf("str3=%s \n", str3); // 由于没有结束标识，会包括相邻内存的数据，直到遇到结束标记

    return 0;
}
```

输出结果：

```
str1=Hello World
str2=tom
str3=jacktom
```

2) 方式二：简化写法

```
#include <stdio.h>

int main()
{
    char str1[] = {"I am happy"}; // 后面自动添加 \0
    char str2[] = "I am happy"; // 省略{}号,后面自动添加 \0

    printf("\n str1=%s", str1);
    printf("\n str2=%s", str2);
}
```

```
    return 0;
}
```

输出结果：

```
str1=I am happy
str2=I am happy
```

10.3.3 字符数组（字符串）的访问和遍历

字符数组（字符串）的访问和遍历，按照一般数组的方式访问和遍历即可。

```
#include <stdio.h>

int main()
{
    // 定义字符串
    char greeting[] = "Hello";

    // 计算字符串长度
    int len = sizeof greeting / sizeof greeting[0];

    printf("%s \n", greeting);
    printf("数组长度: %d \n", len);
    printf("第 3 个字符: %c \n", greeting[2]);
    printf("\n");

    // 遍历字符串
    for (int i = 0; i < len; i++)
    {
        printf("%c \n", greeting[i]);
    }

    return 0;
}
```

输出结果：

```
Hello
数组长度: 6
第 3 个字符: l

H
e
l
l
o
```

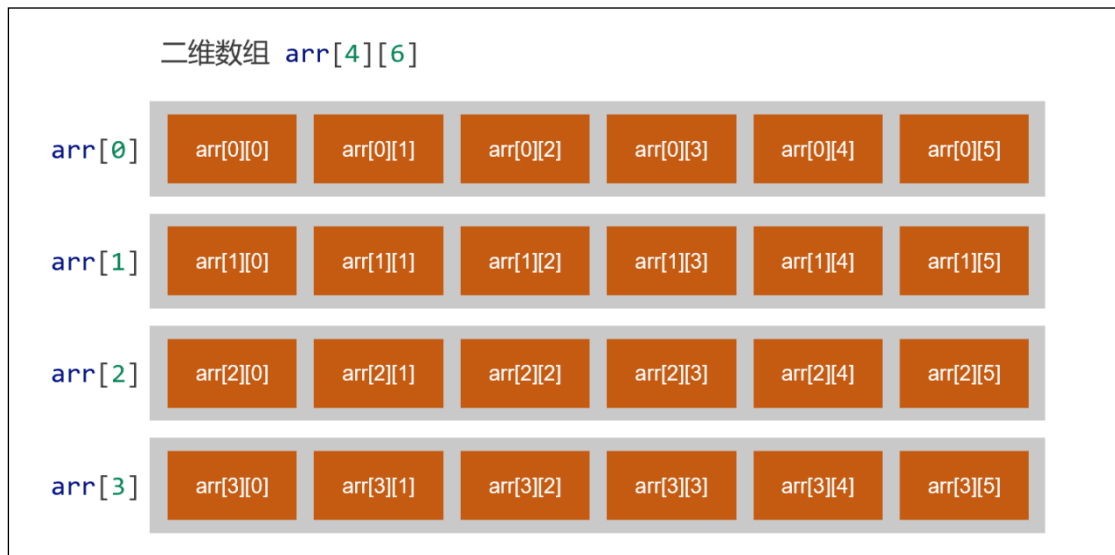
10.4 多维数组

10.4.1 多维数组介绍

如果数组的元素还是数组，这样的数组就称为多维数组。这种多层次的结构允许我们以表格或矩阵的方式组织数据，其中每个维度都对应于不同的行、列或更多的维度，使数据更加结构化和有组织。

多维数组可以分为二维数组、三维数组、四维数组 等，这里我们以二维数组为例进行演示。

下图是一个四行六列的二维数组示意图：



10.4.2 二维数组的定义

1) 方式 1: 先定义再初始化

```
// 定义一个 4 行 6 列的二维数组
int a[4][6];

// 进行初始化赋值
a[0][0] = 10;
a[0][1] = 20;
a[0][2] = 30;
a[0][3] = 40;
a[0][4] = 50;
a[0][5] = 60;
a[1][0] = 100;
a[1][1] = 200;
.....
```

2) 方式 2: 直接定义并初始化

```
// 定义一个 4 行 6 列的二维数组，以矩阵的形式初始化
int a[4][6] = {
    {10, 20, 30, 30, 40, 60},
    {100, 200, 300, 400, 500, 600},
    {1000, 2000, 3000, 4000, 5000, 6000},
    {10000, 20000, 30000, 40000, 50000, 60000}
};

// 定义一个 4 行 6 列的二维数组，会自动匹配到各行各列
int b[4][6] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,
17, 18, 19, 20, 21, 22, 23, 24};

// 如果所赋值的数量可以与元素数量对应，第一维的数组长度可以不给出
int b[][6] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,
17, 18, 19, 20, 21, 22, 23, 24};
```

10.4.3 二维数组的访问和遍历

访问二维数组的元素，需要使用两个下标（索引），一个用于访问行（第一维），另一个用于访问列（第二维），我们通常称为行下标（行索引）或列下标（列索引）。

遍历二维数组，需要使用双层循环结构。

思路分析：

```
int map[3][4] = {
    {1, 2, 3, 4},
    {11, 12, 13, 14},
    {21, 22, 23, 24}
};
```

行下标



列下标



```
map[][]
```

1	2	3	4
11	12	13	14
21	22	23	24

代码示例：

```
#include <stdio.h>
```

```
int main()
{
    // 定义一个 3 行 4 列的数组
    int map[3][4] = {
        {1, 2, 3, 4},
        {11, 12, 13, 14},
        {21, 22, 23, 24}};

    // 计算第一维度的长度
    int rows = sizeof(map) / sizeof(map[0]);
    // 计算第二维度的长度
    int cols = sizeof(map[0]) / sizeof(int);

    // 遍历输出每个元素
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            printf("%d\t", map[i][j]); // \t 可以输出得更加整齐
        }
        printf("\n");
    }

    // 计算所有元素的和
    int sum = 0;
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            sum += map[i][j];
        }
    }
    printf("所有元素的和: %d", sum); // 所有元素的和: 150

    return 0;
}
```

输出结果:

1	2	3	4
11	12	13	14
21	22	23	24

所有元素的和: 150

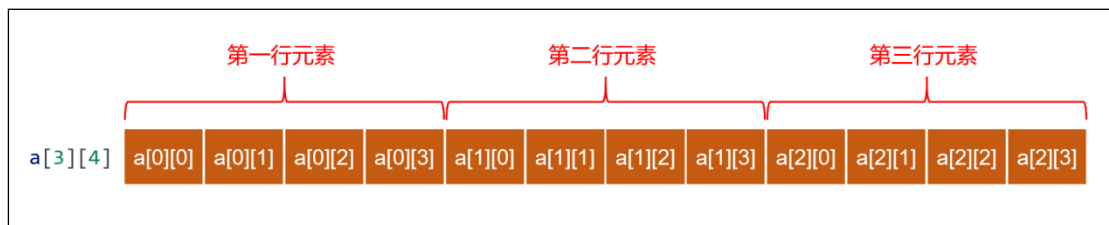
10.4.4 二维数组的内存分析

用矩阵形式（如 3 行 4 列形式）表示二维数组，是逻辑上的概念，能形象地表示出行列

关系。而在内存中，各元素是连续存放的，不是二维的，是线性的。

C 语言中，二维数组中元素排列的顺序是按行存放的。即：先顺序存放第一行的元素，再存放第二行的元素。

比如，举例，数组 `a[3][4]` 在内存中的存放：



10.4.5 二维数组应用案例

现在有三个班，每个班五名同学，用二维数组保存他们的成绩，并求出每个班级平均分、以及所有班级平均分，数据要求从控制台输入。

思路分析：

总成绩	0	行下标	0			
班级总成绩	0	列下标	0			
第一个班平均分	45	67	80	92	62	第一个班级
第二个班平均分	82	55	69	94	72	第二个班级
第三个班平均分	99	61	72	72	58	第三个班级
总平均分						

代码：

```
#include <stdio.h>

int main()
{
    // 定义 一个 3 行 5 列的数组
    double scores[3][5];

    // 定义变量存储第一个维度长度和第二个维度长度
```

```
int rows = 3, cols = 5;

// 遍历二维数组进行赋值
for (int i = 0; i < rows; i++)
{
    for (int j = 0; j < cols; j++)
    {
        printf("请输入第%d个班的第%d个学生的成绩: ", i + 1, j + 1);
        scanf("%lf", &scores[i][j]);
    }
}

// 遍历数组，计算每个班级的平均分和总的平均分
// 定义变量记录所有班级总分数
double total_sum = 0;
// 定义变量记录每个班级总分数
double class_sum = 0;
// 遍历班级（行）
for (int i = 0; i < rows; i++)
{
    // 将当前班级总分数重置为 0
    class_sum = 0;
    // 遍历当前班级的每个成绩（该行的每一列）
    for (int j = 0; j < cols; j++)
    {
        class_sum += scores[i][j];
    }
    // 输出当前班级的平均分
    printf("第%d个班级的平均分为: %.2f \n", i + 1, class_sum / cols);
    // 将该班级总分加入到所有总分中
    total_sum += class_sum;
}

// 输出所有班级平均分
printf("所有班级的平均分为: %.2f", total_sum / (rows * cols));

return 0;
}
```

10.5 课后练习

- (1) 从终端循环输入 5 个成绩，保存到 double 数组，并输出。
- (2) 一个养鸡场有 6 只鸡，它们的体重分别是 3kg, 5kg, 1kg, 3.4kg, 2kg, 50kg。请

问这六只鸡的总体重是多少?平均体重是多少?

(3) 创建一个 `char` 类型的 26 个元素的数组，分别放置'A'-'Z'。使用 `for` 循环访问所有元素并打印出来。

(4) 请求出一个数组的最小值，并得到对应的索引。

第 11 章 函数

11.1 函数基本概念

11.1.1 为什么需要函数



《街霸》游戏中，实现人物出拳、出脚或跳跃等动作都需要编写 50-80 行的代码，在每次出拳、出脚或跳跃的地方都需要重复地编写这 50-80 行代码，这样程序会变得很臃肿，可读性也非常差。为了解决代码重复编写的问题，可以将出拳、出脚或跳跃的代码提取出来放在一个 `{ }` 中，并为这段代码起个名字，这样在每次的出拳、出脚或跳跃的地方通过这个名字来调用这个 `{ }` 的代码就可以了。

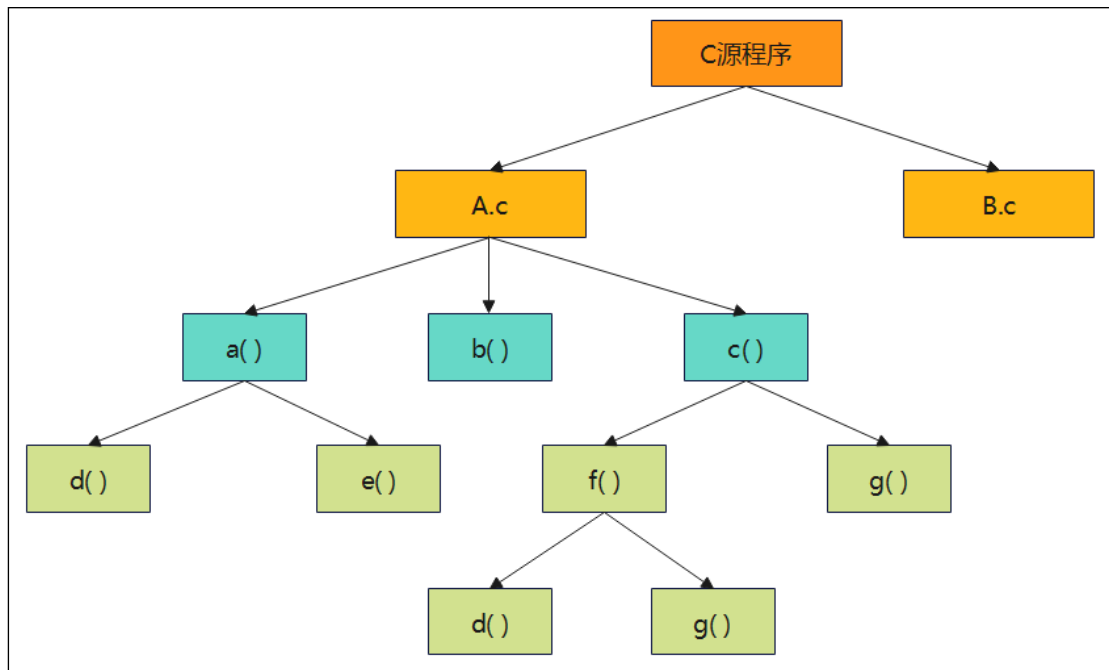
提取出来的代码可以看作是程序中定义的一个函数，程序在需要出拳、出脚或跳跃时调用该函数即可。

11.1.2 什么是函数

函数是一种可重复使用的代码块，用于执行特定的任务或操作。

函数允许我们将代码逻辑组织成独立的单元，从而提高了代码的可读性、可维护性和重用性。

一个 C 程序可以由一个或多个源文件构成（C 文件扩展名是“.c”），一个源文件是一个编译单位。一个源文件可以由若干个函数构成，函数之间可以相互调用。也就是说，函数是 C 程序基本的组成单位。



11.1.3 函数的作用

- （1）封装功能，将一个完整的功能封装成函数，提高代码的结构化和复用性。
- （2）代码模块化，将程序按照功能拆分成若干模块单元,有助于降低复杂度。
- （3）增强可维护性，如果需要修改某项功能，只需要调整对应的函数代码。
- （4）隔离细节，通过函数调用可以隐藏实现细节，只关心输入输出。

11.1.4 函数的分类

C 语言中，从使用的角度，函数可以分类两类。

- （1）库函数，也称为标准函数，是由 C 系统提供的，用户不必自己定义，可直接使用它们，使用库函数，必须包含 `#include` 对应的头文件。
- （2）自定义函数，解决具体需求而自己定义的函数，需先定义再使用。

11.2 函数基本语法

11.2.1 声明函数语法

```

返回类型 函数名（参数列表）
{

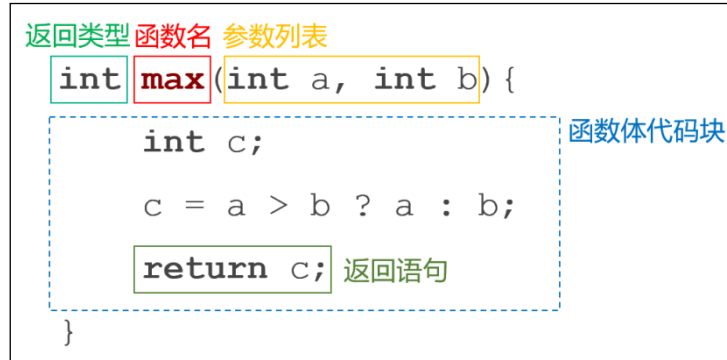
```

```

函数体语句 1;
函数体语句 2;
... 函数体语句 n;
return 返回值;
}

```

1) 函数语法的组成部分



(1) 函数名：函数被调用时使用的名字，函数名要符合标识符规范。

(2) 函数体：函数中所包含的代码块，用于实现函数的具体功能和操作。

(3) 参数：用于接收调用函数时传递进来的值。

(4) 返回值：函数执行完毕后，从函数传回到调用点的值，返回值的类型要与函数名前面的返回类型对应，如果没有返回值，返回类型可以写 `void`。

2) 代码示例

```

#include <stdio.h>

// 声明函数
void func()
{
    printf("hello func\n");
}

// 实现两个数字相减
int minus(int m, int n)
{
    return m - n;
}

// 取两个数字中的最大值
int max(int a, int b)
{
    int c;
    c = a > b ? a : b;
}

```

```
    return c;
}

// 主函数
int main()
{
    return 0;
}
```

3) 函数不能嵌套声明

C 程序中的所有函数都是互相独立的，一个函数并不从属于另一个函数，即函数不能嵌套声明。

```
//错误演示
int func1(int a,int b) //第 1 个函数的定义
{
    ...
    int func2(int c,int d) //第 2 个函数的定义
    {
        ...
    }
    ...
}
```

有些编译器的扩展允许函数嵌套声明，但这不是 C 标准的一部分，代码的可移植性可能会受到影响，强烈不建议。

11.2.2 调用函数

函数名后面加上圆括号即是函数的调用，参数写在小括号中，函数每调用一次，函数体语句都会执行一遍。



参数



函数体



返回值

```
#include <stdio.h>

// 声明函数
void func()
{
    printf("hello func\n");
}

// 实现两个数字相减
int minus(int m, int n)
{
    return m - n;
}

// 取两个数字中的最大值
int max(int a, int b)
{
    int c;
    c = a > b ? a : b;
    return c;
}

// 主函数
int main()
{
    // 函数的调用
    func();
    func();
}
```

```
printf("17-90 的结果: %d\n", minus(17, 90));
printf("21-180 的结果: %d\n", minus(21, 180));
printf("12 和 16 之间较大的是: %d\n", max(12, 16));
printf("45 和 31 之间较大的是: %d\n", max(45, 31));

return 0;
}
```

输出结果:

```
hello func
hello func
17-90 的结果: -73
21-180 的结果: -159
12 和 16 之间较大的是: 16
45 和 31 之间较大的是: 45
```

11.2.3 函数的返回值

函数调用后数能得到一个确定的值，这就是函数的返回值，返回值常常是一个计算的结果，或是用来作为判断函数执行状态的标记。

函数返回值分为以下三种情况：

(1) 无返回值类型：针对函数无返回值或明确不需返回值的情况，使用 `void`（即空类型）表示。

(2) 有返回值类型 指明具体的类型，比如，`int`、`float`、`char` 等。需要在函数体内与 `return` 语句搭配使用。

(3) 如果返回值类型不是 `void`，但函数中没有 `return` 语句，则函数会返回一个不确定的值。

代码示例：

```
#include <stdio.h>

// 无返回值
void fun01()
{
    printf("fun01\n");
}

// 有明确的返回值
double fun02()
{
    return 3.14;
}
```

```
}

// 函数返回一个不确定的值
int fun03()
{
    10 + 20;
}

// 返回类型与 return 的值类型不一致，可能造成精度损失
int fun04()
{
    return 20.89;
}

int main()
{
    fun01();
    printf("%f \n", fun02());
    printf("%d \n", fun03()); // 返回一个不确定的值
    printf("%d \n", fun04()); // 返回值有精度损失

    return 0;
}
```

输出结果：

```
fun01
3.140000
10
20
```

11.2.4 函数的参数

函数的参数分为形参与实参：

- 形参：在定义函数时，函数名后面括号()中声明的变量称为形式参数，简称形参。
- 实参：在调用函数时，函数名后面括号()中的使用的常量、变量、表达式称为实际参数，简称实参。

注意：实参的数量要与形参的数量一致，否则报错。

```
#include <stdio.h>

// x 和 y 即为形参
int func(int x, int y)
{
    return x + y;
}
```

```
int main()
{
    // 3 和 5 即为实参
    int sum = func(3, 5);
    printf("%d \n", sum); // 8;

    // 如果实参数量如形参不一致 会报错
    // func(100, 299, 300);
    // func(100);

    return 0;
}
```

输出结果：

8

11.3 主函数

1) 主函数的作用

主函数是程序的入口函数，即所有的程序一定要包含一个主函数，程序总是从这个函数开始执行，如果没有该函数，程序就无法启动。

主函数中可以调用其它函数，但其它函数不能反过来调用主函数，主函数也不能调用自己。

2) 主函数的返回值

C 语言约定，主函数返回值 0 表示运行成功，如果返回其它非零整数，就表示运行失败。默认情况下，如果主函数里面省略 `return 0` 这一行，编译器会自动加上，即 `main()` 的默认返回值为 0。但是为了保持统一的代码风格，不建议省略。

3) 主函数的参数

主函数的声明中可以带有两个参数，格式如下：

```
int main(int argc, char *argv[])
{
    // 函数体
}
```

其中，形参 `argc`，全称是 `argument count`，表示传给程序的参数个数，其值至少是 1；而 `argv`，全称是 `argument value`，`argv[]` 是一个指针数组（12.3.2 小节会具体讲解），我们可以暂时将 `argv` 理解为是一个数组，数组的每个元素都是字符串。

这种方式可以通过命令行的方式执行源代码，并接收指定的字符串传给参数 `argv`。

我们创建一个名为 demo.c 的源文件，代码如下：

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("argc = %d\n", argc);
    // 函数体
    for (int i = 0; i < argc; i++)
    {
        printf("%s\n", argv[i]);
    }

    return 0;
}
```

在 demo.c 所在的目录下打开命令行工具，使用 gcc 命令进行编译，如下所示：



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.3570]
(c) Microsoft Corporation。保留所有权利。

C:\Preparation\Embedded\01CLang\code\project\project\src>gcc -o demo demo.c 编译
C:\Preparation\Embedded\01CLang\code\project\project\src>demo Beijing Shanghai Shenzhen 运行
argc = 4
demo
Beijing
Shanghai
Shenzhen
C:\Preparation\Embedded\01CLang\code\project\project\src>
```

11.4 函数原型

默认情况下，函数必须先声明，后使用。由于程序总是先运行 main() 函数，导致所有其它函数都必须在 main() 函数之前声明。

如果想将函数声明写在后面，可以在程序开头处给出函数原型。函数原型，就是提前告诉编译器，每个函数的返回类型和参数类型。其它信息都不需要，也不用包括函数体，具体的函数实现可以后面再补上。

```
#include <stdio.h>

// 使用函数原型进行声明
int twice(int);

// 主函数
int main()
{
    printf("%d\n", twice(100));
}
```

```
    return 0;
}

// 函数定义
int twice(int num)
{
    return 2 * num;
}
```

输出结果：

200

11.5 作用域

11.5.1 作用域的概念

作用域用于确定在代码中某个标识符（如变量、标识符常量、数组等）的可见性和访问范围，它决定了在程序的哪些部分可以引用或访问该标识符。

作用域可以分为全局作用域、局部作用域、块级作用域。

同一个作用域中不能声明同名的标识符。

11.5.2 全局作用域

在函数和代码块（分支语句、循环语句等）以外定义的变量、标识符常量、数组等具有全局作用域，在程序的任何地方都可以被访问，通常称它们为全局变量、全局常量、全局数组等。

代码示例：

```
#include <stdio.h>

// 全局变量
double money = 1.1;

// 全局常量
const double PI = 3.14;

// 全局数组
char msg[] = "Hello World";

// 全局函数
void func()
{
    printf("func 函数中使用全局数据: \n");
}
```

```
    printf("money=%.2f \n", money);
    printf("PI=%.2f \n", PI);
    printf("msg=%s \n", msg);
    printf("\n");
    money += 100;
}

// 主函数
int main()
{
    // 调用 func()
    func();

    printf("主函数中使用全局数据: \n");
    printf("money=%.2f \n", money);
    printf("PI=%.2f \n", PI);
    printf("msg=%s \n", msg);
    printf("\n");

    // 调用 func()
    func();

    return 0;
}
```

输出结果:

```
func 函数中使用全局数据:
money=1.10
PI=3.14
msg=Hello World

主函数中使用全局数据:
money=101.10
PI=3.14
msg=Hello World

func 函数中使用全局数据:
money=101.10
PI=3.14
msg=Hello World
```

对于全局变量，如果没有显式指定初始值，它们将自动初始化为零值。

对于全局数组，如果没有显式初始化它们，它们的元素将自动初始化为零值，字符数组，将初始化为空字符 \0。

代码示例:

```
#include <stdio.h>
```

```
// 定义全局变量不进行初始化赋值
int a;    // 自动初始化为 0
double b; // 自动初始化为 0.0
char c;   // 自动初始化为空字符 \0

// 定义全局数组不进行初始化
int arr[5]; // 所有元素自动初始化为 0
char msg[6]; // 所有元素自动初始化为空字符 \0

// 主函数
int main()
{
    // 输出全局变量
    printf("a=%d \n", a); // a=0
    printf("b=%f \n", b); // b=0.000000
    printf("c=%c \n", c); // c=
    printf("\n");

    // 计算数组长度
    int len = sizeof arr / sizeof arr[0];
    // 遍历数组 arr
    for (int i = 0; i < len; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n\n");
    printf("字符数组: %s", msg); // 字符数组:

    return 0;
}
```

输出结果:

```
a=0
b=0.000000
c=

0 0 0 0 0

字符数组:
```

11.5.3 局部作用域

在函数内定义的变量、标识符常量、数组等具有局部作用域，只有在该函数内部才能被访问，通常称它们为局部变量、局部常量、局部数组等。需要注意的是，函数的形参也是局

部变量。

代码示例：

```
#include <stdio.h>

void add(int a)
{
    // 局部变量
    int b = 20;
    // 局部常量
    const double PI = 3.14;
    // 局部数组
    int nums[] = {10, 20, 30};

    printf("(a+b+nums[0])*PI=%f \n", (a + b + nums[0]) * PI);
}

int main()
{
    // 调用函数 add
    add(100); // (a + b + nums[0]) * PI = 408.200000

    // add 函数外部使用局部变量
    // printf("%d \n", a); // 报错 'a' undeclared
    // printf("%d \n", b); // 报错 'b' undeclared
    // printf("%f \n", PI); // 报错 'PI' undeclared
    // printf("%d \n", nums[0]); // 报错 'nums' undeclared

    return 0;
}
```

输出结果：

```
(a+b+nums[0])*PI=408.200000
```

如果局部作用域中定义了与全局作用域中同名的标识符，优先使用本作用域中定义的数据，相关代码示例如下：

```
#include <stdio.h>

// 全局变量
int a = 100;
int b = 200;

void add()
{
```

```
int a = 300;
a += 10;
b += 10;
printf("函数 add 内部: a=%d, b=%d \n", a, b);
}

int main()
{
    // 调用函数 add
    add(); // 函数 add 内部: a=310, b=210
    printf("函数 add 外部: a=%d, b=%d \n", a, b); // 函数 add 外部:
a=100, b=210

    return 0;
}
```

输出结果:

```
函数 add 内部: a=310, b=210
函数 add 外部: a=100, b=210
```

与全局变量和全局数组不同，局部变量和局部数组如果没有显式初始化，它们的初始值是机器垃圾值，即系统之前分配给这块空间的值。所以不对局部变量、局部数组初始化是不安全且强烈不建议的。

代码示例:

```
#include <stdio.h>

int main()
{
    // 定义局部变量不进行初始化赋值
    int a;
    // 定义局部数组不进行初始化
    int arr[5];

    // 输出局部变量
    printf("a=%d \n", a);

    // 计算数组长度
    int len = sizeof arr / sizeof arr[0];
    // 遍历数组 arr
    for (int i = 0; i < len; i++)
    {
        printf("%d ", arr[i]);
    }
}
```

```
    return 0;
}
```

输出结果：

```
a=16
130611760 130611760 130611760 130611760 130611760
```

11.5.4 块级作用域

块级作用域是 C99 标准引入的概念，在代码块（分支语句、循环语句等）中定义的变量、标识符常量、数组等具有块级作用域，只有在该代码块内部才能被访问，代码块通常具有花括号 {} 结构。

这些被称为块级变量、块级常量、块级数组等，同时也可以被称为局部变量、局部常量、局部数组，且与函数中的局部变量、局部常量、局部数组具有相同的特性。

代码示例：

```
#include <stdio.h>

int main()
{
    // 该代码块中具有块级作用域
    {
        // 块级变量
        int a = 20;
        // 块级常量
        const double PI = 3.14;

        printf("a*PI=%f \n", a * PI);
    }

    // 分支语句体具有块级作用域
    if (1)
    {
        // 局部数组
        int nums[] = {10, 20, 30};

        printf("%d %d %d \n", nums[0], nums[1], nums[2]);
    }

    // 循环语句中的标记变量 i 是块级变量
    for (int i = 0; i < 5; i++)
    {
        printf("%d ", i);
    }
}
```

```

}

// printf("%d \n", a); // 报错 'a' undeclared
// printf("%f \n", PI); // 报错 'PI' undeclared
// printf("%d \n", nums[0]); // 报错 'nums' undeclared
// printf("%d \n", i); // 报错 'i' undeclared

return 0;
}

```

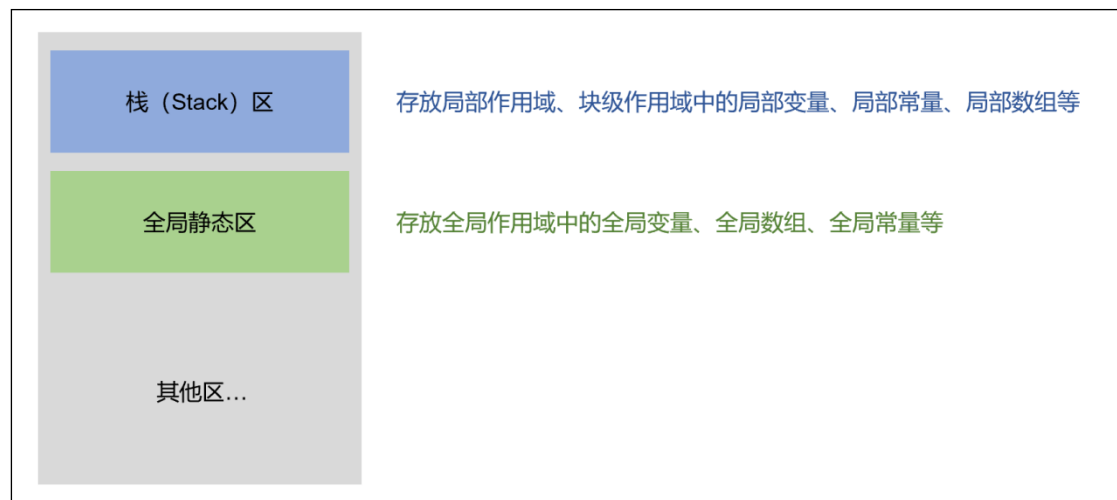
输出结果:

```

b*PI=62.800000
10 20 30
0 1 2 3 4

```

11.5.5 作用域和内存



1) 栈区域

局部变量、局部数组等通常存储在栈 (Stack) 区，这些局部数据的内存分配和释放是自动管理的，它们的生命周期受限于其定义的函数或块级作用域，当函数返回或块级作用域结束时，这些变量的内存会被自动释放。

函数每调用一次就创建一次局部数据，调用结束就销毁，下次调用再创建新的局部数据。

2) 全局静态区域

全局变量、全局数组等存储在全局静态区，这些全局的数据在程序的整个生命周期内都存在，它们在程序启动时被分配，在程序结束时才被释放。

3) 变量生命周期演示

代码示例:

```

#include <stdio.h>

```

```
// 定义全局变量
int len = 5;
// 定义全局数组
int arr[5] = {10, 20, 30, 40, 50};

// 定义函数
void fn(int num)
{
    // 定义局部变量
    int a = 250;
    printf("%d \n", num + a);
}

// 主函数
int main()
{
    // 调用函数
    fn(20);

    // 遍历数组
    for (int i = 0; i < 5; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");

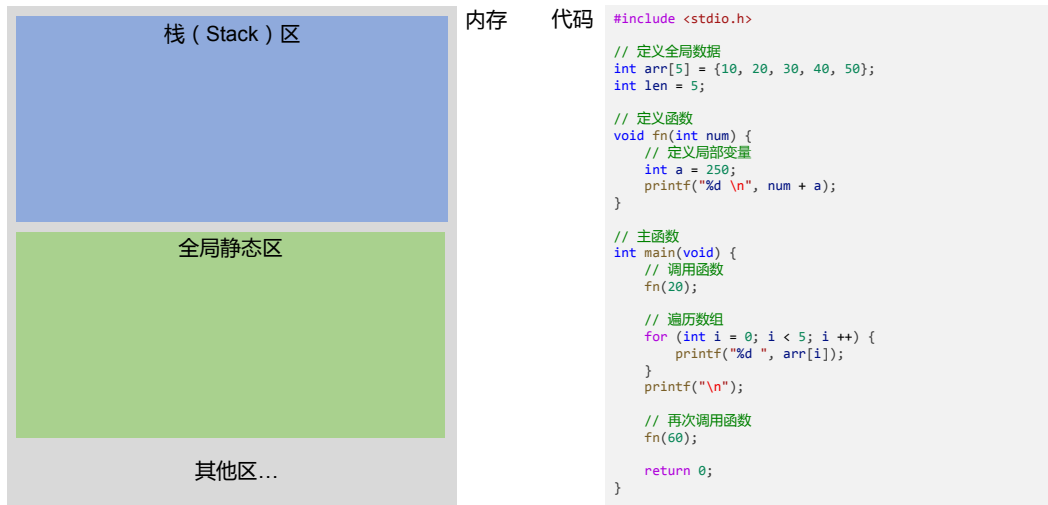
    // 再次调用函数
    fn(60);

    return 0;
}
```

输出结果:

```
270
10 20 30 40 50
310
```

内存分析:



11.5.6 作用域练习题

(1) 思考：下面的代码输出什么内容？

```

#include <stdio.h>

double price = 200.0;

void test01()
{
    printf("%.2f \n", price);
}

void test02()
{
    price = 250.0;
    printf("%.2f \n", price);
}

// main 函数
int main()
{
    printf("main price=%.2f \n", price);
    test01();
    test02();
    test01();

    return 0;
        
```

```
}
```

(2) 思考：下面的代码输出什么内容？

```
#include <stdio.h>

int n = 10;

void func1()
{
    int n = 20;
    printf("func1 n: %d\n", n);
}

void func2(int n)
{
    printf("func2 n: %d\n", n);
}

void func3()
{
    printf("func3 n: %d\n", n);
}

int main()
{
    int n = 30;

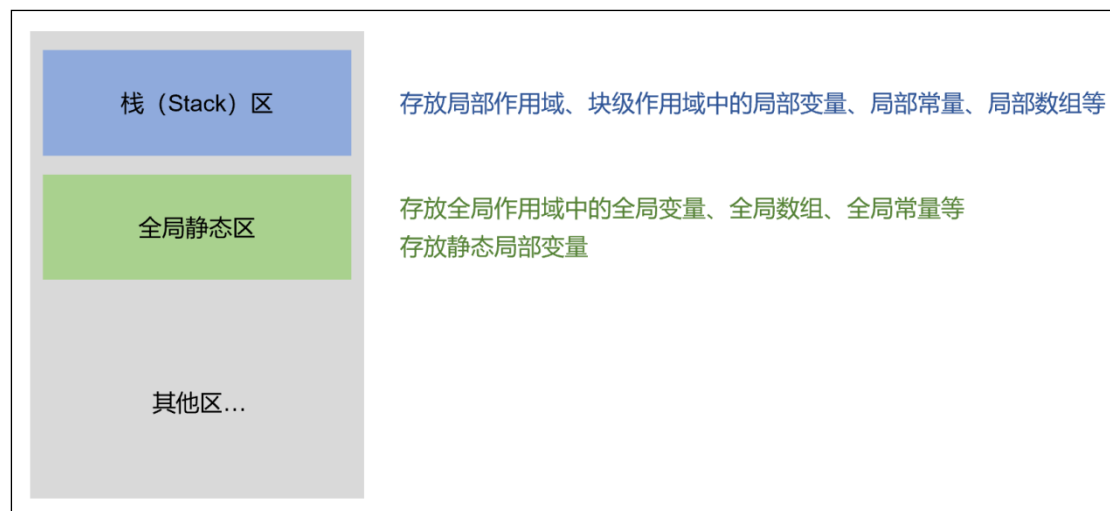
    func1();
    func2(n);
    func3();
    {
        int n = 40;
        printf("block n: %d\n", n);
    }
    printf("main n: %d\n", n);

    return 0;
}
```

11.6 static 和 extern 关键字

static 关键字可以声明静态变量和静态函数，以控制变量和函数的作用范围和生存周期。

11.6.1 静态局部变量



使用 `static` 关键字修饰的局部变量，称为静态局部变量，静态局部变量与全局变量一样存储在内存中的全局静态区。静态局部变量具有如下特点：

(1) 静态局部变量只在函数第一次调用时初始化一次并将生命周期延长到整个程序的执行期间。

(2) 静态局部变量如果声明时没有初始赋值，系统会自动初始化为零，同全局变量的初始化规则一致。

代码示例：

```
#include <stdio.h>

void fn(){
    int n = 10;
    int a;
    printf("n=%d, a=%d \n", n, a);
    n++;
    printf("n++=%d \n", n);
    printf("    \n");
}

void fn_static() {
    static int n = 10;
    static int a;
    printf("static n=%d, a=%d\n", n, a);
    n++;
    printf("static n++=%d \n", n);
    printf("    \n");
}
```

```

}

int main() {
    int a = 100;
    fn();
    fn_static();

    fn();
    fn_static();

    return 0;
}

```

输出结果:

```

n=10, a=16
n++=11

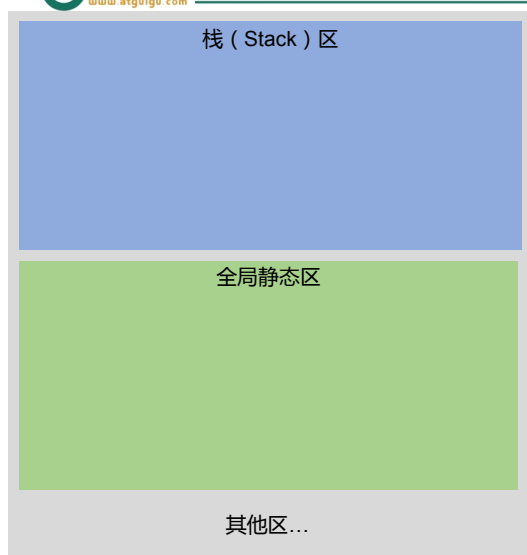
static n=10, a=0
static n++=11

n=10, a=0
n++=11

static n=11, a=0
static n++=12

```

内存分析:



```

#include <stdio.h>

void fn(void)
{
    int n = 10;
    int a;
    printf("n=%d, a=%d \n", n, a);
    n++;
    printf("n++=%d \n", n);
}

void fn_static(void)
{
    static int n = 10;
    static int a;
    printf("static n=%d, a=%d\n", n, a);
    n++;
    printf("static n++=%d \n", n);
}

int main(void)
{
    fn();
    fn_static();
    fn();
    fn_static();
    return 0;
}

```

11.6.2 多文件编译和静态全局变量

11.6.2.1 多文件编译

C 编译器可以将多个源文件编译成一个可执行文件。创建两个源文件，分别命名为 file01.c 和 file02.c，源文件代码如下：

【file01.c】

```
#include <stdio.h>

int num01 = 100;

const double PI01 = 3.14;

char msg01[] = "Hello msg01";

void fn01()
{
    printf("function fn01 \n");
}
```

【file02c】

```
#include <stdio.h>

int main()
{
    printf("Hello file02");
    return 0;
}
```

VS Code 默认是无法同时编译多个源文件的，我们使用命令行终端进行编译，打开 VS Code 内置终端，运行如下命令：

```
gcc file01.c file02.c -o main.exe
```

运行之后，会生产 main.exe 可执行文件，再次通过命令行运行 main.exe 即可。

```
./main.exe
```

运行完会显示“Hello file02”。

11.6.2.2 extern 关键字声明外部链接

下面我们演示如何在 file02.c 中使用 file01.c 中定义的全局变量，修改 file02.c，如下：

```
#include <stdio.h>

int main()
{
```

```
// 使用 file02.c 中定义的数据
printf("%d \n", num01);
printf("%f \n", PI01);
printf("%s \n", msg01);
fn01();

return 0;
}
```

重新使用命令编译：

```
gcc file01.c file02.c -o main.exe
```

会发现编译失败，这是因为默认无法使用外边文件中定义的变量。如果想要使用外部文件中定义的变量，我们可以使用 `extern` 关键字声明外部链接，我们再次修改 `file02.c`，如下：

```
#include <stdio.h>

// 外部声明 file01.c 中定义的全局变量
extern int num01;
extern const double PI01;
extern char msg01[];
extern void fn01();

int main()
{
    // 使用 file01.c 中定义的数据
    printf("%d \n", num01);
    printf("%f \n", PI01);
    printf("%s \n", msg01);
    fn01();

    return 0;
}
```

重新使用命令编译：

```
gcc file01.c file02.c -o main.exe
```

运行：

```
./main.exe
```

可以成功运行。

11.6.2.3 static 关键字声明静态全局变量

使用 `static` 关键字修饰的全局变量称为静态全局变量。

普通全局变量对整个工程可见，其他文件中，使用 `extern` 外部声明后，可以直接使用。

静态全局变量仅对当前文件可见，其他文件不可访问，其他文件中可以定义与其同名的变量，两者互不影响。

静态全局变量对于需要编译多个源代码文件的程序，能够有效地降低源文件之间的耦合，避免不同文件同名变量的冲突。

此外 static 关键字还可以修饰函数（静态函数）、全局数组、全局常量等，功能作用与静态全局变量一致。

修改 file01.c 和 file02.c，如下：

【file01c】

```
#include <stdio.h>

int num01 = 100;

// 声明为静态全局变量
static const double PI01 = 3.14;

char msg01[] = "Hello msg01";

// 声明为静态全局变量
static void fn01()
{
    printf("function fn01 \n");
}
```

【file02c】

```
#include <stdio.h>

// 外部声明 file01.c 中定义的全局变量
extern int num01;
extern const double PI01;
extern char msg01[];
extern void fn01();

int main()
{
    // 使用 file01.c 中定义的数据
    printf("%d \n", num01);
    printf("%s \n", msg01);

    // 对于静态全局变量，即使使用 extren 声明也无法使用
```

```
// fn01();  
// printf("%f \n", PI01);  
  
return 0;  
}
```

重新使用命令编译：

```
gcc file01.c file02.c -o main.exe
```

运行：

```
./main.exe
```

我们发现，file01.c 中定义的静态全局变量，在 file02.c 中即使使用 `extern` 声明也无法被使用。

11.7 递归函数

11.7.1 递归函数概念

一个函数在函数体内又调用了本身，我们称为递归调用，这样的函数就是递归函数。

递归函数成功执行需满足以下两个条件：

- (1) 必须有一个明显的结束条件。
- (2) 必须有一个趋近于结束条件的趋势。

11.7.2 递归函数原理分析

```
#include <stdio.h>  
  
void test(int n)  
{  
    printf("%d\n", n);  
  
    if (n > 1)  
    {  
        test(n - 1);  
    }  
  
    printf("%d\n", n);  
}  
  
int main()  
{  
    // 调用函数  
    test(3);  
}
```

```
return 0;
}
```

输出结果:

```
3
2
1
1
2
3
```

原理分析:

```
void test(int n) {
    printf("%d\n", n);
    if (n > 1) {
        test(n - 1);
    }
    printf("%d\n", n);
}

test(3);

输出: 3 2 1 1 2 3
```

test(3) 开始调用 输出3 3>1 成立, 递归调用, 本次调用挂起

test(2) 开始调用 输出2 2>1 成立, 递归调用, 本次调用挂起

test(1) 开始调用 输出1 1>1 不成立

test(1) 输出1 调用结束

test(2) 输出2 调用结束

test(3) 输出3 调用结束

11.7.3 递归函数案例

1) 案例一

1, 1, 2, 3, 5, 8, 13... 这是一个斐波那契数列, 它的规则是第 1 个和第 2 个斐波那契数是 1, 从第 3 个数开始, 每个斐波那契数都是前两个数之和。编写程序, 输入一个整数, 计算它的斐波那契数?

```
#include <stdio.h>

int fib(int n)
{
    if (n == 1 || n == 2)
    {
        return 1;
    }
    else
    {
```

```
        return fib(n - 1) + fib(n - 2);
    }
}

int main()
{
    // 定义变量，保存整数
    int num;
    // 获取用户输入
    printf("请输入一个整数: ");
    scanf("%d", &num);
    // 计算并输出
    printf("%d 的斐波那契数是: %d", num, fib(num));

    return 0;
}
```

2) 案例二

有一堆桃子，猴子第一天吃了其中的一半，并多吃一个。以后每天猴子都吃其中的一半，然后再多吃一个。当到第十天时，想再吃时（注意：此时还没吃），发现只有 1 个桃子了。问：最初共多少个桃子？

```
#include <stdio.h>

int peach(int day)
{
    if (day == 10)
    {
        return 1;
    }
    else
    {
        return (peach(day + 1) + 1) * 2;
    }
}

int main()
{
    printf("第十天桃子的数量: %d \n", peach(10));
    printf("第九天桃子的数量: %d \n", peach(9));
    printf("第一天桃子的数量: %d \n", peach(1));

    return 0;
}
```

输出结果：

```
第十天桃子的数量： 1
第九天桃子的数量： 4
第一天桃子的数量： 1534
```

11.8 常用系统函数

11.8.1 字符串相关函数

1) <string.h> 头文件中的函数

标准库的头文件 <string.h> 中，有三个常用的字符串函数：

函数名	描述
<code>strlen(str)</code>	返回 str 的长度，类型是 <code>size_t</code>
<code>strcpy(str1,str2)</code>	将 str2 中的字符串复制到 str1 中
<code>strcat(str1,str2)</code>	将 str2 中的字符串追加到 str1 后面

代码演示如下：

```
#include <stdio.h>
#include <string.h> // 该头文件中声明字符串相关的系统函数

int main()
{
    // 声明两个字符串
    char src[] = "abcdefg", dest[50] = "hello";

    // strlen 获取字符串长度
    printf("src 长度: %d \n", strlen(src));
    printf("dest 长度: %d \n", strlen(dest));
    printf("\n");

    // 将 "world" 拷贝到 src，拷贝字符串将原来的内容覆盖
    strcpy(src, " world");
    printf("src=%s \n", src);
    printf("\n");

    // strcat 是将 src 字符串的内容连接到 dest
    strcat(dest, src);
    printf("dest=%s", dest);

    return 0;
}
```

输出结果：

```
src 长度: 7
dest 长度: 5
```

```
src= world

dest=hello world
```

2) <stdio.h> 头文件中的函数

标准库的头文件 <stdio.h> 中，也有两个字符串函数：

`sprintf()`，用于将格式化数据写入字符串。相比于 `printf()`，多了一个参数，第一个参数是要写入的字符串，后面参数与 `printf()` 一致。简单地讲，`sprintf()` 是将内容写入字符串而不是输出。

`sscanf()`，用于从一个字符串中按照指定的格式提取数据。相比于 `scanf()`，多了一个参数，第一个参数是要提取数据的字符串，后面参数与 `scanf()` 一致。简单地讲，`sscanf()` 是从字符串中提取数据而不是从用户输入提取数据。

代码示例：

```
#include <stdio.h>

int main()
{
    // ----- sprintf 的使用 -----
    int age = 12;
    double score = 65.5;
    char name[] = "张三";

    char outputString[100]; // 字符数组，用于存储格式化后的字符串

    // 使用 sprintf 将数据格式化为字符串
    sprintf(outputString, "我叫%s,今年%d 岁,成绩%.2f 分", name, age,
score);

    // 输出格式化后的字符串
    printf("%s\n", outputString);

    // ----- sscanf 的使用 -----
    char inputString[] = "张三成绩:44,孙悟空成绩:65.5,张飞成绩:89";
    float score1, score2, score3;

    // 使用 sscanf 从字符串中提取数据
    sscanf(inputString, "张三成绩:%f,孙悟空成绩:%f,张飞成绩:%f", &score1,
&score2, &score3);

    // 输出提取的数据
```

```
printf("score1=%.2f, score2=%.2f, score3=%.2f", score1, score2,
score3);

return 0;
}
```

输出结果:

```
我叫张三, 今年 12 岁, 成绩 65.50 分
score1=44.00, score2=65.50, score3=89.00
```

11.8.2 日期时间相关函数

日期时间相关函数，在头文件 `<time.h>` 中。

函数名	描述
<code>time (&变量)</code>	获取当前日期赋值到变量中，该变量需是 <code>time_t</code> 类型
<code>ctime (&时间值)</code>	将时间值转为字符串并返回，时间值需是 <code>time_t</code> 类型
<code>difftime (时间值 1, 时间值 2)</code>	返回两个时间值的差，返回值是 <code>double</code> 类型，时间值需是 <code>time_t</code> 类型

注意:

`time_t` 是 C 语言中用于表示时间值的数据类型，它通常是一个整数类型（`int` 或 `long` 或 `long long`），用于存储时间戳。

时间戳是指从 1970 年 1 月 1 日（通常称为 UNIX 纪元）零时零分零秒（UTC 时间）起至某一特定时刻所经过的秒数。

案例演示:

```
#include <stdio.h>
#include <time.h> //该头文件中，声明和日期和时间相关的函数

int main()
{
    // time() 获取当前的时间
    time_t curr_time;
    time(&curr_time);
    printf("当前时间戳: %lld \n", curr_time);

    // ctime() 将时间值转为字符串
    printf("当前时间字符串: %s \n", ctime(&curr_time));

    return 0;
}
```

输出结果:

```
当前时间戳: 1697221633
当前时间字符串: Sat Oct 14 02:27:13 2023
```

接下来，我们使用 `difftime()` 函数来统计程序的执行时间，代码如下：

```
#include <stdio.h>
#include <time.h> // 该头文件中，声明和日期和时间相关的函数

int fn()
{
    int sum = 0;
    for (int i = 0; i < 10000000; i++)
    {
        sum = 0;
        for (int j = 1; j <= 100; j++)
        {
            sum += j;
        }
    }
    return sum;
}

int main()
{
    time_t start_t, end_t;
    time_t diff_t; // 存放时间差

    time(&start_t); // 得到起始时间
    fn();           // 运行 fn 函数，看看执行花费时间
    time(&end_t);   // 函数执行结束后得到时间

    // 计算两个时间值的差
    diff_t = difftime(end_t, start_t);
    printf("执行 fn() 函数耗用了 %lld 秒", diff_t);

    return 0;
}
```

输出结果：

执行 fn() 函数耗用了 2 秒

11.8.3 数学相关函数

数学相关函数，在头文件 `<math.h>` 中。

函数名	描述
<code>sqrt(x)</code>	计算平方根
<code>cbrt(x)</code>	计算立方根
<code>pow(x,y)</code>	计算 x 的 y 次方

fabs(x)	计算 x 的绝对值
ceil(x)	向上取整
floor(x)	向下取整
round(x)	四舍五入取整
trunc(c)	截断小数部分

注意：以上函数的参数与返回值的类型都是 `double` 类型。

案例演示：

```
#include <stdio.h>
#include <math.h>

int main()
{
    printf("10 的平方根: %f \n", sqrt(10));
    printf("8 的立方根: %.f \n", cbrt(8));
    printf("3 的 12 次方: %.f \n", pow(3, 12));
    printf("12.31 向上取整: %.f, -12.31 向上取整: %.f \n", ceil(12.31),
    ceil(-12.31));
    printf("12.51 向下取整: %.f, -12.51 向下取整: %.f \n", floor(12.51),
    floor(-12.51));
    printf("12.51 四舍五入: %.f, -12.51 四舍五入: %.f \n", round(12.51),
    round(-12.51));
    printf("12.51 截断小数: %.f, -12.51 截断小数: %.f \n", trunc(12.51),
    trunc(-12.51));

    return 0;
}
```

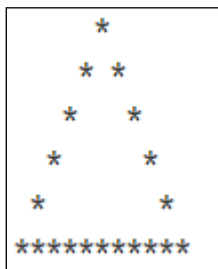
输出结果：

```
2 的绝对值: 2, -12.34 的绝对值: 12.34
10 的平方根: 3.162278
8 的立方方根: 2
3 的 12 次方: 531441
12.31 向上取整: 13, -12.31 向上取整: -12
12.51 向下取整: 12, -12.51 向下取整: -13
12.51 四舍五入: 13, -12.51 四舍五入: -13
12.51 截断小数: 12, -12.51 截断小数: -12
```

11.9 课后练习题

- （1）定义函数，实现求两个 `double` 数字的最大值，并返回。
- （2）定义函数，求出三个 `int` 类型数的和，并返回。
- （3）定义函数，判断一个数字是否是质数。
- （4）函数可以没有返回值案例，编写一个函数，从终端输入一个整数打印出对应的金

字塔，函数可以传入金字塔层数。



第 12 章 指针

12.1 指针的理解

12.1.1 变量的访问方式

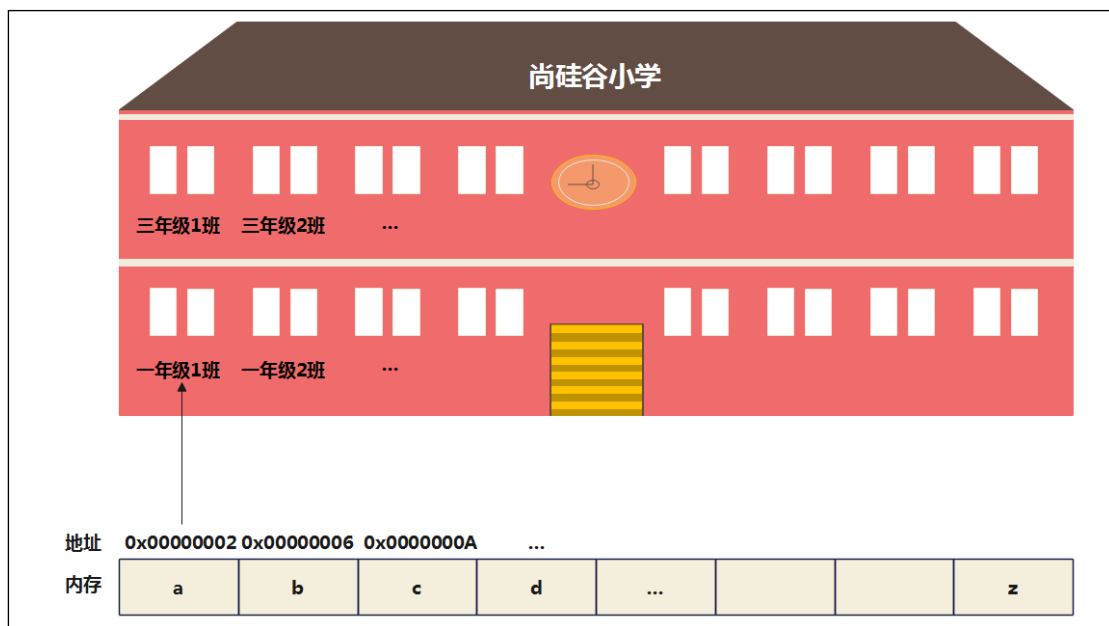
内存是电脑上特别重要的存储器，计算机中程序的运行都是在内存中进行的，为了有效的使用内存，就把内存划分成一个个小的内存单元，每个内存单元通常占用 1 个字节。

变量在内存中分配空间，不同类型的变量占用不同大小的空间，那如何访问内存中的变量数据呢？有两种方式：

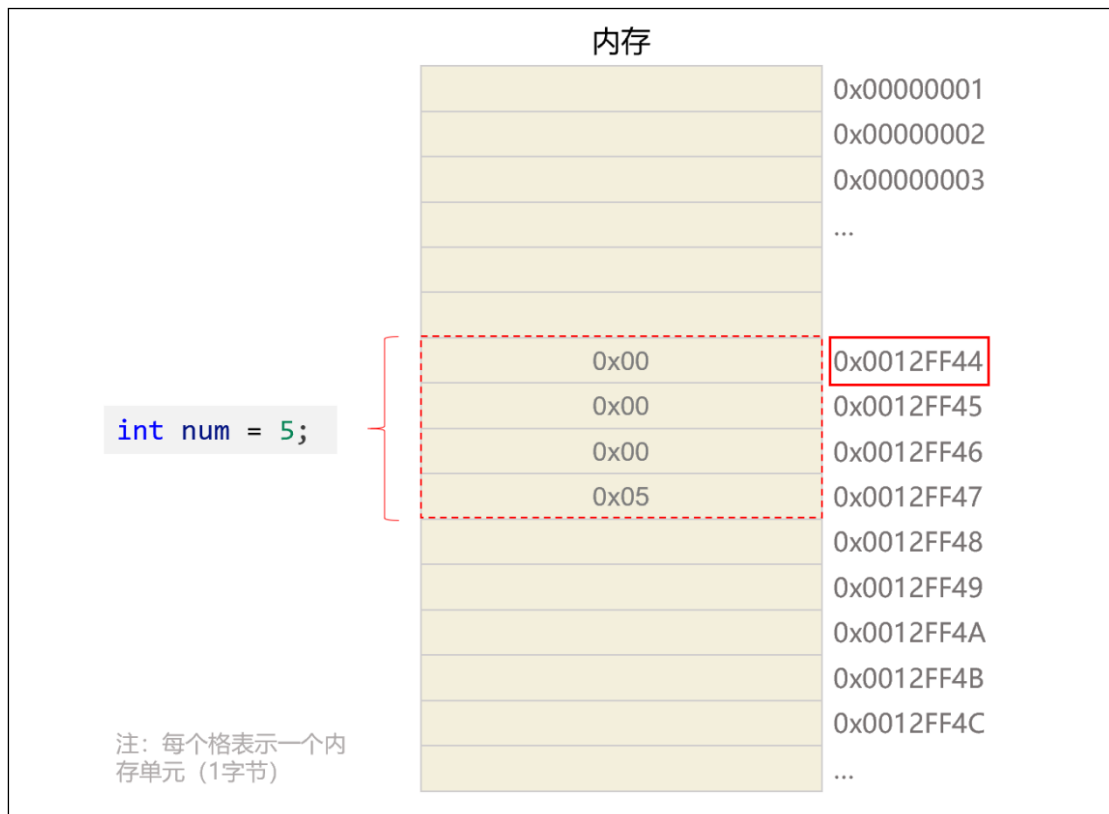
- (1) 直接访问，直接使用变量名进行的访问，以前的程序中都是采用这种方式。
- (2) 间接访问，通过指针来实现。

12.1.2 内存地址

为了能够有效地访问到每个内存单元，就给内存单元进行了编号，这些编号被称为内存地址，因为每个内存单元都有地址，所以变量也是有地址的。



假设有 `int` 型变量 `num`，其在内存中会占用 4 个字节，也就是占用 4 个内存单元，第一个内存单元的地址即是变量 `num` 的地址。如图：



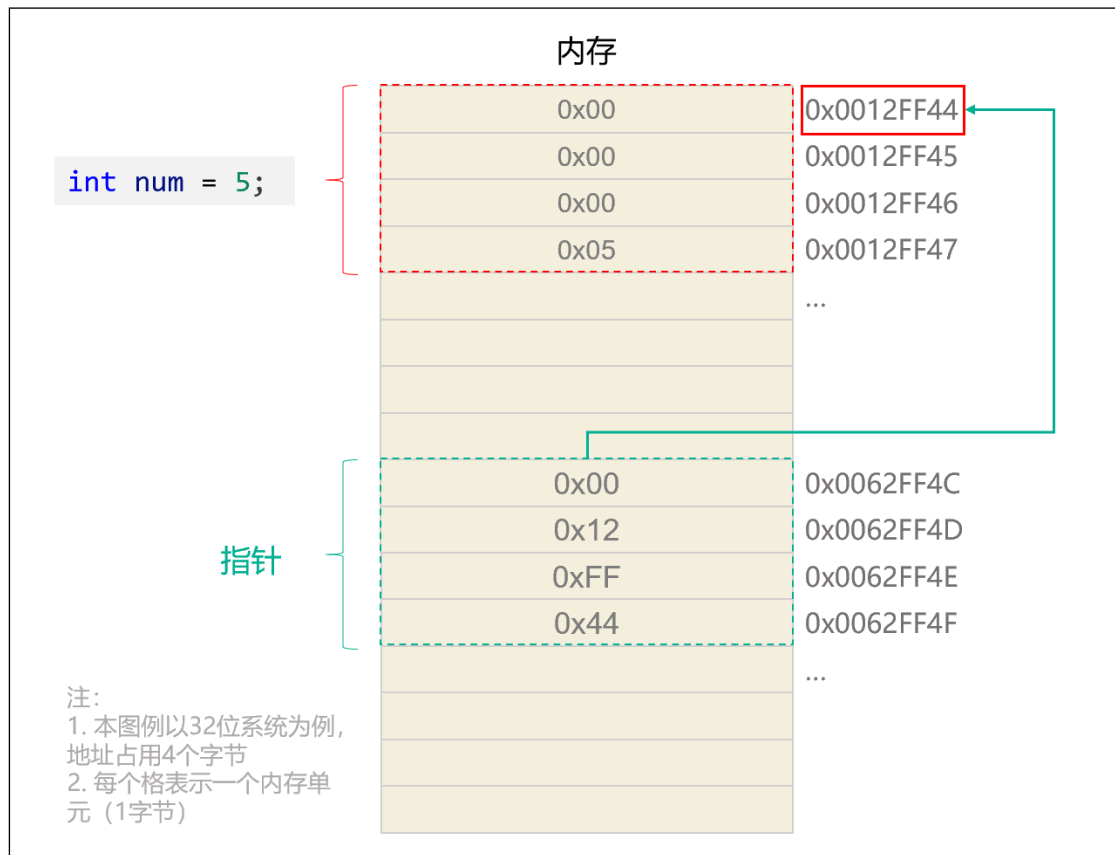
在 32 位系统中，内存地址通常是 32 位二进制数字，即 4 个字节，这允许寻址 2^{32} （大约 4GB）个内存位置。

在 64 位系统中，内存地址通常是 64 位二进制数字，即 8 个字节，这允许寻址 2^{64} 个

内存位置，这个数量相当大，远远超过了通常需要的内存大小。

12.1.3 什么是指针

如果一个变量专门用来存放内存地址，则它称为**指针变量**，通常简称为**指针**。我们可以通过指针间接访问内存中另一个数据。



如图，指针里面存储的是变量 `num` 的地址，我们可以说该指针指向变量 `num`，通过该指针可以间接访问变量 `num`。

12.1.4 指针的定义

一般格式：

```
数据类型 *指针变量名 [=初始地址值];
```

数据类型是指针所指向的地址处的数据类型，如 `int`、`char`、`float` 等。

符号 `*` 用于通知系统，这里定义的是一个指针变量，通常跟在类型关键字的后面，表示指针指向的是什么类型的值。比如，`char *` 表示一个指向字符的指针，`float *` 表示一个指向浮点数的指针。

需要注意的是，以下三种定义指针的写法都是合法的：

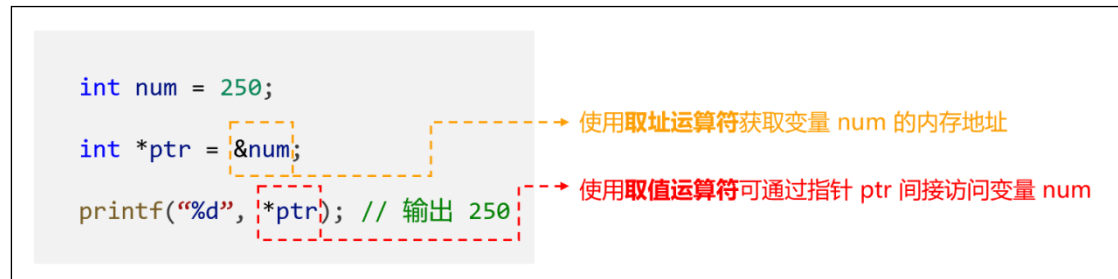
```
int *ptr;
```

```
int* ptr;
int * ptr;
```

12.1.5 取址运算符和取值运算符

取址运算符，使用 `&` 符号表示，作用是取出变量的内存地址。如果要格式化输出地址，需使用格式占位符 `%p`。

取值运算符，使用 `*` 符号表示，作用是取出指针指向的内存地址处的数据值，也称为解引用运算符或间接引用运算符。



```
int num = 250;
int *ptr = &num;
printf(\"%d\", *ptr); // 输出 250
```

使用取址运算符获取变量 `num` 的内存地址

使用取值运算符可通过指针 `ptr` 间接访问变量 `num`

12.1.6 指针应用案例

1) 案例 1

创建一个 `int` 类型的变量，使用取址运算符取出其地址，并将其地址赋值给一个指针，然后分别打印变量的值、变量的地址、指针的值、指针的地址、指针指向的值。

代码如下：

```
#include <stdio.h>

int main()
{
    // 定义变量
    int num = 100;

    // 定义一个指针，指向变量 num 的地址
    // 这里使用 & 符号取出变量 num 的地址，再赋值给指针变量 ptr
    int *ptr = &num;

    // 打印变量 num 的值和地址
    printf("num 的值是 %d \n", num);
    printf("num 的地址是 %p \n\n", &num);

    // 打印 ptr 的值、ptr 的地址、ptr 指向的值
    printf("ptr 的值是 %p \n", ptr);    // 指针 ptr 的值与变量 num 的值是相同的
    printf("ptr 的地址是 %p \n", &ptr);
```

```
printf("ptr 指向的值是 %d \n", *ptr); // 指针 ptr 指向的值就是变量
num 的值

return 0;
}
```

输出结果:

```
num 的值是 100
num 的地址是 000000412d3ffdcc

ptr 的值是 000000412d3ffdcc
ptr 的地址是 000000412d3ffdc0
ptr 指向的值是 100
```

内存示意图:

代码

内存

```
int num = 100;
int *ptr = &num;

// 打印变量 num 的值和地址
printf("num 的值是 %d \n", num);
printf("num 的地址是 %p \n\n", &num);

// 打印 ptr 的值、ptr 的地址、ptr 指向的值
printf("ptr 的值是 %p \n", ptr);
printf("ptr 的地址是 %p \n", &ptr);
printf("ptr 指向的值是 %d \n", *ptr);
```

1) 案例二

通过指针修改所指向的值，代码如下：

```
#include <stdio.h>

int main()
{
    // 创建 double 类型的变量
    double num = 2.88;

    // 创建指针 p1, 指向变量 num
    double *p1 = &num;
```

```
// 创建指针 p2, 将 p1 的值赋值给 p2, p1 存储的是 num 的值, 此时 p1 和 p2
// 都指向 num
double *p2 = p1;

// 打印 num 的值
printf("%.2f \n", num); // 2.88

// 通过指针 p1 修改 num 的值, 并打印
*p1 = 3.88;
printf("%.2f \n", num); // 3.88

// 通过指针 p2 修改 num 的值, 并打印
*p2 += 10;
printf("%.2f \n", num); // 13.88

return 0;
}
```

输出结果:

```
2.88
3.88
13.88
```

内存示意图:



指针应用案例: 通过指针修改所指向的值

让天下没有难学的技术

代码 内存

```
// 创建 double 类型的变量
double num = 2.88;

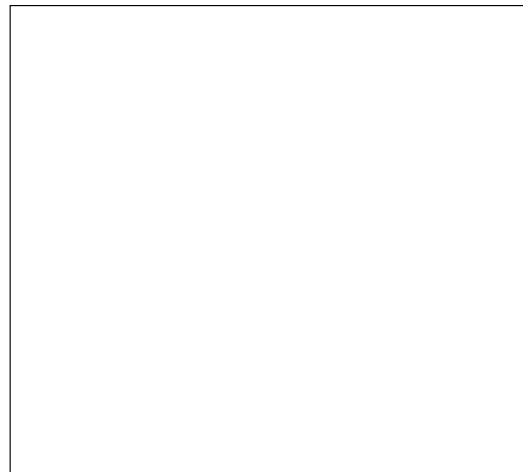
// 创建指针 p1, 指向变量 num
double *p1 = &num;

// 创建指针 p2, 将 p1 的值赋值给 p2
double *p2 = p1;

// 打印 num 的值
printf("%.2f \n", num);

// 通过指针 p1 修改 num 的值, 并打印
*p1 = 3.88;
printf("%.2f \n", num);

// 通过指针 p2 修改 num 的值, 并打印
*p2 += 10;
printf("%.2f \n", num);
```



12.2 指针运算

12.2.1 指针加减整数

指针与整数的加减运算, 表示指针所指向的内存地址的移动 (加, 向后移动; 减, 向前

移动)，指针移动多少，与指针指向的数据类型有关，数据类型占据多少个字节，每单位就移动多少个字节，比如一个 `int` 类型指针，+1 向后移动 4 个字节，-2 向前移动 8 个字节。

数组的元素在内存中连续存储的，我们通过数组元素来演示指针加减整数的情况。

代码如下：

```
#include <stdio.h>

int main()
{
    // 创建数组
    int nums[] = {10, 20, 30, 40, 50};

    // 创建指针并指向数组第一个元素的地址
    int *ptr = &nums[0];

    // 打印指针的值和指向的值
    printf("ptr=%p, *ptr=%d \n", ptr, *ptr);

    // 指针加 3，指针指向 int 类型，每个占 4 个字节，此时指针会向后移动 12 个字节
    ptr += 3;
    printf("ptr=%p, *ptr=%d \n", ptr, *ptr);

    // 指针-2，此时指针会向前移动 8 个字节
    ptr -= 2;
    printf("ptr=%p, *ptr=%d \n", ptr, *ptr);

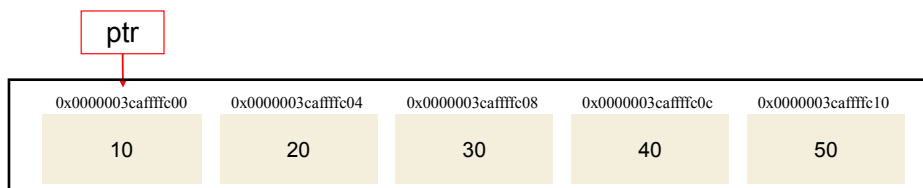
    return 0;
}
```

输出结果：

```
ptr=000000000061FE00, *ptr=10
ptr=000000000061FE0C, *ptr=40
ptr=000000000061FE04, *ptr=20
```

内存示意图：

```
int nums[] = {10, 20, 30, 40, 50}; // 创建数组
int *ptr = &nums[0]; // 创建指针并指向数组第一个元素的地址
printf("ptr=%p, *ptr=%d \n", ptr, *ptr);           ptr=0000003caffffc00, *ptr=10
ptr += 3; // ptr = ptr + 3
printf("ptr=%p, *ptr=%d \n", ptr, *ptr);           ptr=0000003caffffc0c, *ptr=40
ptr -= 2; // ptr = ptr - 2
printf("ptr=%p, *ptr=%d \n", ptr, *ptr);           ptr=0000003caffffc04, *ptr=20
```



12.2.2 指针自增自减

指针自增、自减本质上就是指针加减整数，自增地址后移，自减地址前移。下面我们利用指针的自增自减实现数组的遍历，代码如下：

```
#include <stdio.h>

int main()
{
    // 创建数组，元素都是 short 类型，占据 2 个字节
    short nums[] = {10, 20, 30, 40, 50};
    // 定义常量记录数组长度
    const int len = 5;

    // 利用指针自增遍历数组元素
    // 创建指针并指向数组第一个元素的地址
    short *ptr = &nums[0];
    // 循环
    for (int i = 0; i < len; i++)
    {
        printf("元素索引: %d, 元素地址: %p, 元素值: %d \n", i, ptr, *ptr);
        ptr++; // 指针自增
    }
    printf("\n");

    // 利用指针自减再次遍历数组
    // 此时指针超出数组界限，需先自减一次
    for (int i = len - 1; i >= 0; i--)
```

```
{
    ptr--; // 指针自减
    printf("元素索引: %d, 元素地址: %p, 元素值: %d \n", i, ptr, *ptr);
}

return 0;
}
```

输出结果:

元素索引: 0, 元素地址: 0000000e1efffe02, 元素值: 10
元素索引: 1, 元素地址: 0000000e1efffe04, 元素值: 20
元素索引: 2, 元素地址: 0000000e1efffe06, 元素值: 30
元素索引: 3, 元素地址: 0000000e1efffe08, 元素值: 40
元素索引: 4, 元素地址: 0000000e1efffe0a, 元素值: 50

元素索引: 4, 元素地址: 0000000e1efffe0a, 元素值: 50
元素索引: 3, 元素地址: 0000000e1efffe08, 元素值: 40
元素索引: 2, 元素地址: 0000000e1efffe06, 元素值: 30
元素索引: 1, 元素地址: 0000000e1efffe04, 元素值: 20
元素索引: 0, 元素地址: 0000000e1efffe02, 元素值: 10

内存示意图:



案例演示: 利用指针自增自减实现数组遍历

让天下没有难学的技术

```
short nums[] = {10, 20, 30, 40, 50}; // 创建数组, 元素都是short类型, 每个占2个字节
```

```
short *ptr = &nums[0]; // 创建指针并指向数组第一个元素的地址
```

```
// 利用指针自增遍历数组元素
for (int i = 0; i < 5; i++)
{
    printf("元素索引: %d, 元素地址: %p, 元素值: %d \n", i, ptr, *ptr);
    ptr++; // 指针自增
}
```

元素索引: 0, 元素地址: 0000000e1efffe02, 元素值: 10
元素索引: 1, 元素地址: 0000000e1efffe04, 元素值: 20
元素索引: 2, 元素地址: 0000000e1efffe06, 元素值: 30
元素索引: 3, 元素地址: 0000000e1efffe08, 元素值: 40
元素索引: 4, 元素地址: 0000000e1efffe0a, 元素值: 50

```
// 利用指针自减再次遍历数组
for (int i = 4; i >= 0; i--)
{
    ptr--; // 指针自减, 此时指针超出数组界限, 需先自减一次
    printf("元素索引: %d, 元素地址: %p, 元素值: %d \n", i, ptr, *ptr);
}
```

元素索引: 4, 元素地址: 0000000e1efffe0a, 元素值: 50
元素索引: 3, 元素地址: 0000000e1efffe08, 元素值: 40
元素索引: 2, 元素地址: 0000000e1efffe06, 元素值: 30
元素索引: 1, 元素地址: 0000000e1efffe04, 元素值: 20
元素索引: 0, 元素地址: 0000000e1efffe02, 元素值: 10

		ptr				
		↓				
地址		0x0000000e1efffe02	0x0000000e1efffe04	0x0000000e1efffe06	0x0000000e1efffe08	0x0000000e1efffe0a
值		10	20	30	40	50
索引		0	1	2	3	4

数组越界
遍历结束

12.2.3 同类型指针相减

相同类型的指针可以进行减法运算, 返回它们之间的距离, 即相隔多少个数据单位。高位地址减去低位地址, 返回的是正值; 低位地址减去高位地址, 返回的是负值。

同类型指针相减的结果是一个 ptrdiff_t 类型数据, ptrdiff_t 类型是一个带符号的整数, 格式输出中对应的格式占位符是 %td, 相关案例如下:

```
#include <stdio.h>

int main()
{
    // 创建数组
    int nums[] = {10, 20, 30, 40, 50};

    // 创建指针并指向数组第一个元素的地址
    int *ptr1 = &nums[0];
    // 创建指针并指向数组第四个元素的地址
    int *ptr2 = &nums[3];

    printf("ptr2-ptr1=%td \n", ptr2 - ptr1); // ptr2-ptr1=3
    printf("ptr1-ptr2=%td \n", ptr1 - ptr2); // ptr1-ptr2=-3

    // 再创建两个指针进行相减
    double d1 = 1.0;
    double d2 = 2.0;
    double *p1 = &d1, *p2 = &d2;
    printf("p1-p2=%td \n", p1 - p2); // p1-p2=1
    printf("p2-p1=%td \n", p2 - p1); // p2-p1=-1

    return 0;
}
```

输出结果:

```
ptr2-ptr1=3
ptr1-ptr2=-3
p1-p2=1
p2-p1=-1
```

内存示意图:

```
// 创建数组
int nums[] = {10, 20, 30, 40, 50};

// 创建指针并指向数组第一个元素的地址
int *ptr1 = &nums[0];
// 创建指针并指向数组第四个元素的地址
int *ptr2 = &nums[3];

printf("ptr2-ptr1=%td \n", ptr2 - ptr1);
printf("ptr1-ptr2=%td \n", ptr1 - ptr2);
```

ptr2 - ptr1 = 3

ptr1 - ptr2 = -3

	ptr1			ptr2	
地址	0x0000000e1efffe02	0x0000000e1efffe06	0x0000000e1efffe0a	0x0000000e1efffe0e	0x0000000e1efffe12
值	10	20	30	40	50
索引	0	1	2	3	4

12.2.4 指针的比较运算

指针之间可以进行比较运算，如 ==、<、<=、>、>=，比较的是各自指向的内存地址的大小，返回值是 int 类型整数 1（true）或 0（false）。案例演示如下：

```
#include <stdio.h>

int main()
{
    // 创建数组
    int nums[] = {10, 20, 30, 40, 50};
    double n = 1.0;

    // 创建指针并指向数组第一个元素的地址
    int *ptr1 = &nums[0];
    // 创建指针并指向数组第四个元素的地址
    int *ptr2 = &nums[3];
    // 创建指针也向数组第一个元素的地址
    int *ptr3 = &nums[0];
    // 创建指针指向变量 n 的地址
    double *ptr4 = &n;

    // 输出指针指向的地址
    printf("ptr1=%p\n", ptr1);
    printf("ptr2=%p\n", ptr2);
    printf("ptr3=%p\n", ptr3);
    printf("ptr4=%p\n\n", ptr4);
```

```
// 进行比较
printf("ptr1>ptr2: %d \n", ptr1 > ptr2);
printf("ptr1<ptr2: %d \n", ptr1 < ptr2);
printf("ptr1==ptr3: %d \n", ptr1 == ptr3);
printf("ptr4>ptr1: %d \n", ptr4 > ptr1); // 由于是不同类型的指针进行比较, 所以会有一个警告

return 0;
}
```

输出结果:

```
ptr1=000000d4385ffd10
ptr2=000000d4385ffd1c
ptr3=000000d4385ffd10
ptr4=000000d4385ffd08

ptr1>ptr2: 0
ptr1<ptr2: 1
ptr1==ptr3: 1
ptr4>ptr1: 0
```

内存示意图:



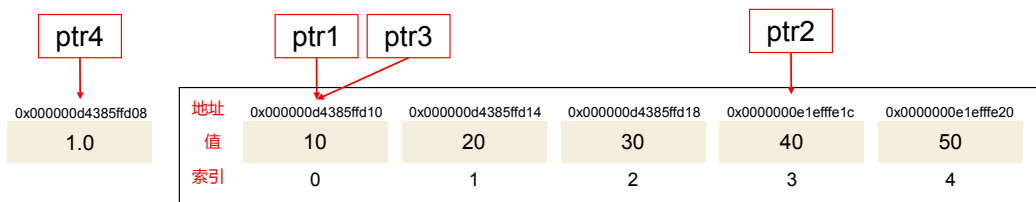
案例演示: 指针比较运算

让天下没有难学的技术

```
// 创建数组
int nums[] = {10, 20, 30, 40, 50};
double n = 1.0;

// 创建指针并指向数组第一个元素的地址
int *ptr1 = &nums[0];
// 创建指针并指向数组第四个元素的地址
int *ptr2 = &nums[3];
// 创建指针也向数组第一个元素的地址
int *ptr3 = &nums[0];
// 创建指针指向变量n的地址
double *ptr4 = &n;
```

```
printf("ptr1>ptr2: %d \n", ptr1 > ptr2);    0
printf("ptr1<ptr2: %d \n", ptr1 < ptr2);    1
printf("ptr1==ptr3: %d \n", ptr1 == ptr3);  1
printf("ptr4>ptr1: %d \n", ptr4 > ptr1);    0
```



12.3 指针和数组

12.3.1 数组名

数组名在大多数情况下会被隐式地转换为指向数组第一个元素的指针, 在特定情况下数组名可以被视为一个指针, 具有一些指针的特性。

但是数组名与真正的指针是不同的, 主要有以下几点区别:

- (1) 使用 `sizeof` 运算符，数组名得到的是整个数组的大小；指针得到的是本身的大小。
- (2) 数组名不能进行自增、自减运算。
- (3) 数组名的指向不可更改。

示例代码：

```
#include <stdio.h>

int main()
{
    // 创建数组
    int nums[5] = {10, 20, 30, 40, 50};
    // 创建指针指向数组的第一个元素
    int *ptr = &nums[0];

    // 数组名中存储了第一个元素的地址
    printf("%p, %d \n", nums, *nums);
    printf("%p, %d \n", ptr, *ptr);
    // nums 和 ptr 比较
    if (nums == ptr)
    {
        printf("nums 和 ptr 相等 \n\n");
    }
    else
    {
        printf("nums 和 ptr 不相等 \n\n");
    }

    // 数组名与真正的指针存在差别
    // 1. sizeof 运算符返回的是整个数组的大小，而指针返回的是本身的大小
    printf("%zu, %zu \n", sizeof nums, sizeof ptr);

    // 2. 数组名不能进行自增、自减运算
    // nums ++;    // 报错
    ptr++; // 地址指向下一个元素
    printf("%p, %d \n", ptr, *ptr);

    // 3. 数组名的指向不可修改
    int n = 100;
    // nums = &n;  // 报错
    ptr = &n;
    printf("%p, %d \n", ptr, *ptr);

    return 0;
}
```

}

输出结果:

```
00000000cdb5ffe60, 10
00000000cdb5ffe60, 10
nums 和 ptr 相等

20, 8
00000000cdb5ffe64, 20
00000000cdb5ffe5c, 100
```

内存分析:



案例演示: 数组指针

让天下没有难学的技术

```
// 创建数组
int nums[5] = {10, 20, 30, 40, 50};
// 创建指针指向数组的第一个元素
int *ptr = &nums[0];
```

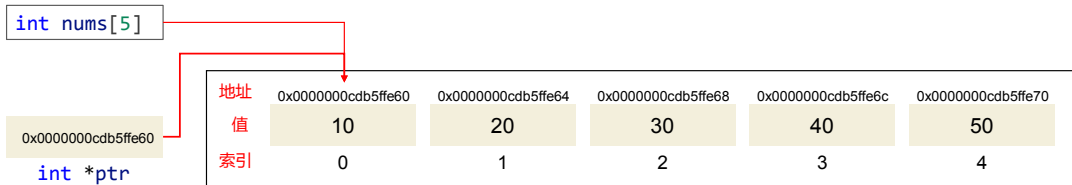
```
printf("%p, %d \n", nums, *nums);
printf("%p, %d \n", ptr, *ptr);
```

```
// nums 和 ptr 比较
if (nums == ptr)
{
    printf("nums 和 ptr 相等 \n\n");
}
else
{
    printf("nums 和 ptr 不相等 \n\n");
}
```

```
// 1. sizeof 运算符返回的是整个数组的大小, 而指针返回的是本身的大小
printf("%zu, %zu \n", sizeof nums, sizeof ptr); // 20, 8
```

```
// 2. 数组名不能进行自增、自减运算
// nums++; // 报错
ptr++; // 地址指向下一个元素
printf("%p, %d \n", ptr, *ptr);
```

```
// 3. 数组名的指向不可修改
int n = 100;
// nums = &n; // 报错
ptr = &n;
printf("%p, %d \n", ptr, *ptr);
```



12.3.2 指针数组

指针数组 (Pointer Array) 是一个数组, 其中的每个元素都是指针。

语法规则:

数据类型 *指针数组名[长度];

示例代码:

```
#include <stdio.h>

int main()
{
    // 创建三个变量
    int num1 = 10, num2 = 20, num3 = 30;

    // 创建一个长度为 3 的指针数组
    int *ptrArr[3];

    // 指针数组的每个元素指向不同的整数
```

```
ptrArr[0] = &num1;
ptrArr[1] = &num2;
ptrArr[2] = &num3;

// 遍历指针数组
for (int i = 0; i < 3; i++)
{
    printf("%d, %p, %d \n", i, ptrArr[i], *ptrArr[i]);
}

return 0;
}
```

输出结果:

```
0, 0000009c22fff898, 10
1, 0000009c22fff894, 20
2, 0000009c22fff890, 30
```

内存示意图:



案例演示: 指针数组

让天下没有难学的技术

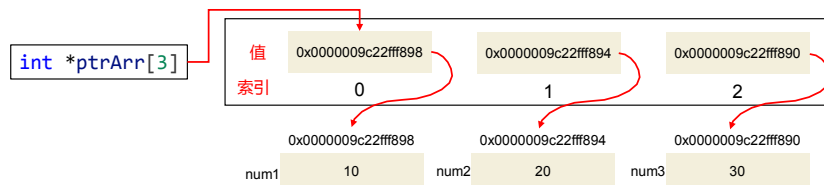
```
// 创建三个变量
int num1 = 10, num2 = 20, num3 = 30;
```

```
// 创建一个长度为3的指针数组
int *ptrArr[3];
```

```
// 指针数组的每个元素指向不同的整数
ptrArr[0] = &num1;
ptrArr[1] = &num2;
ptrArr[2] = &num3;
```

```
// 遍历指针数组
for (int i = 0; i < 3; i++)
{
    printf("%d, %p, %d \n", i, ptrArr[i], *ptrArr[i]);
}
```

```
0, 0000009c22fff898, 10
1, 0000009c22fff894, 20
2, 0000009c22fff890, 30
```



12.3.3 数组指针

数组指针 (Array Pointer) 是一个指针, 它指向一个数组。注意, 数组指针指向的是整个数组的地址而不是第一个元素的地址, 虽然二者值是相同的, 但在运算中会表现出不同。

语法规则:

数据类型 (*数组指针名)[长度];

示例代码:

```
#include <stdio.h>
```

```
int main()
{
    // 创建整数数组
    int arr[5] = {10, 20, 30, 40, 50};

    // 创建指针指向数组 arr  &arr 表示整个数组的地址
    int(*ptr)[5] = &arr;
    // int (*ptr)[5] = arr;    // 会有警告, arr 的类是 int *, ptr 类型是
int (*)[5]

    // 二者值是相同的
    printf("arr=%p \n", arr);
    printf("ptr=%p \n\n", ptr);

    // 数组指针指向的是数组的地址, 而不是第一个元素的地址
    // 数组指针+1 会向后移动 4*5=20 个字节; 数组+1 会向后移动 4 个字节
    printf("arr+1=%p \n", arr + 1);
    printf("ptr+1=%p \n\n", ptr + 1);

    // 使用数组指针遍历数组
    for (int i = 0; i < 5; i++)
    {
        printf("%d \n", (*ptr)[i]);
    }

    return 0;
}
```

输出结果:

```
arr=000000f6d35ff9c0
ptr=000000f6d35ff9c0

arr+1=000000f6d35ff9c4
ptr+1=000000f6d35ff9d4

10
20
30
40
50
```

内存示意图:

```
// 创建整数组  
int arr[5] = {10, 20, 30, 40, 50};
```

```
// 创建指针指向数组arr &arr表示整个数组的地址  
int (*ptr)[5] = &arr;
```

```
// int (*ptr)[5] = arr; // 会有警告, arr 的类是 int *, ptr 类型是  
// int (*)[5]
```

```
// 数组名的值和数值指针的值是相同的  
printf("arr=%p \n", arr); 输出: arr=0x000000f6d35ff9c0  
printf("ptr=%p \n", ptr); 输出: ptr=0x000000f6d35ff9c0
```

```
// 数组指针指向的是数组的地址, 而不是第一个元素的地址  
// 数组指针+1会向后移动4*5=20个字节; 数组+1会向后移动4个字节  
printf("arr+1=%p \n", arr + 1); 输出: arr+1=0x000000f6d35ff9c4  
printf("ptr+1=%p \n", ptr + 1); 输出: ptr+1=0x000000f6d35ff9d4
```

```
// 使用数组指针遍历数组  
for (int i = 0; i < 5; i++)  
{  
    printf("%d \n", (*ptr)[i]);  
}
```

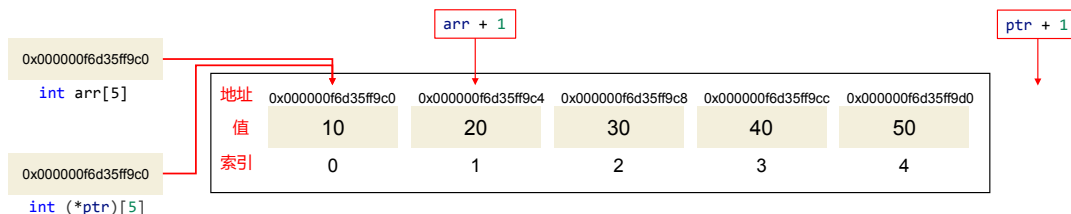
输出: 10

20

30

40

50



数组指针和数组名的区别:

- (1) 指向不同: 数组名指向元素首地址, 数组指针指向数组的地址。
- (2) 类型不同: 上面案例中, `arr` 的类型是 `int[5]`; `ptr` 的类型是 `int(*)[5]`。
- (3) 可变性: 数组名通常是不可变的; 数组指针是可变的, 你可以将它指向不同的数组。
- (4) 初始化: 数组名不需要显式初始化, 它会自动指向数组的首元素; 数组指针需要显式初始化, 指定它将指向的数组。
- (5) 访问元素: 数组名访问数组元素不需要解引用; 数组指针通常需要解引用才能访问数组元素。

12.3.4 字符指针

1) 基本介绍

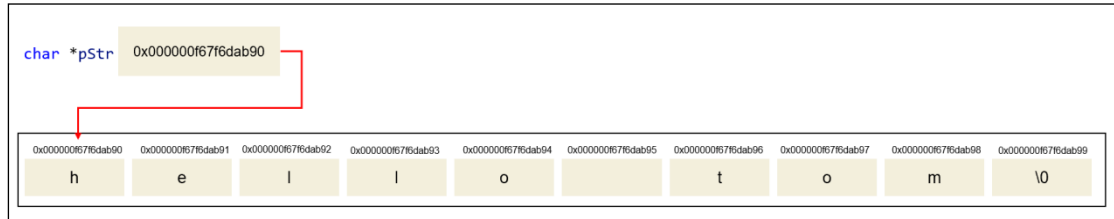
字符指针变量 (简称字符指针) 是 C 语言中的一种指针类型, 它用于指向字符或字符串 (字符数组), 通常用于处理字符串 (字符数组)。

2) 用字符指针指向一个字符串

```
char *pStr= "hello tom";
```

C 语言对字符串 "hello tom" 是按字符数组处理的, 在内存中开辟了一个字符数组用来存放字符串, 程序在定义字符串指针 `pStr` 时只是把字符串首地址 (即存放字符串的字符数组的首地址) 赋给 `pStr`。

对应的内存布局图：



3) 字符数组名和字符指针表示字符串的区别

(1) 对字符数组只能对各个元素赋值，不能对字符数组名重新赋值。

```
char str[14];
str="hello tom"; //错误
str[0] = 'i'; //ok
```

(2) 字符指针是可变的，允许使用下面方法重新赋值，指向新的字符串。

```
char *a = "hello tom";
a = "tom";
```

内存示意图：

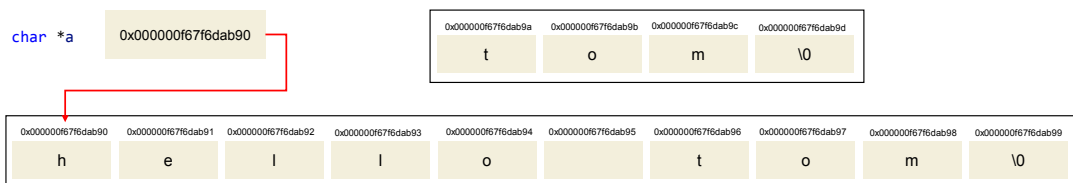


案例演示：字符指针

让天下没有难学的技术

```
char *a = "hello tom";
```

```
a = "tom";
```



12.4 指针和函数

12.4.1 传递指针给函数

当函数的形参类型是指针类型时，使用该函数时，需要传递指针、地址或者数组给该形参。

1) 传地址或指针给函数

示例代码:

```
#include <stdio.h>

// 函数原型
void func(int *);

int main()
{
    int num = 100;
    int *ptr = &num;

    // 调用函数,传地址
    func(&num);
    printf("num = %d\n", num);

    // 调用函数,传指针
    func(ptr);
    printf("num = %d\n", num);

    return 0;
}

// 函数声明
void func(int *p)
{
    *p += 1;
}
```

输出结果:

```
num = 101
num = 102
```

内存示意图:

```
int main()
{
    int num = 100;
    int *ptr = &num;

    // 调用函数,传地址
    func(&num);
    printf("num = %d\n", num);

    // 调用函数,传指针
    func(ptr);
    printf("num = %d\n", num);

    return 0;
}
```

```
void func(int *p)
{
    *p += 1;
}
```

main	
	0x000000f67fa9ff00
num	100
ptr	0x000000f67fa9ff00

2) 传数组给函数

数组名本身就代表数组首地址，因此传数组的本质就是传地址。

示例代码：

```
#include <stdio.h>

/* 函数声明 */
double getAverage(int *arr, int size);

int main()
{
    /* 带有 5 个元素的整型数组 */
    int balance[5] = {1000, 2, 3, 17, 50};
    double avg;

    /* 传递一个指向数组的指针作为参数 */
    avg = getAverage(balance, 5);
    /* 输出返回值 */
    printf("Average value is: %f\n", avg);

    return 0;
}

// 函数定义
double getAverage(int *arr, int size)
{
    int sum = 0;
```

```
double avg;
for (int i = 0; i < size; ++i)
{
    sum += *arr;
    printf("arr 存放的地址:%p \n", arr);
    arr++;
}
avg = (double)sum / size;

return avg;
}
```

输出结果:

```
arr 存放的地址:000000f47adff850
arr 存放的地址:000000f47adff854
arr 存放的地址:000000f47adff858
arr 存放的地址:000000f47adff85c
arr 存放的地址:000000f47adff860
Average value is: 214.400000
```

内存示意图:

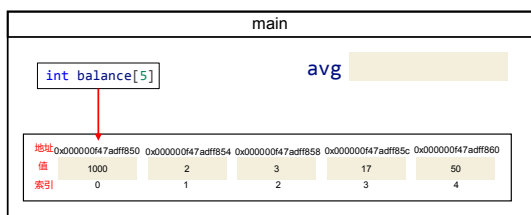
```
int main()
{
    /* 带有 5 个元素的整型数组 */
    int balance[5] = {1000, 2, 3, 17, 50};
    double avg;

    /* 传递一个指向数组的指针作为参数 */
    avg = getAverage(balance, 5);
    /* 输出返回值 */
    printf("Average value is: %f\n", avg);

    return 0;
}
```

```
double getAverage(int *arr, int size)
{
    int sum = 0;
    double avg;
    for (i = 0; i < size; ++i)
    {
        sum += *arr;
        printf("arr存放的地址:%p \n", arr);
        arr++;
    }
    avg = (double)sum / size;

    return avg;
}
```



问: 如果在 getAverage()函数中, 通过指针修改了数组的值, 那么 main 函数中的 balance 数组的值是否会相应变化?

答: 会的, 因为 getAverage 函数中的指针, 指向的就是 main 函数中的 balance 数组。

12.4.2 指针函数（返回指针的函数）

1) 基本介绍和使用

如果函数的返回值是一个指针，这样的函数称为指针函数。

语法规则：

返回类型 *指针函数名(参数列表)

代码示例：

请编写一个函数 `strlong()`，返回两个字符串中较长的一个。

```
#include <stdio.h>
#include <string.h>

char *strlong(char *str1, char *str2)
{
    if (strlen(str1) >= strlen(str2))
    {
        return str1;
    }
    else
    {
        return str2;
    }
}

int main()
{
    char str1[30], str2[30], *str;
    printf("\n 请输入第 1 个字符串: ");
    scanf("%s", &str1);
    printf("\n 请输入第 2 个字符串: ");
    scanf("%s", &str2);

    str = strlong(str1, str2);
    printf("\nLonger string: %s \n", str);

    return 0;
}
```

2) 返回值不能指向局部变量

函数运行结束后会销毁在它内部定义的所有局部数据，包括局部变量、局部数组和形式参数，所以函数返回的指针不能指向这些数据。

如果确实有这样的需求，需要定义为静态局部变量。

示例代码：

```
#include <stdio.h>
```

```
int *func()
{
    // int n = 100; // 局部变量, 在 func 返回时, 就会销毁
    static int n = 100; // 如果这个局部变量是 static 性质的, 那么 n 存放数据
    的空间在静态数据区
    return &n;
}

int main()
{
    int *p = func();
    int n = *p;
    printf("value=%d", n);

    return 0;
}
```

输出结果:

```
value=100
```

3) 应用案例

编写一个函数, 它会生成 10 个随机数, 并使用数组名作为返回值。

说明: 生成随机数可以使用 rand() 函数, 由 C 语言标准库 <stdlib.h> 提供, 可以返回一个随机整数。

```
#include <stdio.h>
#include <stdlib.h>

// 函数返回一个数组
int *randArr()
{
    static int arr[10]; // 必须加上 static , 让 arr 的空间在静态数据区分配

    for (int i = 0; i < 10; i++)
    {
        arr[i] = rand();
    }

    return arr;
}

int main()
{
```

```
int *p = randArr(); // p 指向是在 f1 生成的数组的首地址(即第一个元素的地址)
for (int i = 0; i < 10; i++)
{
    printf("%d\n", *(p++));
}

return 0;
}
```

输出结果:

```
41
18467
6334
26500
19169
15724
11478
29358
26962
24464
```

12.4.3 函数指针（指向函数的指针）

一个函数总是占用一段连续的内存区域,函数名在表达式中有时也会被转换为该函数所在内存区域的首地址,这和数组名非常类似。

把函数的这个首地址（或称入口地址）赋予一个指针变量,使指针变量指向函数所在的内存区域,然后通过指针变量就可以找到并调用该函数,这种指针就是函数指针。

语法规则:

返回类型 (*函数指针名)(参数列表);

注意: () 的优先级高于*, 第一个括号不能省略, 如果省略, 就成了指针函数。

示例代码:

用函数指针来实现对函数的调用, 返回两个整数中的最大值。

```
#include <stdio.h>

// 返回两个数中较大的一个
int max(int a, int b)
{
    return a > b ? a : b;
}

int main()
{
    int x, y, maxVal;
```

```
// 定义函数指针
// int (*pmax)(int x, int y) = max;
int (*pmax)(int, int) = max; // 可以省略参数名

printf("Input first number:");
scanf("%d", &x);
printf("Input second number:");
scanf("%d", &y);

// 通过函数指针调用函数
maxVal = (*pmax)(x, y);

printf("Max value: %d\n", maxVal);
printf("pmax's value is %p, pmax's address is %p\n", pmax, &pmax);
printf("max's value is %p, max's address is %p\n", max, &max);

return 0;
}
```

内存示意图:



案例演示：函数指针

让天下没有难学的技术

```
// 返回两个数中较大的一个
int max(int a, int b)
{
    return a > b ? a : b;
}

int main()
{
    int x, y, maxVal;

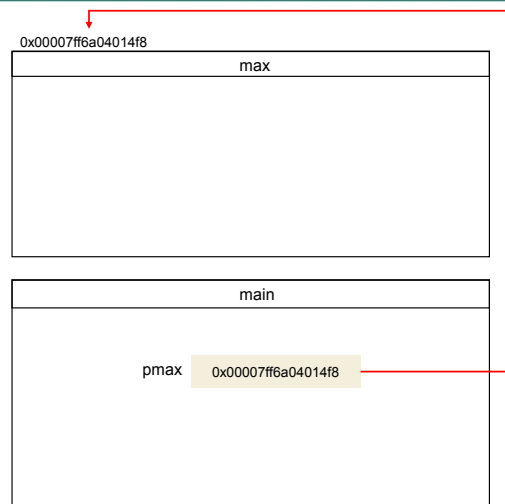
    // 定义函数指针
    // int (*pmax)(int x, int y) = max;
    int (*pmax)(int, int) = max; // 可以省略参数名

    printf("Input first number:");
    scanf("%d", &x);
    printf("Input second number:");
    scanf("%d", &y);

    // 通过函数指针调用函数
    maxVal = (*pmax)(x, y);

    printf("Max value: %d\n", maxVal);
    printf("pmax's value is %p, pmax's address is %p\n", pmax, &pmax);
    printf("max's value is %p, max's address is %p\n", max, &max);

    return 0;
}
```



12.4.4 回调函数

函数指针可以作为某个函数的参数来使用的,回调函数就是一个通过函数指针调用的函数。

简单的讲,回调函数是由别的函数执行时调用你传入的函数。

示例代码：

使用回调函数的方式，给一个整型数组 `int arr[10]` 赋 10 个随机数。

```
#include <stdlib.h>
#include <stdio.h>

// 1. f 就是函数指针
// 2. f 在 initArray 中被调用，是回调函数
void initArray(int *array, int arraySize, int (*f)())
{
    // 循环
    for (int i = 0; i < arraySize; i++)
    {
        array[i] = f(); // 通过 函数指针调用了 getNextRandomValue 函数
    }
}

int main()
{
    int myarray[10];

    // rand 是系统函数，会返回一个随机整数，位于 <stdlib.h> 标准库中
    // 传入 rand 作为参数
    initArray(myarray, 10, rand);

    // 输出赋值后的数组
    for (int i = 0; i < 10; i++)
    {
        printf("%d \n", myarray[i]);
    }

    return 0;
}
```

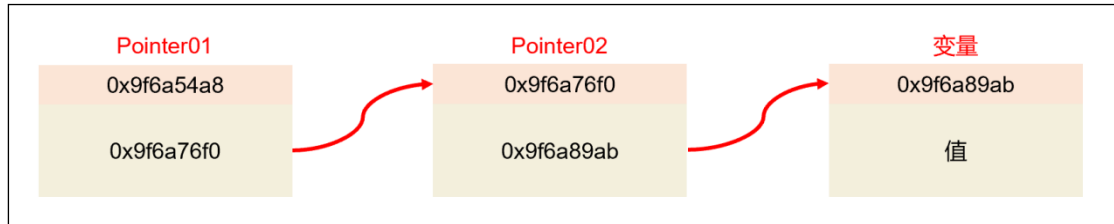
输出结果：

```
41
18467
6334
26500
19169
15724
11478
29358
26962
24464
```

12.5 指向指针的指针(多级指针)

1) 基本介绍

指向指针的指针是一种多级间接寻址的形式，或者说是一个指针链。通常，一个指针包含一个变量的地址。当我们定义一个指向指针的指针时，第一个指针包含了第二个指针的地址，第二个指针指向包含实际值的位置。



2) 多级指针的定义与使用

(1) 声明多级指针时，需要使用多个星号来表示指针的级别。

```

int *ptr;          // 一级指针
int **pptr;        // 二级指针
int ***ppptr;      // 三级指针
  
```

(2) 初始化多级指针时，你需要逐级给指针赋值，确保每个级别的指针指向正确的目标。

```

int var;
int *ptr = &var;          // 一级指针指向 var
int **pptr = &ptr;        // 二级指针指向 ptr
int ***ppptr = &pptr;     // 三级指针指向 pptr
  
```

(3) 解引用多级指针时，需要根据指针的级别使用适当数量的星号解引用操作。

```

printf("Value of var: %d\n", var);
printf("Value of ptr: %d\n", *ptr);          // 解引用一次
printf("Value of pptr: %d\n", **pptr);      // 解引用两次
printf("Value of ppptr: %d\n", ***ppptr);   // 解引用三次
  
```

3) 多级指针案例演示

```

#include <stdio.h>

int main()
{
    int var;
    int *ptr;          // 一级指针
    int **pptr;        // 二级指针
    int ***ppptr;      // 三级指针

    var = 3000;
  
```

```
ptr = &var;    // var 变量的地址赋给 ptr
pptr = &ptr;   // 表示将 ptr 存放的地址，赋给 pptr
ppptr = &pptr; // 表示将 pptr 存放的地址，赋给 ppptr

printf("var`s address is %p \nvar`s value is %d \n\n", &var, var);
printf("ptr`s address is %p \nptr`s value is %p \n*ptr=%d \n\n",
&ptr, ptr, *ptr);
printf("pptr`s address is %p \npptr`s value is %p \n*pptr=%p
\n**pptr=%d \n\n", &pptr, pptr, *pptr, **pptr);
printf("ppptr`s address is %p \nppptr`s value is %p \n*ppptr=%p
\n**ppptr=%p \n***ppptr=%d \n\n", &ppptr, ppptr, *ppptr, **ppptr,
***ppptr);

return 0;
}
```

输出结果

```
var`s address is 0000000cffbffdec
var`s value is 3000

ptr`s address is 0000000cffbffde0
ptr`s value is 0000000cffbffdec
*ptr=3000

pptr`s address is 0000000cffbffdd8
pptr`s value is 0000000cffbffde0
*pptr=0000000cffbffdec
**pptr=3000

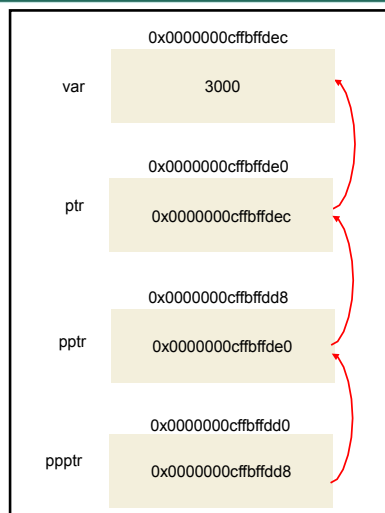
ppptr`s address is 0000000cffbffdd0
ppptr`s value is 0000000cffbffdd8
*ppptr=0000000cffbffde0
**ppptr=0000000cffbffdec
***ppptr=3000
```

内存布局图:

```
int var;
int *ptr;    // 一级指针
int **pptr;  // 二级指针
int ***ppptr; // 三级指针

var = 3000;
ptr = &var;    // var 变量的地址赋给 ptr
pptr = &ptr;   // 表示将 ptr 存放的地址，赋给 pptr
ppptr = &pptr; // 表示将 pptr 存放的地址，赋给 ppptr
```

```
printf("var's address is %p \nvar's value is %d \n", &var, var);
printf("ptr's address is %p \nptr's value is %p \n*ptr=%d \n", &ptr,
ptr, *ptr);
printf("pptr's address is %p \npptr's value is %p \n*pptr=%p \n**pptr=%d
\n", &pptr, pptr, *pptr, **pptr);
printf("ppptr's address is %p \nppptr's value is %p \n*ppptr=%p
\n**ppptr=%p \n***ppptr=%d \n", &ppptr, pptr, *pptr, **pptr,
***pptr);
```



12.6 空指针

赋为 NULL 值的指针被称为空指针，NULL 是一个定义在标准库 <stdio.h>中的值为零的宏常量。

声明指针变量的时候，如果没有确切的地址赋值，为指针变量赋一个 NULL 值是好的编程习惯。

```
#include <stdio.h>

int main()
{
    int *p = NULL;
    int num = 34;
    p = &num;

    printf("p=%p\n", p);
    printf("*p=%d\n", *p);

    return 0;
}
```

输出结果：

```
p=000000000061FE14
*p=34
```

12.7 野指针

12.7.1 什么是野指针

野指针就是指针指向的位置是不可知（随机性，不正确，没有明确限制的）。

12.7.2 野指针的成因

1) 指针使用前未初始化

指针变量在定义时如果未初始化，其值是随机的，此时操作指针就是去访问一个不确定的地址，所以结果是不可知的。此时 p 就为野指针。

```
int *p;
printf("%d\n", *p);
```

在没有给指针变量显式初始化的情况下，一系列的操作(包括修改指向内存的数据的值)也是错误的。

```
int *p;
*p = 10;
```

2) 指针越界访问

```
int arr[4] = {10, 20, 30, 40};
int *p = arr;
p += 4;
printf("%d", *p); // 此时 *p 即为越界访问
```

当 p += 4 之后，此时 *p 访问的内存空间不在数组有效范围内，此时 *p 就属于非法访问内存空间，p 为野指针。

3) 指针指向已释放的空间

```
#include <stdio.h>

int *test()
{
    int a = 10;
    return &a; // &a=0x0012ff40
}

int main()
{
    int *p = test();
    printf("%d", *p);
    return 0;
}
```

调用 test() 函数将返回值赋给 p，test 函数的返回值是局部变量 a 的地址，函数调用结束局部变量会被销毁，其所占用的内存空间会被释放，p 指向的是已经释放的内存空间，所以 p 是野指针。

12.7.3 如何避免野指针

(1) 指针初始化如果没有确切的地址赋值，为指针变量赋一个 NULL 值是好的编程习惯。

(2) 小心指针越界。

(3) 避免返回指向局部变量的指针。

(4) 指针使用之前检查指针是否为 NULL。

12.8 指针使用一览

变量定义	类型表示	含义
int i	int	i 是一个整型变量。
int *p	int *	p 是一个指向整型数据的指针
int a[5]	int[5]	a 是一个 5 个元素的整型数组。
int *p[4]	int *[4]	p 是一个指针数组，每个元素都是指向整型数据的指针。
int (*p)[4]	int (*)[4]	p 是一个数组指针，指向一个 4 个元素的整型数组。
int f()	int ()	f 是一个返回整型数据的函数
int *f()	int *()	f 是一个指针函数，返回指向整型数据的指针。
int (*p)()	int (*)()	p 是一个函数指针，指向一个返回整型数据的函数。
int **p	int **	p 是一个指向指针的指针。

第 13 章 自定义数据类型

13.1 枚举

13.1.1 什么是枚举

枚举 (Enumeration) 是 C 语言中的一种用户自定义数据类型，用于定义一组相关的常量。枚举使我们可以为这些常量分配有意义的名字，从而提高代码的可读性和可维护性。

如果我们定义一个变量，用来表示一周中的一天，该变量的值只能从七个有效值中选择，枚举可以帮助我们限制变量的有效值。

13.1.2 如何定义枚举

1) 语法格式

枚举使用 `enum` 关键字来定义，枚举名称和枚举元素名称都是标识符，定义一个枚举就是定义了一种枚举数据类型，语法如下：

```
enum 枚举名称
{
    枚举元素 1,
    枚举元素 2,
    ...
    枚举元素 N
};
```

2) 枚举常量

枚举元素也称为枚举成员或枚举常量，具有如下特点：

- (1) 枚举元素的值必须在同一枚举中是唯一的
- (2) 枚举元素的值必须是整数类型，通常是 `int`
- (3) 如果没有为枚举元素指定值，编译器会自动为它们分配值，从 0 开始，依次递增。
- (4) 定义枚举的时候也可以为枚举元素自定义值，需保证唯一性和整型类型。

3) 代码演示

// 定义枚举类型,表示星期一到星期天,枚举元素的值自动分配,从 0 到 6。

```
enum Weekday
{
    MONDAY,
    TUESDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY,
    SATURDAY,
    SUNDAY
};
```

// 定义枚举类型,表示一月到十二月。给枚举元素 JANUARY 设置了值 1,后面的枚举元素从 1 开始自增,分别是 1 到 12

```
enum Month
{
    JANUARY = 1,
    FEBRUARY,
    MARCH,
    APRIL,
```

```
MAY,  
JUNE,  
JULY,  
AUGUST,  
SEPTEMBER,  
OCTOBER,  
NOVEMBER,  
DECEMBER  
};  
  
// 定义枚举类型，表示北京地区一年四季的平均温度。为每个枚举元素都设置值，自己  
// 设置的值需要保证唯一性和整型类型  
enum Season  
{  
    SPRING = 18,  
    SUMMER = 30,  
    AUTUMN = 20,  
    WINTER = -5  
};
```

13.1.3 枚举变量

1) 什么是枚举变量

定义变量时指定类型是我们所定义的枚举类型，该变量称为枚举变量。枚举变量的值应该是枚举类型中的枚举常量中的一个，不能是其他值。

2) 定义枚举变量的语法

(1) 第一种：先定义枚举类型，再定义枚举变量。

```
// 定义枚举类型  
enum DAY  
{  
    MON = 1,  
    TUE,  
    WED,  
    THU,  
    FRI,  
    SAT,  
    SUN  
};  
// 使用枚举类型定义枚举变量  
enum DAY day;
```

(2) 第二种：定义枚举类型的同时声明枚举变量。

```
enum DAY
```

```
{  
    MON = 1,  
    TUE,  
    WED,  
    THU,  
    FRI,  
    SAT,  
    SUN  
} day;
```

(3) 第三种：定义枚举类型的同时声明枚举变量，且省略枚举名称。

```
enum  
{  
    MON = 1,  
    TUE,  
    WED,  
    THU,  
    FRI,  
    SAT,  
    SUN  
} day;
```

3) 示例代码

```
#include <stdio.h>  
  
int main()  
{  
    // 定义枚举类型  
    enum Weekday  
    {  
        MONDAY,  
        TUESDAY,  
        WEDNESDAY,  
        THURSDAY,  
        FRIDAY,  
        SATURDAY,  
        SUNDAY  
    };  
  
    // 定义枚举变量并赋值  
    enum Weekday day = WEDNESDAY;  
    printf("%d \n", day);  
  
    // 修改枚举变量的值  
    day = FRIDAY;
```

```
printf("%d \n", day);

// 修改枚举变量的值
// 100 并不是枚举常量中的一个，这样做虽然不会报错，但失去了枚举的意义，不建议这么做
day = 100;
printf("%d \n", day);

return 0;
}
```

输出结果：

```
2
4
100
```

13.1.4 枚举的应用

1) 枚举遍历

如果枚举常量的值是连续的，我们可以使用循环遍历；如果枚举常量的值不是连续的，则无法遍历。

```
#include <stdio.h>

int main()
{
    // 定义枚举类型同时定义枚举变量
    enum Weekday
    {
        MONDAY = 1,
        TUESDAY,
        WEDNESDAY,
        THURSDAY,
        FRIDAY,
        SATURDAY,
        SUNDAY
    } day;

    // 使用循环遍历出所有的枚举常量
    for (day = MONDAY; day <= SUNDAY; day++)
    {
        printf("%d \n", day);
    }

    // 如果枚举常量的值并不是连续的数字，则无法遍历
```

```
// 定义枚举类型同时定义枚举变量
enum Season
{
    SPRING = 18,
    SUMMER = 30,
    AUTUMN = 20,
    WINTER = -5
} season;

for (season = SPRING; season <= WINTER; season++)
{
    printf("%d \n", season);
}

return 0;
}
```

输出结果:

```
1
2
3
4
5
6
7
```

2) 枚举在 switch 中使用

枚举可以与 switch 结合使用，请看如下案例：

```
#include <stdio.h>

int main()
{
    // 定义枚举类型
    enum SEASONS
    {
        SPRING = 1,
        SUMMER,
        AUTUMN,
        WINTER
    };

    // 定义枚举变量
    enum SEASONS season;

    // 获取用户输入
```

```
printf("请输入你喜欢的季节: (1. spring, 2. summer, 3. autumn 4
winter): ");
scanf("%d", &season);

// 判断
switch (season)
{
case SPRING:
    printf("你喜欢的季节是春天");
    break;
case SUMMER:
    printf("你喜欢的季节是夏天");
    break;
case AUTUMN:
    printf("你喜欢的季节是秋天");
    break;
case WINTER:
    printf("你喜欢的季节是冬天");
    break;
default:
    printf("你没有选择你喜欢的季节");
}

return 0;
}
```

13.2 结构体

13.2.1 为什么需要结构体

C 语言内置的数据类型，除了几种原始的基本数据类型，只有数组属于复合类型，可以同时包含多个值，但是只能包含相同类型的数据，实际使用场景受限。

需求分析 1:

现有一个需求，编写学生档案管理系统，这里需要描述一个学生的信息。该学生的信息包括学号、姓名、性别、年龄、家庭住址等，这些数据共同说明一个学生的总体情况。



学号	姓名	性别	年龄	家庭住址
1001	尚大壮	男	10	...
1002	谷小明	男	11	...
1003	佟丽丽	女	12	...

需求分析 2:

隔壁老王养了两只猫咪，一只名字叫小黄，今年 2 岁，橘色；另一只叫小黑，今年 3 岁，黑色。请编写一个程序，当用户输入小猫的名字时，就显示该猫的名字，年龄，颜色。如果用户输入的小猫名错误，则显示老王没有这只猫。

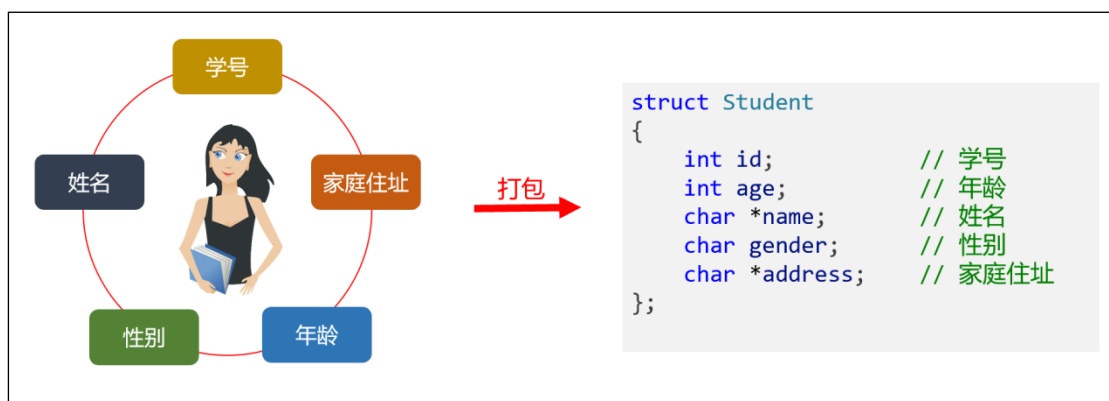


传统的解决方法:

尝试 1: 单独定义多个变量存储，实现需求，但是，多个变量，不便于数据的管理。

尝试 2: 使用数组，它是一组具有相同类型的数据的集合。但在编程中，往往还需要一组类型不同的数据，例如猫的名字使用字符串、年龄是 `int`，颜色是字符串，因为数据类型不同，不能用一个数组来存放。

13.2.2 什么是结构体



C 语言提供了 `struct` 关键字，允许自定义复合数据类型，将不同类型的值组合在一起，这种类型称为结构体（structure）类型。

C 语言没有其他语言的对象（object）和类（class）的概念，`struct` 结构很大程度上提供了对象和类的功能。

13.2.3 声明结构体

1) 格式

构建一个结构体类型的一般形式为：

```
struct 结构体名
{
    数据类型 1 成员名 1;
    数据类型 2 成员名 2;
    .....
    数据类型 n 成员名 n;
};
```

2) 代码示例

(1) 举例：学生

```
// 定义结构体：学生
struct Student
{
    int id;           // 学号
    int age;          // 年龄
    char *name;       // 姓名
    char gender;      // 性别
    char *address;    // 家庭住址
};
```

(2) 举例：通讯录

```
// 定义结构体：通讯录
struct Contacts
{
    char name[50];    //姓名
    int year;         //年
    int month;        //月
    int day;          //日
    char email[100];  //电子邮箱
    char mobile_number[12]; //手机号
};
```

13.2.4 声明结构体变量

1) 方式 1 先定义结构体，然后再创建结构体变量

```
// 定义结构体
struct Student
{
    int id;           // 学号
    int age;          // 年龄
```

```
char *name;    // 姓名
char gender;   // 性别
char *address; // 家庭住址
};
```

```
// 定义结构体变量
struct Student stu1, stu2;
```

2) 方式 2 在定义结构体的同时定义结构体变量

```
// 定义结构体的同时定义 stu1 和 stu2 两个变量
struct Student
{
    int id;        // 学号
    int age;       // 年龄
    char *name;    // 姓名
    char gender;   // 性别
    char *address; // 家庭住址
} stu1, stu2;
```

3) 方式 3 在定义时也可以不给出结构体名

```
// 不给出结构体的名字
struct
{
    int id;        // 学号
    int age;       // 年龄
    char *name;    // 姓名
    char gender;   // 性别
    char *address; // 家庭住址
} stu1, stu2;
```

注意：在 C 语言中，结构体（struct）和结构体变量是两个不同的概念。

（1）结构体是一种自定义的数据类型，像一种模板，定义了数据的格式。

（2）结构体变量是根据结构体类型创建的变量，代表了一个具体的对象，用于存储数据。

13.2.5 成员的获取和赋值

成员是结构体的一个组成部分，一般是基本数据类型、也可以是数组、指针、结构体等。

结构体的成员也可以称为属性。

结构体和数组类似，也是一组数据的集合，结构体使用点号 . 获取单个成员，可以进行赋值和取值。

1) 案例 1：结构体成员逐个赋值

```
#include <stdio.h>

int main()
{
    // 声明结构体
    struct Student
    {
        int id;           // 学号
        int age;          // 年龄
        char *name;       // 姓名
        char gender;      // 性别
        char *address;    // 家庭住址
    };

    // 声明结构体变量
    struct Student stu;

    // 逐个对成员进行赋值
    stu.id = 1001;
    stu.age = 19;
    stu.name = "Jack Ma";
    stu.gender = 'm';
    stu.address = "Changping BeiJing";

    // 获取成员的值
    printf("id: %d \n", stu.id);
    printf("name: %s \n", stu.name);
    printf("age: %d \n", stu.age);
    printf("gender: %c \n", stu.gender);
    printf("address: %s \n", stu.address);

    return 0;
}
```

输出结果:

```
id: 1001
name: Jack Ma
age: 19
gender: m
address: Changping BeiJing
```

2) 案例 2: 使用大括号一次性对结构体所有成员赋值

```
#include <stdio.h>

int main()
{
```

```
// 声明结构体以及结构体变量, 并给结构体成员赋值
struct Student
{
    int id;        // 学号
    int age;       // 年龄
    char *name;    // 姓名
    char gender;   // 性别
    char *address; // 家庭住址
} stu1 = {1002, 19, "Jim Liu", 'f', "Songjiang Shanghai"}, stu2 =
{1003, 16, "Tom Chen", 'm', "Baonan Shenzhen"};

// 获取成员的值
printf("id: %d \n", stu1.id);
printf("name: %s \n", stu1.name);
printf("age: %d \n", stu1.age);
printf("gender: %c \n", stu1.gender);
printf("address: %s \n \n", stu1.address);

printf("id: %d \n", stu2.id);
printf("name: %s \n", stu2.name);
printf("age: %d \n", stu2.age);
printf("gender: %c \n", stu2.gender);
printf("address: %s \n", stu2.address);

return 0;
}
```

输出结果:

```
id: 1002
name: Jim Liu
age: 19
gender: f
address: Songjiang Shanghai

id: 1003
name: Tom Chen
age: 16
gender: m
address: Baonan Shenzhen
```

13.2.6 结构体指针

1) 结构体指针介绍

结构体指针是指向结构体的指针变量, 允许间接访问和操作结构体的成员, 它提供了一种方便的方式来处理结构体数据。

声明结构体指针的语法格式:

```
struct 结构体名 *结构体指针变量名;
```

2) 声明结构体指针

通常需要先定义一个结构体变量，再创建结构体指针，并取结构体变量的地址作为结构体指针的值。

```
// 声明结构体
struct Student
{
    int id;        // 学号
    int age;       // 年龄
    char *name;    // 姓名
    char gender;   // 性别
    char *address; // 家庭住址
};
// 声明结构体变量
struct Student stu;
// 声明结构体指针并初始化
struct Student *ptr = &stu;
```

3) 通过结构体指针访问成员

结构体指针可以通过“->”操作符访问结构体的成员。

```
#include <stdio.h>

int main()
{
    // 结构体
    struct Student
    {
        char *name;
        int age;
        char gender;
    };

    // 结构体变量
    struct Student s = {"张三", 20, 'M'};
    // 通过结构体变量访问成员
    printf("name=%s,age=%d,gender=%c \n", s.name, s.age, s.gender);

    // 声明结构体指针
    struct Student *ps = &s;

    // 结构体指针通过解引用访问成员
```

```
printf("name=%s,age=%d,gender=%c\n", (*ps).name, (*ps).age,
(*ps).gender);

// 结构体指针使用->访问成员
printf("name=%s,age=%d,gender=%c\n", ps->name, ps->age,
ps->gender);

return 0;
}
```

输出结果:

```
name=张三,age=20,gender=M
name=张三,age=20,gender=M
name=张三,age=20,gender=M
```

总结: 如果指针变量 p 指向一个结构体变量 stu, 以下 3 种用法等价:

- (1) stu.成员名
- (2) (*p).成员名
- (3) p->成员名

13.2.7 综合案例

1) 小狗案例

- (1) 编写一个 Dog 结构体, 包含 name(char *)、age(int)、weight(double)属性。
- (2) 编写一个 say 函数, 返回字符串, 方法返回信息中包含所有成员值。
- (3) 在 main 函数中, 创建 Dog 结构体变量, 调用 say 函数, 将调用结果打印输出。

```
#include <stdio.h>

// 定义 Dog 结构体
struct Dog
{
    char *name;
    int age;
    double weight;
};

// say 函数, 返回字符串, 信息中包含所有成员值
char *say(struct Dog dog)
{
    // 将这个信息放入到一个字符串(字符数组)
    static char info[50]; // 局部静态变量
    sprintf(info, "name=%s age=%d weight=%.2f", dog.name, dog.age,
dog.weight);
}
```

```
    dog.name = "小花";
    return info;
}

int main()
{
    // 测试
    // 定义结构体变量
    struct Dog dog;
    char *info = NULL;
    dog.name = "小黄";
    dog.age = 1;
    dog.weight = 3.4;
    info = say(dog); // 结构体变量默认是值传递
    printf("小狗信息: %s \n", info);
    printf("main 中小狗名字: %s \n", dog.name);

    return 0;
}
```

2) 盒子案例

(1) 编程创建一个 Box 结构体，在其中定义三个成员表示一个长方体的长、宽和高，长宽高可以通过控制台输入。

(2) 定义一个函数获取长方体的体积（volume）。

(3) 创建一个结构体指针，打印给定尺寸的长方体的体积。

```
#include <stdio.h>

// 定义 Box 结构体
struct Box
{
    double length;
    double width;
    double height;
};

// 获取立方体体积的函数
double getVolume(struct Box *box)
{
    return box->length * box->width * box->height;
}

int main()
```

```
{
    // 创建结构体变量
    struct Box box;
    // 创建结构体指针
    struct Box *cube = &box;

    printf("Enter length: ");
    scanf("%lf", &cube->length);

    printf("Enter width: ");
    scanf("%lf", &cube->width);

    printf("Enter height: ");
    scanf("%lf", &cube->height);

    // 调用函数获取体积并打印
    printf("Volume is: %.2f\n", getVolume(cube));

    return 0;
}
```

3) 景区门票案例

- (1) 一个景区根据游人的年龄收取不同价格的门票。
- (2) 请编写游人结构体 (Visitor)，根据年龄段决定能够购买的门票价格并输出。
- (3) 规则：年龄>18，门票为 20 元，其它情况免费。
- (4) 可以循环从控制台输入名字和年龄，打印门票收费情况，如果名字输入 n，则退出程序。

```
#include <stdio.h>
#include <string.h>

// 定义结构体
struct Visitor
{
    char name[10];
    int age;
    double pay; // 应付票价
};

// 编写函数处理业务
void ticket(struct Visitor *visitor)
{
    // 判断
```

```
    if (visitor->age > 18)
    {
        visitor->pay = 20;
    }
    else
    {
        visitor->pay = 0;
    }
}

int main()
{
    // 测试
    // 创建结构体变量(创建一个游客)
    struct Visitor visitor;
    // 循环的输入名字和年龄
    while (1)
    {

        printf("\n 请输入游客名字: ");
        scanf("%s", visitor.name);
        // 判断如果名字输入 n ,则退出程序
        if (!strcmp("n", visitor.name))
        {
            break;
        }
        printf("\n 请输入游客年龄: ");
        scanf("%d", &visitor.age);
        // 调用函数 ticket , 获取应付的票价
        ticket(&visitor);
        printf("\n 该游客应付票价: %.2f\n", visitor.pay);
    }

    printf("退出程序");

    return 0;
}
```

13.2.8 总结

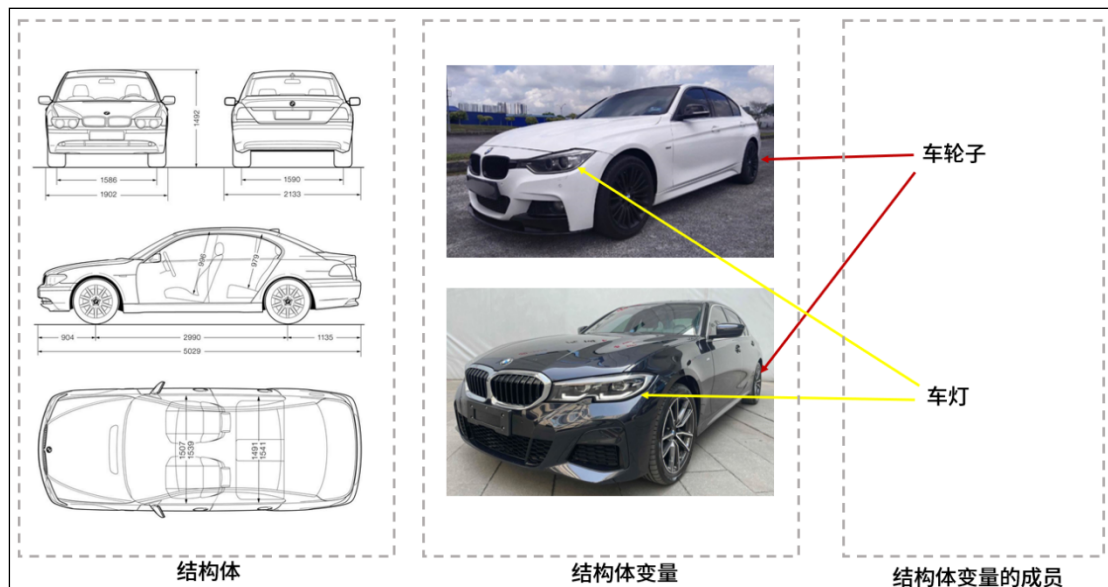
区分三个概念：结构体、结构体变量、结构体变量的成员。

结构体是自定义的数据类型，表示的是一种数据类型。结构体变量代表一个具体变量。

类比：

```
int num1;           // int 是数据类型，而 num1 是一个具体的 int 变量
struct Car car1;    // Car 是结构体数据类型，而 car1 是一个 Car 变量
```

Car 就像一个“汽车图纸”，生成出来的具体的一辆辆汽车，就类似于一个个的结构体变量。这些结构体变量都含有相同的成员，将结构体变量的成员比作“零件”，同一张图纸生产出来的零件的作用都是一样的。



13.3 共用体

13.3.1 什么是共用体

有时需要一种数据结构，不同的场合表示不同的数据类型。比如，如果只用一种数据结构表示学生的“成绩”，这种结构就需要有时是整数（80、90），有时是字符（'A'、'B'），有时是浮点数（80.5、60.5）。

C 语言提供了共用体类型（Union 结构），用来自定义可以灵活变更的数据结构。它内部可以包含各种属性，但同一时间只能有一个属性，因为所有属性都保存在同一个内存地址，后面写入的属性会覆盖前面的属性。这样做的最大好处是节省空间。

“共用体”与“结构体”的定义形式相似，但它们的含义是不同的。

（1）结构体变量所占内存长度，可以认为是各成员占的内存长度的叠加；每个成员分别占有其自己的内存单元。

（2）共用体变量所占的内存长度等于最长的成员的长度；几个成员共用一个内存区。

13.3.2 声明共用体

（1）格式：

```
union 共用体类型名称
{
    数据类型 成员名 1;
    数据类型 成员名 2;
    ...
    数据类型 成员名 n;
};
```

(2) 举例:

```
union data
{
    int m;
    float x;
    char c;
};
```

上例中，union 命令定义了一个包含三个属性的数据类型 data。虽然包含三个属性，但是同一时间只能取到一个属性。最后赋值的属性，就是可以取到值的那个属性。

13.3.3 声明共用体变量

1) 方式 1: 先定义共用体类型，再定义共用体变量

```
// 声明共用体类型
union data
{
    short m;
    float x;
    char c;
};

// 声明共用体变量
union data a, b;
```

2) 方式 2: 定义共用体类型的同时定义共用体变量

```
union data
{
    short m;
    float x;
    char c;
} a, b;
```

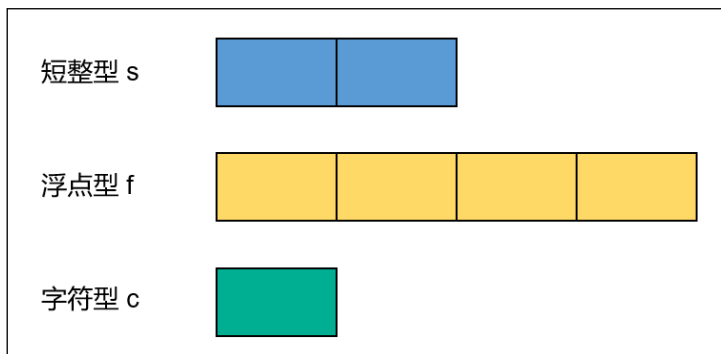
3) 方式 3: 在定义时也可以不给出共用体名

```
union
{
    short m;
    float x;
```

```
char c;
} a, b;
```

13.3.4 共用体内存分析

以上一节方式 2 中共用体变量 `a` 为例，它由 3 个成员组成，分别是 `m`、`x` 和 `c`，系统会按照最长的成员为它分配内存，由于成员 `x` 的长度最长，它占 4 个字节，所以共用体变量 `a` 的内存空间也为 4 个字节。



13.3.5 成员的获取和赋值

同结构体一样，共用体也使用点号.获取单个成员，可以进行赋值和取值。

1) 方式 1:

```
union data a;
a.c = 4;
```

2) 方式 2: 声明共用体变量的同时，给任一成员赋值

```
union data a = {.c = 4};
```

3) 方式 3: 声明共用体变量的同时，给首成员赋值

```
union data a = {8};
```

注意：方式 3 不指定成员名，所以只能为第一个成员进行赋值。

13.3.6 课堂案例

现有一张关于学生信息和教师信息的表格。学生信息包括姓名、编号、性别、职业、分数，教师的信息包括姓名、编号、性别、职业、教学科目：可以参考下面的表格。

请利用共用体，只使用一个结构体保存每个人的信息。

Name	Num	Sex	Profession	Score / Course
孙二娘	501	女(f)	学生(s)	89.5
吴用	1011	男(m)	老师(t)	math
顾大嫂	109	女(f)	老师(t)	English
林冲	982	男(m)	学生(s)	95.0

案例演示：

```
#include <stdio.h>
#define TOTAL 2 // 人员总数

// 定义了一个结构体 Person
struct Person
{
    char name[20]; // name
    int num; // 编号
    char sex; // 性别 f => 女 m=>男
    char profession; // 职业 s=>学生 t=>老师
    union
    {
        float score;
        char course[20];
    } sc; // sc 是一个共用体变量
};

int main()
{
    int i;
    struct Person persons[TOTAL]; // 定义了一个结构体数组

    // 输入人员信息
    for (i = 0; i < TOTAL; i++)
    {
        printf("Input info: ");
        scanf("%s %d %c %c", persons[i].name, &(persons[i].num),
            &(persons[i].sex), &(persons[i].profession));

        if (persons[i].profession == 's')
        { // 如果是学生
            printf("请输入该学生成绩:");
            scanf("%f", &persons[i].sc.score);
        }
        else
        { // 如果是老师
            printf("请输入该老师课程:");
            scanf("%s", persons[i].sc.course);
        }
        fflush(stdin); // 刷新
    }

    // 输出人员信息
    printf("\nName\t\tNum\tSex\tProfession\tScore / Course\n");
```

```
for (i = 0; i < TOTAL; i++)
{
    if (persons[i].profession == 's')
    { // 如果是学生
        printf("%s\t\t%d\t%c\t%c\t\t%f\n", persons[i].name,
persons[i].num, persons[i].sex, persons[i].profession,
persons[i].sc.score);
    }
    else
    { // 如果是老师
        printf("%s\t\t%d\t%c\t%c\t\t%s\n", persons[i].name,
persons[i].num, persons[i].sex, persons[i].profession,
persons[i].sc.course);
    }
}

return 0;
}
```

13.4 typedef

13.4.1 typedef 介绍

C 语言允许为一个数据类型起一个新的别名，就像给人起“绰号”一样。

起别名的目的不是为了提高程序运行效率，而是为了编码方便。例如，有一个结构体的名字是 student，定义一个结构体变量 stu1，代码如下：

```
struct student stu1;
```

struct 看起来就是多余的，但不写又会报错。如果为 struct student 起了一个别名 Student，书写起来就简单了：

```
Student stu1;
```

这种写法更加简练，意义也非常明确，不管是在标准头文件中还是以后的编程实践中，都会大量使用这种别名。

13.4.2 typedef 应用场景

用 typedef 声明数组类型、指针类型，结构体类型、共用体类型等，使得编程更加方便。

1) 为某个基本类型起别名

typedef 命令用来为某个类型起别名。

```
typedef 类型名 别名;
```

习惯上，常把用 `typedef` 声明的类型名的第 1 个字母用大写表示，以便与系统提供的标准类型标识符相区别。

代码示例：

```
typedef int Integer; //用 Integer 作为 int 类型别名，作用与 int 相同
Integer a, b;
a = 1;
b = 2;

typedef unsigned char Byte; //为类型 unsigned char 起别名 Byte
Byte c = 'z';
```

注意：使用 `typedef` 可以为基本类型一次起多个别名。

```
typedef int chocolate, doughnut, mushroom; //一次性为 int 类型起了三个别名
```

2) 为结构体、共用体起别名

为 `struct`、`union` 等命令定义的复杂数据结构创建别名，从而便于引用。

```
struct Treenode
{
    // ...
};

typedef struct Treenode Tree; //Tree 是 struct Treenode 的别名
```

`typedef` 也可以与 `struct` 定义结构体的命令写在一起。

```
typedef struct Treenode
{
    char *name;
    int legs;
    int speed;
} Tree;
```

上例中，自定义数据类型时，同时使用 `typedef` 命令，为 `struct Treenode` 起了一个别名 `Tree`。

这种情况下，C 语言允许省略结构体名。

```
typedef struct
{
    char *name;
    int legs;
    int speed;
} Tree;
```

上例相当于为一个匿名的结构体起了别名 `Tree`，进而：

```
// 使用 typedef 之前
```

```
struct Treenode tr;

// 使用 typedef 之后
Tree tr;
```

再举例：typedef 命令可以为共用体起别名。

```
typedef union
{
    short count;
    float weight;
    float volume;
} Quantity;
```

上例中，union 命令定义了一个包含三个属性的数据类型，typedef 命令为它起别名为 Quantity。

3) 为指针起别名

typedef 可以为指针起别名。

```
typedef int *intptr;
int a = 10;
intptr x = &a;
```

上例中，intptr 是 int* 的别名。不过，使用的时候要小心，这样不容易看出来，变量 x 是一个指针类型。

再举例：

```
typedef char *String;

char *str1 = "hello"; // 之前的写法
String str2 = "hello"; // 现在的写法
```

为字符指针起别名为 String，以后使用 String 声明变量时，就可以轻易辨别该变量是字符串。

4) 为数组类型起别名

typedef 也可以用来为数组类型起别名。

```
// 举例 1 给 int[5] 取别名 fiveInts
typedef int fiveInts[5];
fiveInts x = {11, 22, 33, 44, 55}; // 相当于 int x[5] = {11, 22, 33, 44, 55};

// 举例 2 给 int[100] 取别名 Num
typedef int Num[100]; // 声明 Num 为整型数组类型名
Num a;                // 相当于 int a[100];
```

举例，使用 typedef 为数组指针取别名。

```
// 为 int (*)[5] 取别名为 IntArrayPointer
typedef int (*IntArrayPointer)[5];
// 定义数组
int arr[5] = {1, 2, 3, 4, 5};
// 使用别名定义数组指针
IntArrayPointer ptr = &arr; // 使用别名指向数组
```

13.4.3 综合使用演示

```
#include <stdio.h>

// 给字符指针取别名
typedef char *String;

// 给结构体取别名
typedef struct
{
    int id;
    String name;
    String profile;
} User;

int main()
{
    // 声明一个结构体变量
    User u;

    // 初始化结构体变量的属性赋值
    u.id = 101;
    u.name = "张三";
    u.profile = "一个纯粹的人，一个高尚的人，一个脱离了低级趣味的人。";

    // 打印结构体变量的属性
    printf("id: %d\n", u.id);
    printf("name: %s\n", u.name);
    printf("profile: %s\n", u.profile);

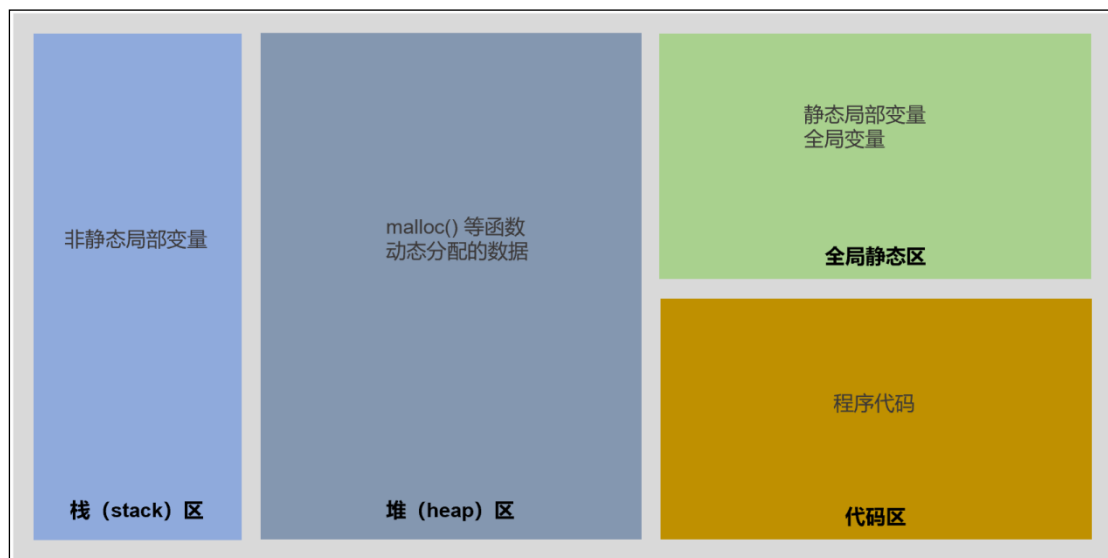
    return 0;
}
```

输出结果：

```
id: 101
name: 张三
profile: 一个纯粹的人，一个高尚的人，一个脱离了低级趣味的人。
```

第 14 章 动态内存分配

14.1 C 程序内存分配



动态内存分配指根据需要向系统申请所需大小的空间，由于未在声明部分定义其为变量或者数组，不能通过变量名或者数组名来引用这些数据，只能通过**指针**来引用）

14.2 void 指针（无类型指针）

1) void 指针介绍

C99 允许定义一个类型为 void 的指针变量，它可以指向任何类型的数据。

2) void 指针作用

指针变量必须有类型，否则编译器无法知道如何解读内存块保存的二进制数据。但是，有时候向系统请求内存的时候，还不确定会有什么类型的数据写入内存，需要要先获得内存块，稍后再确定写入的数据类型。

这种情况下就可以使用 void 指针，它只有内存块的地址信息，没有类型信息，等到使用该块内存的时候，再向编译器补充说明，里面的数据类型是什么。

3) void 指针特点

(1) void 指针与其他所有类型指针之间是互相转换关系，任一类型的指针都可以转为 void 指针，而 void 指针也可以转为任一类型的指针。

(2) 由于不知道 void 指针指向什么类型的值，所以不能用 * 运算符取出它指向的值（解引用）。

4) 代码示例

```
#include <stdio.h>

int main()
{
    int num = 42;
    double pi = 3.14159;

    // int 指针转为 void 指针
    void *viPtr = &num; // 隐式类型转换

    // double 指针转为 void 指针
    void *vdPtr = &pi; // 隐式类型转换

    // void 指针转换为 int 指针并解引用
    // int *intPtr = viPtr; // 隐式类型转换
    int *intPtr = (int *)viPtr; // 显式类型转换
    printf("整数值: %d \n", *intPtr);

    // void 指针转换为 double 指针并解引用
    // double *doublePtr = vdPtr; // 隐式类型转换
    double *doublePtr = (double *)vdPtr; // 显式类型转换
    printf("浮点数: %f \n", *doublePtr);

    // void 指针不能解引用 会报错
    // printf("%d", *viPtr);
    // printf("%d", *vdPtr);

    return 0;
}
```

输出结果:

```
整数值: 42
浮点数: 3.141590
```

5) 友情提示

(1) 其他类型指针赋给 void 指针，使用隐式转换即可，因为 void 指针不包含指向的数据类型的信息，通常是安全的。

(2) void 指针赋给其他类型指针，建议使用显式类型转换，这样更加安全，如果使用隐式类型转换，有些编译器会触发警告。

14.3 内存分配相关函数

头文件 <stdlib.h> 声明了四个关于内存动态分配的函数：

1) malloc() 函数

malloc() 函数用于分配一块连续的内存空间。

函数原型：

```
void *malloc(size_t size);
```

返回值说明：

如果内存分配成功，返回一个 void 指针，指向新分配内存块的地址；如果分配失败（例如内存不足），返回一个空指针（NULL）。

参数说明：

size 是要分配的内存块的大小，以字节为单位。

代码示例 1：

动态分配整型数据的空间。

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p;
    p = (int *)malloc(sizeof(int));

    // 判断是否分配成功
    if (p == NULL)
    {
        printf("内存分配失败\n");
        return 1; // 退出程序
    }

    // 使用解引用赋值并输出
    *p = 120;
    printf("p 指向的地址: %p \n", p);
    printf("p 指向的值: %d", *p);

    // 释放分配的内存
    free(p);

    return 0;
```

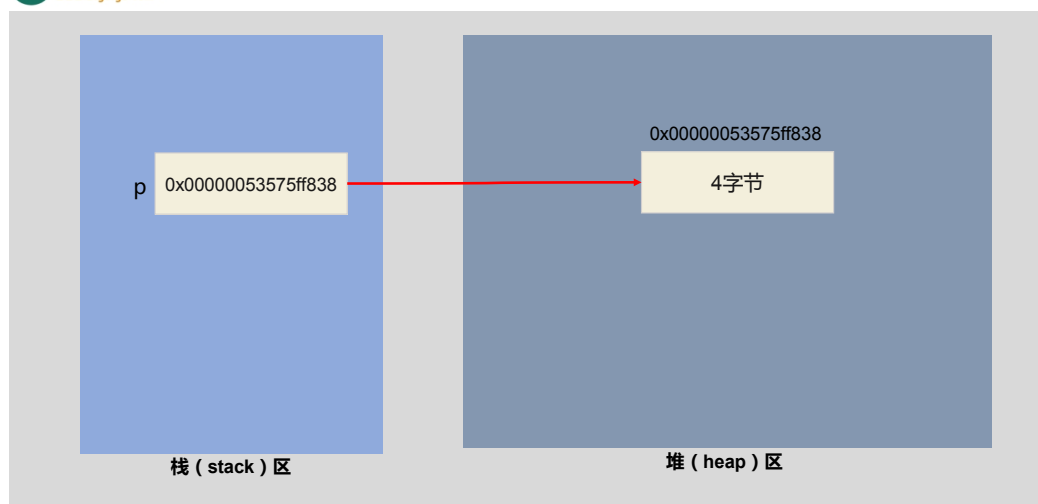
}

输出结果:

p 指向的地址: 00000053575ff838

p 指向的值: 120

内存分析:



代码示例 2:

动态分配数组空间。

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p;    // 定义整型指针
    int n = 5; // 定义数组长度

    // 动态分配内存，将地址赋给指针 p
    p = (int *)malloc(n * sizeof(int));

    // 判断是否分配成功
    if (p == NULL)
    {
        printf("内存分配失败\n");
        return 1; // 退出程序
    }

    // 给数组元素赋值
```

```
for (int i = 0; i < n; i++)
{
    p[i] = i * 10;
}

// 输出数组的元素
for (int i = 0; i < n; i++)
{
    printf("p[%d]=%d \n", i, p[i]);
}

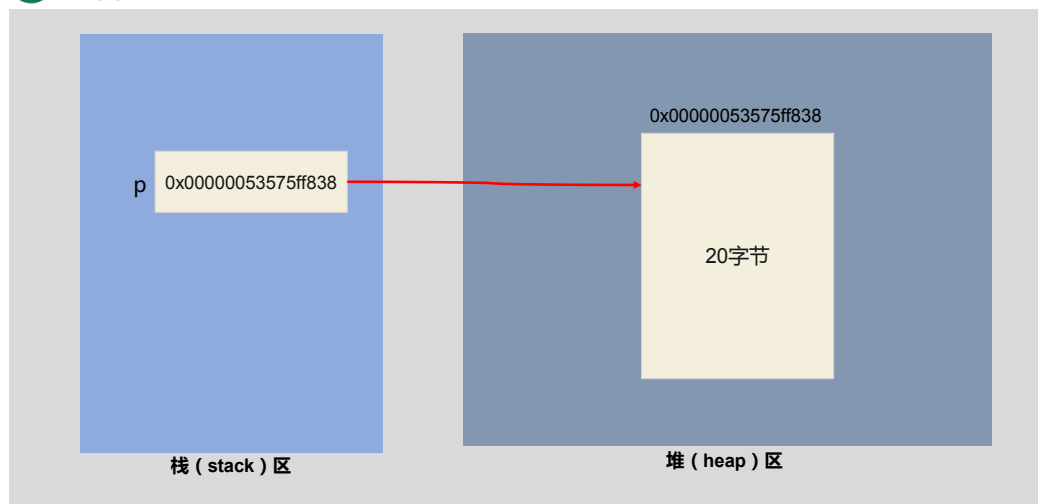
// 释放分配的内存
free(p);

return 0;
}
```

输出结果:

```
p[0]=0
p[1]=10
p[2]=20
p[3]=30
p[4]=40
```

内存分析:



2) calloc() 函数

`calloc()` 函数用于分配内存并将其初始化为零，它在分配内存块时会自动将内存中的每个字节都设置为零。

函数原型:

```
void *calloc(size_t numElements, size_t sizeofElement);
```

返回值说明:

如果内存分配成功, 返回一个 void 指针, 指向新分配内存块的地址; 如果分配失败 (例如内存不足), 返回一个空指针 (NULL)。

参数说明:

- (1) numElements 是要分配的元素的数量。
- (2) sizeofElement 是每个元素的大小 (以字节为单位)。

代码示例:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p;    // 定义整型指针
    int n = 5; // 定义数组长度

    // 动态分配内存, 将地址赋给指针 p
    p = (int *)calloc(n, sizeof(int));

    // 判断是否分配成功
    if (p == NULL)
    {
        printf("内存分配失败\n");
        return 1; // 退出程序
    }

    // 输出数组的元素的值
    for (int i = 0; i < n; i++)
    {
        printf("p[%d]=%d \n", i, p[i]);
    }

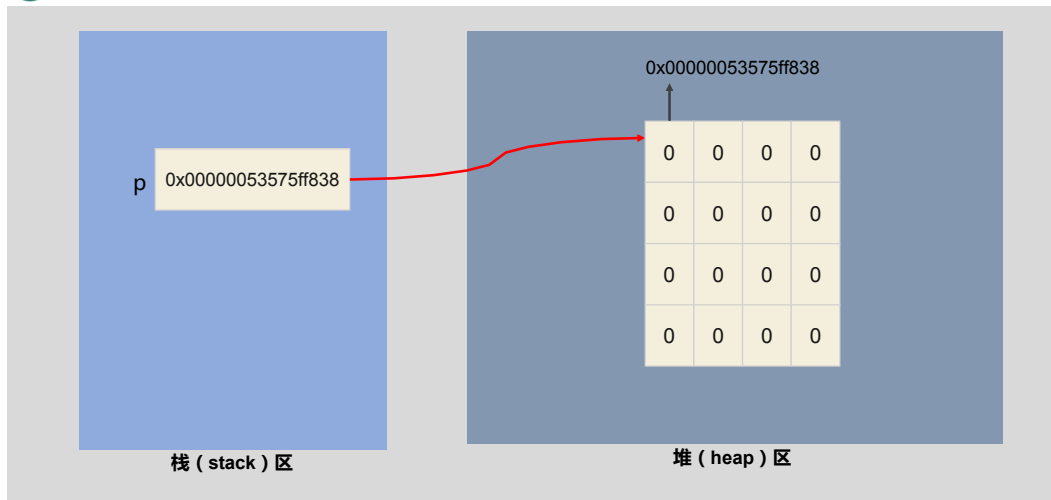
    // 释放分配的内存
    free(p);

    return 0;
}
```

输出结果:

```
p[0]=0
p[1]=0
p[2]=0
p[3]=0
p[4]=0
```

内存示意图：



3) realloc() 函数

realloc() 函数用于重新分配 malloc() 或 calloc() 函数所获得的内存块的大小。

函数原型：

```
void* realloc(void *ptr, size_t size);
```

返回值说明：

返回一个指向重新分配内存块的指针。如果内存重新分配成功，返回的指针可能与原始指针相同，也可能不同；如果内存分配失败，返回一个空指针（NULL）。

如果在原内存块上进行缩减，通常返回的原先的地址。

参数说明：

- （1）ptr 是要重新分配的内存块的指针。
- （2）size 是新的内存块的大小（以字节为单位）。

代码示例：

注意，本案例中使用了<malloc.h>头文件中的_msize()函数，该函数可以获取指定内存块的大小。

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <malloc.h>

int main()
{
    // 声明指针
    int *b;

    // 分配内存
    b = malloc(sizeof(int) * 100);
    printf("b=%p, size:%zu \n", b, _msize(b));

    // 调整内存大小
    b = realloc(b, sizeof(int) * 2000);
    printf("b=%p, size:%zu \n", b, _msize(b));

    // 再次调整内存大小
    b = realloc(b, sizeof(int) * 200);
    printf("b=%p, size:%zu \n", b, _msize(b));

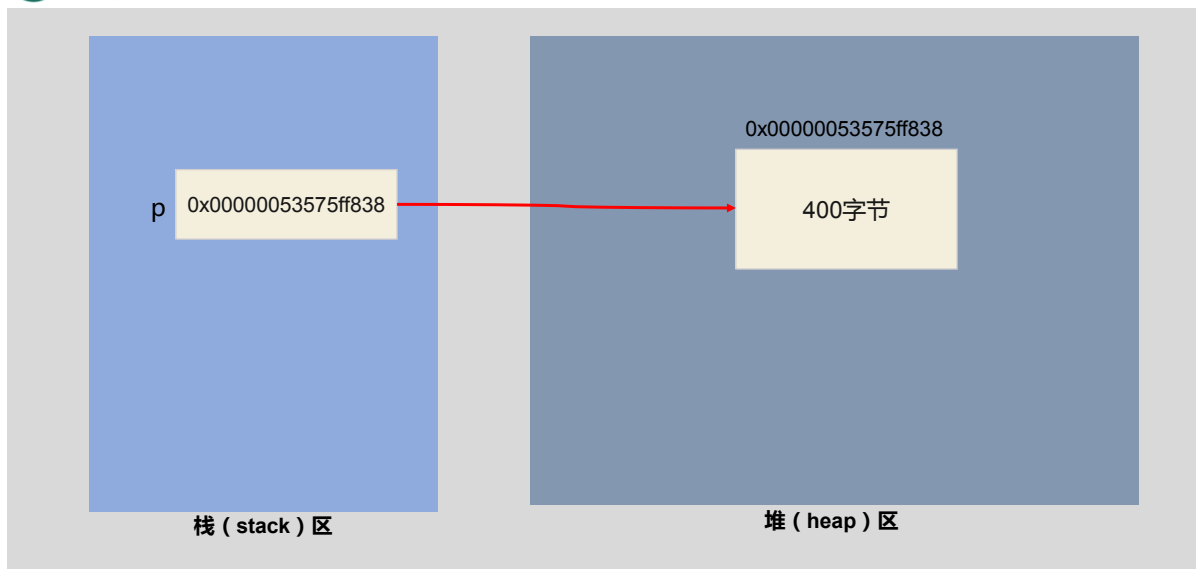
    // 释放内存
    free(b);

    return 0;
}
```

输出结果:

```
b=000001f5fc3f4770, size:400
b=000001f5fc3f47d0, size:8000
b=000001f5fc3f47d0, size:800
```

内存分析:



4) free() 函数

如果动态分配的内存空间没有被正确释放，这种情况称为内存泄漏，内存泄漏会导致系统中的可用内存逐渐减少，直到耗尽系统可用的内存资源。

free() 函数用于释放动态分配的内存，以便将内存返回给操作系统，防止内存泄漏。

函数原型：

```
void free(void *ptr);
```

返回值说明：

没有有返回值。

参数说明：

ptr 是指向要释放的内存块的指针，ptr 必须是 malloc() 或 calloc() 动态分配的内存块地址。

注意：

(1) 分配的内存块一旦释放，就不应该再次操作已经释放的地址，也不应该再次使用 free() 对该地址释放第二次。

(2) 如果忘记调用 free() 函数，会导致无法访问未回收的内存块，构成内存泄漏。

14.4 内存分配应用实例

动态创建数组，输入 5 个学生的成绩，再定义一个函数检测成绩低于 60 分的，输出不

合格的成绩。

```
#include <stdlib.h>
#include <stdio.h>

// 函数原型声明
void check(int *);

int main()
{
    int *p;
    // 在堆区开辟一个 5 * 4 的空间
    p = (int *)malloc(5 * sizeof(int));

    printf("请输入 5 个成绩: ");

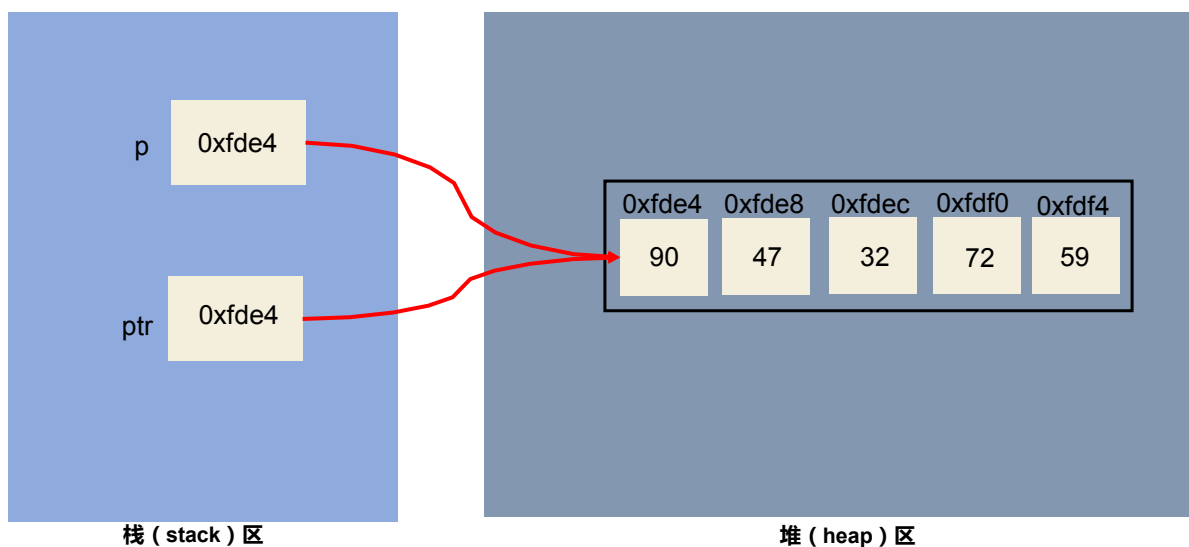
    for (int i = 0; i < 5; i++)
    {
        scanf("%d", p + i);
    }

    check(p);
    free(p); // 销毁 堆区 p 指向的空间

    return 0;
}

// 函数定义
void check(int *p)
{
    printf("\n不及格的成绩有: ");
    for (int i = 0; i < 5; i++)
    {
        if (p[i] < 60)
        {
            printf(" %d ", p[i]);
        }
    }
}
```

内存分析:



14.5 内存分配的基本原则

(1) 避免分配大量的小内存块。分配堆上的内存有一些系统开销，所以分配许多小的内存块比分配几个大内存块的系统开销大。

(2) 仅在需要时分配内存。只要使用完堆上的内存块，就需要及时释放它，否则可能出现内存泄漏。

(3) 总是确保释放已分配的内存。在编写分配内存的代码时，就要确定好在代码的什么地方释放内存。

第 15 章 预处理器

15.1 基本介绍

15.1.1 预处理器

C 语言编译器在编译程序之前，会先使用预处理器 (preprocessor) 处理代码，代码经过预处理之后再送入编译器进行编译。

预处理器的主要任务包括宏替换、文件包含、条件编译等。

15.1.2 预处理指令

预处理过程中会执行预处理指令，预处理指令以 # 号开头，用于指导预处理器执行不

同的任务。

预处理指令具有如下特点：

(1) 预处理指令应该放在代码的开头部分。

(2) 预处理指令都以 # 开头，指令前面可以有空白字符（比如空格或制表符），# 和指令的其余部分之间也可以有空格，但是为了兼容老的编译器，一般不留空格。

```
// 推荐写法
#include <stdio.h>
```

```
// 不推荐写法
#include <stdio.h>
# include <stdio.h>
```

(3) 预处理指令都是一行的，除非在行尾使用反斜杠，将其折行。

```
#include <std\
io.h>
```

(4) 预处理指令不需要分号作为结束符，指令结束是通过换行符来识别的。

```
#include <stdio.h>; // 这里有分号会报错
#define PI 3.14; // 分号会成为 PI 的值的一部分
```

(5) 预处理指令通常不能写在函数内部，有些编译器的扩展允许将预处理指令写在函数里，但强烈不建议这么干。

```
int main ()
{
    // 一般不允许写在这里
    #include <stdio.h>
    return 0;
}
```

15.2 宏定义

15.2.1 基本介绍

宏定义，就是用一个标识符（宏名称）来表示一个替换文本，如果在后面的代码中出现了宏名称，预处理器会将它替换为对应的文本，称为宏替换或宏展开。

宏定义的基本语法形式如下：

```
#define 宏名称 替换文本
```

宏名称：宏的名称，是一个标识符，通常使用大写字母表示，以便与变量名区分开来。

替换文本：宏名称在代码中的每次出现都会被替换为这段文本。

前面的案例中，我们曾经使用宏定义来定义常量和布尔类型。

15.2.2 使用宏定义常量

```
#include <stdio.h>

// 定义常量
#define PI 3.14

int main()
{
    // 定义变量保存半径，值通过用户输入获取
    double radius;
    printf("请输入半径: ");
    scanf("%lf", &radius);

    // 计算面积并输出
    printf("圆的面积: %.2f", radius * PI * PI);

    return 0;
}
```

上面代码中，使用宏定义声明了 N，后面代码中出现了 N，会替换成 100。

15.2.3 使用宏定义数据类型

```
#include <stdio.h>

// 宏定义
#define BOOL int
#define TRUE 1
#define FALSE 0

int main()
{
    // 使用整型表示真假两种状态
    // int isPass = 0;
    // int isOk = 1;

    // 借助于宏定义
    BOOL isPass = FALSE;
    BOOL isOk = TRUE;

    if (isPass)
    {
        printf("Pass");
    }
}
```

```

    if (isOk)
    {
        printf("Ok");
    }

    return 0;
}

```

上面代码中使用宏定义声明了 BOOL、TURE、FALSE，后面代码中出现 BOOL 会替换成 int，出现 TRUE 会替换成 1，出现 FALSE 替换成 0。

15.2.4 宏定义的替换文本

替换文本可以含任何字符，它可以是字面量、表达式、if 语句、函数等，预处理程序对它不作任何检查，直接进行文本替换，如有错误，只能在编译已被宏展开后的源程序时发现。

代码示例：

```

#include <stdio.h>
#define M (n * n + 3 * n)
#define PRINT_SUM printf("sum=%d", sum)

int main()
{
    int n = 3;
    int sum = 3 * M + 4 * M + 5 * M; // 宏展开
    3*(n*n+3*n)+4*(n*n+3*n)+5*(n*n+3*n);
    PRINT_SUM; // 宏展开 printf("sum=%d", sum)

    return 0;
}

```

输出结果：

```
sum=216
```

15.2.5 宏定义嵌套

宏定义允许嵌套，在宏定义的替换文本中可以使用已经定义的宏名，在宏展开时由预处理程序层层替换。

代码示例：

```

#include <stdio.h>
#define PI 3.1415926
#define S PI *y *y

```

```
int main()
{
    int y = 2;
    printf("%f", S); // 宏替换变为 printf("%f", 3.1415926*y*y);

    return 0;
}
```

输出结果:

```
12.566370
```

15.2.6 取消宏定义

如要取消宏定义使用#undef 命令。

```
#include <stdio.h>
#define PI 3.14159

int main()
{
    printf("PI=%f", PI);
    return 0;
}

#undef PI // 取消宏定义
void func()
{
    // printf("PI=%f", PI); //错误,这里不能使用到PI了
}
```

输出结果:

```
PI=3.141590
```

15.3 带参数的宏定义

15.3.1 基本介绍

C 语言允许宏带有参数。在宏定义中的参数称为“形式参数”，在宏调用中的参数称为“实际参数”，这点和函数有些类似。

对带参数的宏，在展开过程中不仅要进行文本替换，还要用实参去替换形参。

带参宏定义的一般形式为：

```
#define 宏名称(形参列表) 可以包含形参的替换文本
```

带参宏调用的一般形式为：

```
宏名称(实参列表);
```

代码示例：

```
#include <stdio.h>

// 定义带参数的宏定义取两个数字中较大的一共
#define MAX(a, b) a > b ? a : b

int main()
{

    printf("两个数字中较大的: %d \n", MAX(101, 451));
    return 0;
}
```

输出结果：

451

15.3.2 注意事项

(1) 带参宏定义中，形参之间可以出现空格，但是宏名和形参列表之间不能有空格出现。

(2) 在带参宏定义中，不会为形式参数分配内存，因此不必指明数据类型，而在宏调用中，实参包含了具体的数据，要用它们去替换形参，因此实参必须要指明数据类型。

(3) 在宏定义中，替换文本内的形参通常要用括号括起来以避免出错。

代码示例：

```
#include <stdio.h>

// 定义带参数的宏定义计算某数字平方，尽量在替换文本中用()把形参括起来
#define SQ1(n) n *n
#define SQ2(n) (n) * (n)

int main()
{
    int a = 7;
    printf("a 的平方: %d \n", SQ1(a + 1)); // 执行宏替换 a + 1 * a + 1
    printf("a 的平方: %d \n", SQ2(a + 1)); // 执行宏替换 (a + 1) * (a + 1)

    return 0;
}
```

输出结果：

a 的平方: 15
a 的平方: 64

15.3.3 带参宏定义和函数的区别

(1) 宏展开仅仅是文本的替换，不会对表达式进行计算；宏在编译之前就被处理掉了，它没有机会参与编译，也不会占用内存。

(2) 函数是一段可以重复使用的代码，会被编译，会给它分配内存，每次调用函数，就是执行这块内存中的代码。

案例：分别使用函数和带参数的宏输出 1 到 5 所有数字的立方：

```
#include <stdio.h>

// 定义宏定义 实现某个数字的立方
#define CUBE(n) (n) * (n) * (n)

// 定义函数 返回某个数字的立方
int cube(int n)
{
    return n * n * n;
}

int main()
{
    // 使用函数输出 1 到 5 所有数字的立方
    printf("函数: \n");
    int i = 1;
    while (i <= 5)
    {
        printf("%d \n", cube(i++));
    }

    // 使用带参数的宏定义输出 1 到 5 所有数字的立方
    printf("\n 带参数的宏定义: \n");
    int j = 1;
    while (j <= 5)
    {
        printf("%d \n", CUBE(j++)); // 文本替换 (i++) * (i++) * (i++)
    }

    return 0;
}
```

输出结果：

函数：

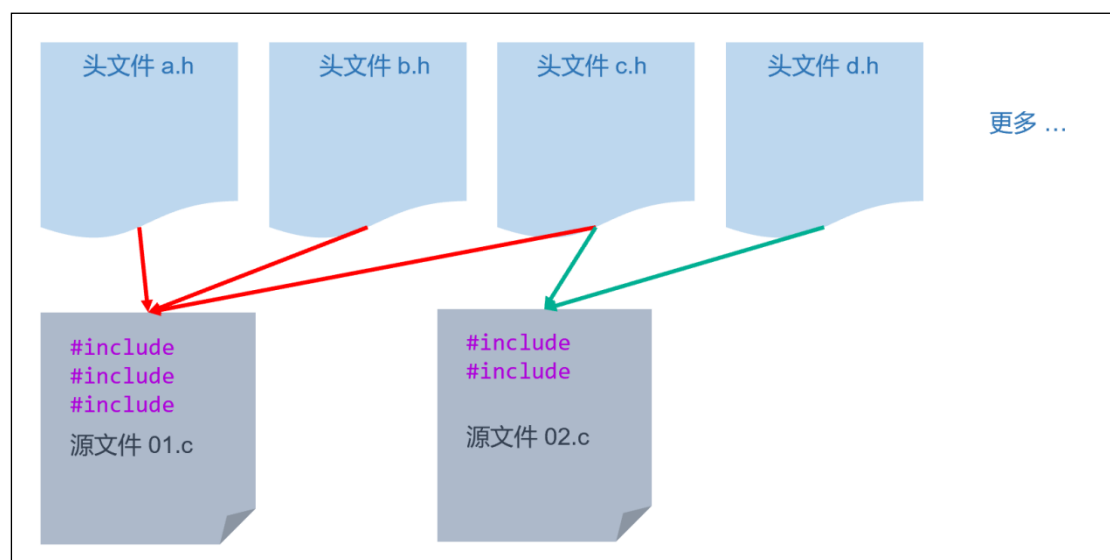
```
1
8
27
64
125
```

带参数的宏定义：

```
6
120
```

15.4 文件包含

`#include` 指令用于引入标准库头文件、自定义头文件或其他外部源代码文件，以便在当前源文件中使用其中定义的函数、变量、宏等内容。



一个源文件可以导入多个头文件，一个头文件也可以被多个源文件导入。

标准库头文件、自定义头文件的扩展名都是 `.h`。

15.4.1 包含标准库头文件

标准库头文件是系统提供的头文件，直接引入即可，像我们前面用过的 `stdio.h`、`stdlib.h`、`string.h`、`time.h` 等。引入标准库头文件需要使用尖括号将文件名包裹起来，格式如下：

```
#include <头文件名.h>
```

15.4.2 包含自定义头文件

自定义头文件的文件名写在双引号里面，格式如下：

```
#include "文件名.h"
```

建议把所有的常量、宏、系统全局变量和函数原型写在自定义的头文件中，在需要的时候随时引用这些头文件。

1) 使用相对路径

如果自定义的头文件在源文件的同级目录或源文件所在目录的下级目录，使用 `./` 开头的路径，`./` 可以省略。

如果自定义的头文件在源文件所在目录的上级或者更上级，使用 `../` 开头的路径。

参照以下目录结构：

```
├─ myheader05.h
└─ project
    ├─ inc
    │   └─ myheader04.h
    ├─ myheader03.h
    └─ src
        ├─ includes
        │   └─ myheader02.h
        ├─ main.c
        └─ myheader01.h
```

如果要在源文件 `main.c` 中引入以上自定义的头文件，需按照如下写法：

```
#include "myheader01.h" // 等价于 #include "./myheader01.h"
#include "includes/myheader02.h" // 等价于 #include "./myheader01.h"
#include "../myheader03.h"
#include "../inc/myheader04.h"
#include "../../myheader05.h"
```

需要注意的是，建议将头文件放置在源文件所在目录或子目录中，以保持项目的组织结构清晰。

2) 使用绝对路径

绝对路径是文件在文件系统中的完整路径，它从文件系统的盘符（Windows 系统）或根目录（Linux 系统、MacOS 系统）开始，沿着文件系统的目录结构一直到达目标文件。

Windows 系统中使用绝对路径引入自定义头文件，示例如下：

```
#include "C:\\Preparation\\Embedded\\01CLang\\code\\project\\foo.h"
```

Linux 系统或 MacOS 系统中使用绝对路径引入自定义头文件，示例如下：

```
#include "/usr/local/lib/foo.h"
```

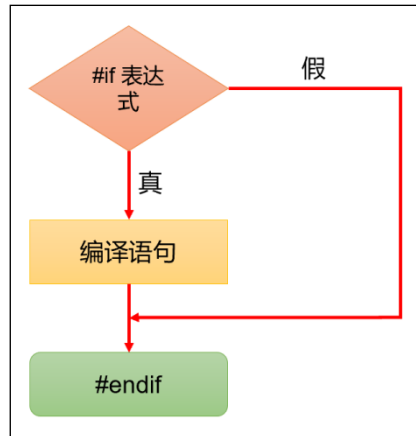
需要注意的是，通常我们建议使用相对路径引入自定义头文件，因为相对路径更加灵活和可移植。

15.5 条件编译

15.5.1 #if

1) #if ... #endif

#if...#endif 指令用于预处理器的条件判断，满足条件时，内部的行会被编译，否则就被编译器忽略。



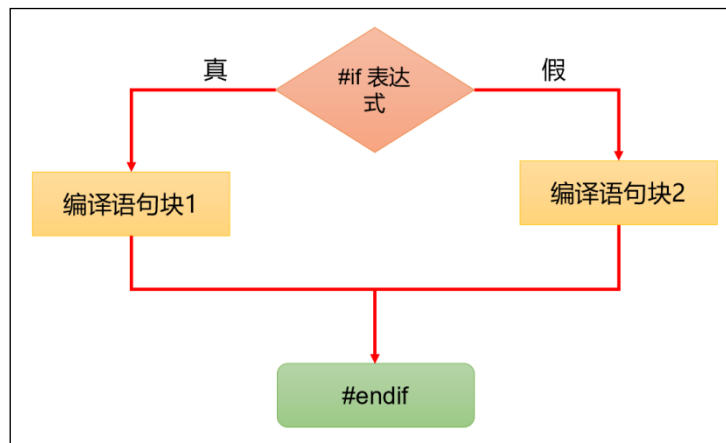
```
#if 0
    const double pi = 3.1415; // 不会执行
#endif
```

上面示例中，#if 后面的 0，表示判断条件不成立。所以，内部的变量定义语句会被编译器忽略。#if 0 这种写法常用来当作注释使用，不需要的代码就放在 #if 0 里面。

#if 后面的判断条件，通常是一个表达式。如果表达式的值不等于 0，就表示判断条件为真，编译内部的语句；如果表达式的值等于 0，表示判断条件为伪，则忽略内部的语句。

2) #if ... #else ... #endif

#if...#endif 之间还可以加入 #else 指令，用于指定判断条件不成立时，需要编译的语句。



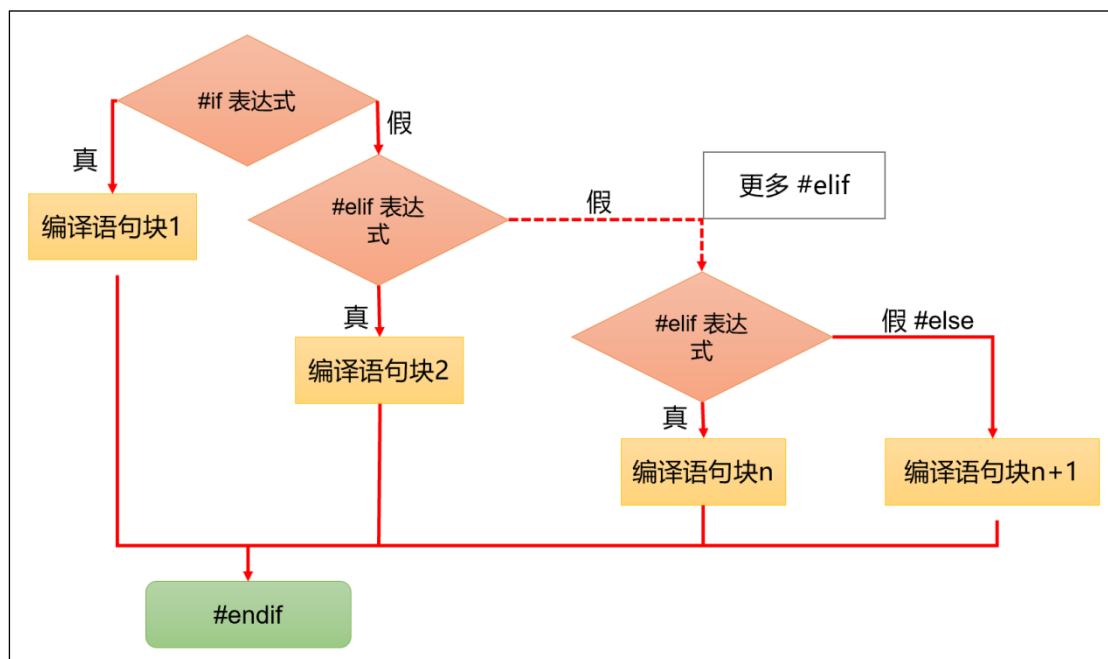
```
#define FOO 1
```

```
#if FOO
    printf("defined\n");
#else
    printf("not defined\n");
#endif
```

上面示例中，宏 FOO 如果定义过，会被替换成 1，从而输出 defined，否则输出 not defined。

3) #if ... #elif ... #else ... #endif

如果有多个判断条件，还可以加入 #elif 命令。



```
#if HAPPY_FACTOR == 0
    printf("I'm not happy!\n");
#elif HAPPY_FACTOR == 1
    printf("I'm just regular\n");
#else
    printf("I'm extra happy!\n");
#endif
```

上面示例中，通过 #elif 指定了第二重判断。注意，#elif 的位置必须在 #else 之前。如果多个判断条件皆不满足，则执行 #else 的部分。

没有定义过的宏，等同于 0。因此如果 UNDEFINED 是一个没有定义过的宏，那么 #if UNDEFINED 为伪，而 #if !UNDEFINED 为真。

15.5.2 #ifdef

`#ifdef...#endif` 指令用于判断某个宏是否定义过。

有时源码文件可能会重复加载某个库，为了避免这种情况，可以在库文件里使用 `#define` 定义一个空的宏。通过这个宏，判断库文件是否被加载了。

```
#define EXTRA_HAPPY
```

上面示例中，`EXTRA_HAPPY` 就是一个空的宏。

然后，源码文件使用 `#ifdef...#endif` 检查这个宏是否定义过。

```
#ifdef EXTRA_HAPPY
    printf("I'm extra happy!\n");
#endif
```

上面示例中，`#ifdef` 检查宏 `EXTRA_HAPPY` 是否定义过。如果已经存在，表示加载过库文件，就会打印一行提示。

`#ifdef` 可以与 `#else` 指令配合使用。

```
#ifdef EXTRA_HAPPY
    printf("I'm extra happy!\n");
#else
    printf("I'm just regular\n");
#endif
```

上面示例中，如果宏 `EXTRA_HAPPY` 没有定义过，就会执行 `#else` 的部分。

`#ifdef...#else...#endif` 可以用来实现条件加载。

```
#ifdef MAVIS
    #include "foo.h"
    #define STABLES 1
#else
    #include "bar.h"
    #define STABLES 2
#endif
```

上面示例中，通过判断宏 `MAVIS` 是否定义过，实现加载不同的头文件。

15.5.3 #if defined

上一节的 `#ifdef` 指令，等同于 `#if defined`。

```
#ifdef FOO
// 等同于
#if defined FOO
```

上面示例中，`defined` 是一个预处理运算符，如果它的参数是一个定义过的宏，就会返回 1，否则返回 0。

使用这种语法，可以完成多重判断。

```
#if defined FOO
    x = 2;
#elif defined BAR
    x = 3;
#else
    x = 4;
#endif
```

15.5.4 #ifndef

`#ifndef...#endif` 指令跟 `#ifdef...#endif` 正好相反。它用来判断，如果某个宏没有被定义过，则执行指定的操作。

```
#ifdef EXTRA_HAPPY
    printf("I'm extra happy!\n");
#endif

#ifndef EXTRA_HAPPY
    printf("I'm just regular\n");
#endif
```

上面示例中，针对宏 `EXTRA_HAPPY` 是否被定义过，`#ifdef` 和 `#ifndef` 分别指定了两种情况各自需要编译的代码。

`#ifndef` 常用于防止重复加载。举例来说，为了防止头文件 `myheader.h` 被重复加载，可以把它放在 `#ifndef...#endif` 里面加载。

```
#ifndef MYHEADER_H
    #define MYHEADER_H
    #include "myheader.h"
#endif
```

上面示例中，宏 `MYHEADER_H` 对应文件名 `myheader.h` 的大写。只要 `#ifndef` 发现这个宏没有被定义过，就说明该头文件没有加载过，从而加载内部的代码，并会定义宏 `MYHEADER_H`，防止被再次加载。

15.5.5 应用案例

1) 具体要求

开发一个 C 语言程序，让它暂停 5 秒以后再输出内容 "hello, 尚硅谷!~"，并且要求跨平台，在 Windows 和 Linux 下都能运行，如何处理。

2) 提示

(1) Windows 平台下的暂停函数的原型是 `void Sleep(DWORD dwMilliseconds)`, 参数的单位是“毫秒”, 位于 `<windows.h>` 头文件。

(2) Linux 平台下暂停函数的原型是 `unsigned int sleep (unsigned int seconds)`, 参数的单位是“秒”, 位于 `<unistd.h>` 头文件。

3) 代码演示

```
#include <stdio.h>

#if _WIN32    //如果是 windows 平台, 就引入 <windows.h>
#include <windows.h>
#define SLEEP(t) Sleep(t * 1000)
#elif __linux__ // 如果是 linux 平台, 就引入<unistd.h>
#include <unistd.h>
#define SLEEP sleep
#endif

int main()
{
    SLEEP(5);
    printf("hello, 尚硅谷~");

    return 0;
}
```

15.6 预处理命令总结

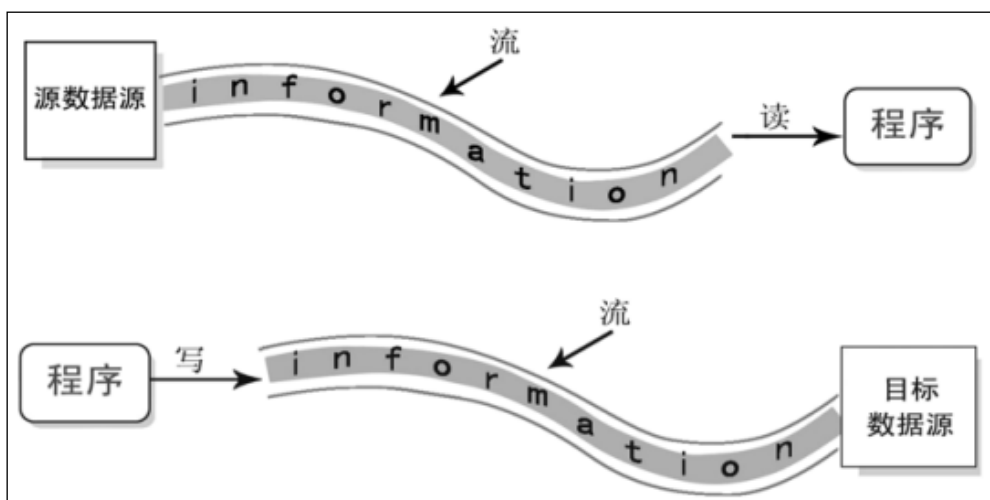
指令	说明
<code>#include</code>	包含一个源代码文件
<code>#define</code>	定义宏
<code>#undef</code>	取消已定义的宏
<code>#if</code>	如果给定条件为真, 则编译下面代码
<code>#ifdef</code>	如果宏已经定义, 则编译下面代码
<code>#ifndef</code>	如果宏没有定义, 则编译下面代码
<code>#elif</code>	如果前面的 <code>#if</code> 给定条件不为真, 当前条件为真, 则编译下面代码
<code>#endif</code>	结束一个 <code>#if.....#else</code> 条件编译块

第 16 章 IO 操作

16.1 基本介绍

“Input/Output” 简称 “IO”, 也就是 “输入/输出”。

C 程序中, 数据的输入/输出操作以“流”的方式进行, 可以看做是一种数据的流动。



- **输入流 (Input Stream)：**数据从数据源（例如文件、键盘等）到程序（内存）的传输路径。
- **输出流 (Output Stream)：**数据从程序（内存）到数据源（例如文件、屏幕等）的传输路径。

16.2 终端输入输出

16.2.1 标准文件

C 语言把所有的设备都当作文件。

在程序启动时，会自动打开 `stdin`、`stdout` 这两个文件指针，分别指向键盘和屏幕。这两个文件指针是标准库头文件“`stdio.h`”中定义的全局变量。`stdin` 用于获取键盘输入，`stdout` 用于输出到屏幕。

标准文件	文件指针	设备
标准输入	<code>stdin</code>	键盘
标准输出	<code>stdout</code>	屏幕

16.2.2 `scanf()`和 `printf()`函数

C 语言中的 I/O （输入/输出）通常使用 `printf()`和 `scanf()`两个函数，`scanf()`函数从标准输入流 `stdin` 读取输入，`printf()`函数把输出写入到标准输出流 `stdout`。这两个函数我们在前面的案例中已经大量使用。

案例演示：从键盘获取输入并将内容再输出

```
#include <stdio.h>
```

```
int main()
{
    // 定义变量
    char str[100];
    int i;

    // 获取输入
    printf("Please Input A String And A Number :");
    scanf("%s %d", str, &i);

    // 输出到屏幕终端
    printf("You entered: %s %d ", str, i);
    printf("\n");

    return 0;
}
```

16.2.3 getchar() & putchar() 函数

1) getchar 函数

getchar() 函数用于从标准输入流（通常是键盘输入）读取一个字符。这个函数在同一个时间内只会读取一个单一的字符。

函数原型：

```
int getchar(void);
```

返回值说明：

返回一个整数值，表示读取的字符。

2) putchar 函数

putchar()函数用于将一个字符写入到标准输出流（通常是屏幕）。这个函数在同一个时间内只会输出一个单一的字符。

函数原型：

```
int putchar(int character);
```

返回值说明：

返回一个整数值，是写入的字符的 ASCII 码值。

参数说明：

character 是要写入的字符的 ASCII 码值。

3) 代码示例

```
#include <stdio.h>
```

```
int main( )
{
    int c;
    printf( "Enter a value :");
    c = getchar();

    printf( "\nYou entered: ");
    putchar(c);
    printf("\n");

    return 0;
}
```

16.2.4 gets()和 puts() 函数

1) gets 函数

gets() 函数用于从标准输入流（通常是键盘）读取一行文本，并将其存储在一个字符数组中（gets() 的参数），直到遇到换行符为止。

函数原型：

```
char *gets(char *str);
```

返回值说明：

返回一个指向存储在 str 中的字符串的指针

参数说明：

字符数组（字符串）的指针，用于存储读取的输入数据。

2) puts 函数

puts() 函数用于将字符串输出到标准输出流（通常是屏幕），并自动添加换行符。它接受一个字符串作为参数，然后将其显示在屏幕上。

函数原型：

```
int puts(const char *str);
```

返回值说明：

返回成功写入的字符数，如果写入失败或出现错误，它会返回 特殊值 EOF（（EOF 是一个定义在 stdio.h 头文件中的常量，值通常为 -1））。

参数说明：

str 是要输出的字符串。

3) 代码示例

```
#include <stdio.h>

int main()
{
    char str[20];

    printf("Enter a line of text :");
    gets(str);

    printf("You entered: ");
    // printf("%s", str);
    puts(str);

    return 0;
}
```

16.3 文件操作

文件，无论是文本文件还是二进制文件，都以一系列字节的形式存在，C 语言提供了一系列函数处理存储设备上的文件。

16.3.1 打开文件

使用 `fopen()` 函数来创建一个新的文件或者打开一个已有的文件。该函数位于标准库的 `stdio.h` 头文件中。

函数原型：

```
FILE *fopen( const char *filename, const char *mode );
```

返回值说明：

返回一个指向 `FILE` 结构的指针，该结构表示文件流，可以使用这个指针来进行文件的读取和写入操作。如果打开文件失败，返回一个空指针（`NULL`）。

参数说明：

（1）`filename` 是一个以字符串形式指定的文件名，表示要打开的文件的名称，可以包括文件路径和文件名。

（2）`mode` 是一个以字符串形式指定的打开模式，表示打开文件的方式，可以是以下值之一：

模式	描述
r	只读模式。 打开一个已有的文本文件，只允许读取文件。
w	只写模式。

	打开一个文本文件，从头写入文件。如果文件不存在，则会创建一个新文件并写入；如果文件存在，则清空文件并从头写入。
a	追加模式。 打开一个文本文件，追加写入文件。如果文件不存在，则会创建一个新文件并写入；如果文件存在，在已有内容后面追加写入。
rb	只读二进制模式。 打开一个已有的二进制文件，只允许读取文件。
wb	只写二进制模式。 打开一个二进制文件，从头写入文件。如果文件不存在，则会创建一个新文件并写入；如果文件存在，则清空文件并从头写入。
ab	追加二进制模式。 打开一个二进制文件，追加写入文件。如果文件不存在，则会创建一个新文件并写入；如果文件存在，在已有内容后面追加写入。
r+	读写模式。 打开一个文本文件，允许读写文件。
w+	读写模式。 打开一个文本文件，从头读写文件。如果文件不存在，则会创建一个新文件并读写；如果文件存在，则清空文件并从头读写。
a+	读写模式。 打开一个文本文件，读取或追加写入文件。如果文件不存在，则会创建一个新文件并读写；如果文件存在，读取或在已有内容后面追加写入。
rb+	读写二进制模式。 打开一个二进制文件，允许读写文件。
r+b	
wb+	读写二进制模式。 打开一个二进制文件，从头读写文件。如果文件不存在，则会创建一个新文件并读写；如果文件存在，则清空文件并从头读写。
w+b	
ab+	读写二进制模式。 打开一个二进制文件，读取或追加写入文件。如果文件不存在，则会创建一个新文件并读写；如果文件存在，读取或在已有内容后面追加写入。
a+b	

16.3.2 关闭文件

文件读写完毕后，一定要关闭文件，使用 `fclose()` 函数可以关闭文件。该函数位于标准库的 `stdio.h` 头文件中。

函数原型：

```
int fclose(FILE *stream);
```

返回值说明：

返回一个整数值，通常为零(0)，表示关闭操作成功。如果关闭失败，它返回特殊值 EOF (EOF 是一个定义在 `stdio.h` 头文件中的常量，值通常为 -1)。

参数说明：

stream 是一个指向 FILE 结构的指针，表示要关闭的文件流。

16.3.3 写入文件

1) fputc 函数

fputc() 函数用于逐字符写入文件。位于标准库的 stdio.h 头文件中。

函数原型：

```
int fputc(int character, FILE *stream);
```

返回值说明：

返回一个整数值，通常是写入的字符的 ASCII 码值。如果写入成功，返回的值与输入的 character 值相同；如果写入失败，它返回特殊值 EOF。

参数说明：

(1) character 是要写入的字符，通常以整数形式表示，即字符的 ASCII 码值。

(2) stream 是一个指向 FILE 结构的指针，表示要写入字符的文件流。

代码示例：

```
#include <stdio.h>

int main()
{
    // FILE *file = fopen("output.txt", "w"); // 从头写入
    FILE *file = fopen("output.txt", "a"); // 追加写入

    if (file != NULL)
    {
        // 使用 fputc 写入一个字符
        char ch = 'A';
        int result1 = fputc(ch, file);
        if (result1 != EOF)
        {
            printf("成功使用 fputc 写入字符 '%c', 返回值: %d.\n", ch,
result1);
        }
        else
        {
            printf("使用 fputc 写入字符时出现错误.\n");
        }

        // 关闭文件
        fclose(file);
    }
}
```

```
    }  
    else  
    {  
        printf("打开文件时出现错误。\\n");  
    }  
  
    return 0;  
}
```

2) fputs 函数

fputs() 函数用于将字符串写入文件。位于标准库的 stdio.h 头文件中。

函数原型:

```
int fputs(const char *str, FILE *stream);
```

返回值说明:

返回一个整数值，如果写入成功，则返回非负整数（通常是成功写入的字符数，具体取决于编译器），否则返回特殊值 EOF。

参数说明:

(1) str 是要写入的字符串，通常以 const char* 指针的形式传递。

(2) stream 是指向输出流的指针，通常是文件指针。

代码示例:

```
#include <stdio.h>  
  
int main()  
{  
    FILE *file = fopen("output.txt", "w"); // 从头写入  
    // FILE *file = fopen("output.txt", "a"); // 追加写入  
  
    if (file != NULL)  
    {  
        // 使用 fputs 写入一个字符串  
        const char *text = "fputs: Hello Shangguigu\\n";  
        int result2 = fputs(text, file);  
        if (result2 != EOF)  
        {  
            printf("成功使用 fputs 写入字符串，返回值: %d。\\n", result2);  
        }  
        else  
        {  
            printf("使用 fputs 写入字符串时出现错误。\\n");  
        }  
    }  
}
```

```
        // 关闭文件
        fclose(file);
    }
    else
    {
        printf("打开文件时出现错误。\\n");
    }

    return 0;
}
```

3) fprintf 函数

fprintf() 函数用于格式化写入文件。位于标准库的 stdio.h 头文件中。

函数原型：

```
int fprintf(FILE *stream, const char *format, ...);
```

返回值说明：

返回一个整数值，如果写入成功，则返回非负整数（通常是成功写入的字符数，具体取决于编译器），否则返回特殊值 EOF。

参数说明：

- (1) stream 是一个指向 FILE 结构的指针，表示要写入的文件流。
- (2) format 是一个格式化字符串，类似于 printf() 函数中的格式化字符串。
- (3) ...表示可变数量的参数，根据格式化字符串中的格式占位符对应。

代码示例：

```
#include <stdio.h>

int main()
{
    FILE *file = fopen("output.txt", "w"); // 从头写入
    // FILE *file = fopen("output.txt", "a"); // 追加写入

    if (file != NULL)
    {
        // 使用 fputs 写入一个字符串
        const char *text = "fprintf: %s is studying in Shangguigu\\n";
        char *name = "Alice";
        int result3 = fprintf(file, text, name);
        if (result3 != EOF)
        {
```

```
        printf("成功使用 fprintf 写入字符串，返回值: %d。\\n",
result3);
    }
    else
    {
        printf("使用 fprintf 写入字符串时出现错误。\\n");
    }

    // 关闭文件
    fclose(file);
}
else
{
    printf("打开文件时出现错误。\\n");
}

return 0;
}
```

4) 代码演示:

打开文本文件，并分别使用 fputc()、fputs()、fprintf() 写入内容。

```
#include <stdio.h>

int main()
{
    // FILE *file = fopen("output.txt", "w"); // 从头写入
    FILE *file = fopen("output.txt", "a"); // 追加写入

    if (file != NULL) {
        // 使用 fputc 写入一个字符
        char ch = 'A';
        int result1 = fputc(ch, file);

        if (result1 != EOF) {
            printf("成功使用 fputc 写入字符 '%c'，返回值: %d。\\n", ch,
result1);
        } else {
            printf("使用 fputc 写入字符时出现错误。\\n");
        }

        // 使用 fputs 写入一个字符串
        const char *text = "fputs: Hello Shangguigu\\n";
        int result2 = fputs(text, file);
    }
```

```
    if (result2 != EOF)
    {
        printf("成功使用 fputs 写入字符串, 返回值: %d。\\n", result2);
    } else {
        printf("使用 fputs 写入字符串时出现错误。\\n");
    }

    // 使用 fprintf 写入格式化数据
    int num = 42;
    float pi = 3.14159;
    const char *formattedText = "fprintf: %d %.2f\\n";
    int result3 = fprintf(file, formattedText, num, pi);

    if (result3 >= 0) {
        printf("成功使用 fprintf 写入格式化数据, 返回值: %d。\\n",
result3);
    } else {
        printf("使用 fprintf 写入格式化数据时出现错误。\\n");
    }

    fclose(file); // 关闭文件
} else {
    printf("打开文件时出现错误。\\n");
}

return 0;
}
```

16.3.4 读取文件

1) fgetc 函数

fgetc() 函数用于从文件中逐字符读取。位于标准库的 stdio.h 头文件中。

函数原型:

```
int fgetc(FILE *stream);
```

返回值说明:

如果读取成功, 它返回所读取字符的 ASCII 码值 (0-255 之间的整数), 如果到达文件结束或发生错误, 它返回特殊值 EOF。

参数说明:

stream 是一个指向 FILE 结构的指针, 表示要写入字符的文件流。

代码示例:

```
#include <stdio.h>

int main()
{
    FILE *file = fopen("output.txt", "r"); // 打开文件以供读取

    if (file != NULL)
    {
        int ch; // 用于存储读取的字符

        // 使用 fgetc 逐字符读取文件
        while ((ch = fgetc(file)) != EOF)
        {
            printf("%c", ch); // 将字符显示在屏幕上
        }

        fclose(file); // 关闭文件
    }
    else
    {
        printf("Error opening the file.\n");
    }

    return 0;
}
```

2) fgetc 函数

fgetc() 函数从文件中逐行读取，遇到换行符即读取结束，读取的内容中包含换行符。该函数位于标准库的 `stdio.h` 头文件中。

函数原型：

```
char *fgetc(char *str, int num, FILE *stream);
```

返回值说明：

如果读取成功，它返回指向 `str` 的指针；如果到达文件结束或发生错误，它返回一个空指针（`NULL`）。

参数说明：

- (1) `str` 是一个指向字符数组的指针，用于存储读取的字符串。
- (2) `num` 是要读取的最大字符数（包括字符串终止符 `\0`），通常是 `str` 的长度。
- (3) `stream` 是一个文件流，通常是标准输入流（`stdin`）或其他文件流，用于指定从哪里读取数据。

代码示例：

```
#include <stdio.h>

int main()
{
    FILE *file = fopen("output.txt", "r"); // 打开文件以供读取

    if (file != NULL)
    {
        char buffer[256]; // 用于存储读取的字符串

        // 使用 fgets 逐行读取文件
        while (fgets(buffer, sizeof(buffer), file) != NULL)
        {
            printf("%s", buffer); // 将读取的字符串显示在屏幕上
        }

        fclose(file); // 关闭文件
    }
    else
    {
        printf("Error opening the file.\n");
    }

    return 0;
}
```

3) fscanf 函数

fscanf() 函数用于从文件中解析数据并存储到变量中，使用空白字符（空格、制表符、换行符等）分隔内容赋值给不同的变量。该函数位于标准库的 `stdio.h` 头文件中。

函数原型：

```
int fscanf(FILE *stream, const char *format, ...);
```

返回值说明：

返回成功读取和分配的参数数目，如果没有成功读取任何参数，它返回 0；如果读取过程中发生错误，它返回特殊值 EOF。

参数说明：

- (1) stream 是一个指向 FILE 结构的指针，表示要从中读取数据的文件流。
- (2) format 是一个格式化字符串，类似于 printf() 函数中的格式化字符串。
- (3) ... 表示可变数量的参数，根据格式化字符串中的格式指定要存储数据的变量。

代码示例：

```
#include <stdio.h>

int main()
{
    FILE *file = fopen("example.txt", "r"); // 打开文件以供读取

    if (file != NULL)
    {
        // 变量
        char msg1[100], msg2[100], msg3[100];
        int num;

        // 读取
        fscanf(file, "%s %s %s %d", msg1, msg2, msg3, &num);

        // 输出
        printf("%s\n%s\n%s\n%d", msg1, msg2, msg3, num);

        // 关闭文件
        fclose(file);
    }
    else
    {
        printf("打开文件时出现错误。\\n");
    }

    return 0;
}
```

第 17 章 附录

17.1 缓冲区换行符残留

C 语言中，输入缓冲区是用于存储从输入设备（通常是键盘）读取的数据的临时存储区域。当使用诸如 `scanf` 或 `fgets` 等函数从输入设备中读取数据时，数据会首先存储在输入缓冲区中，然后由程序进行处理。

使用 `scanf` 等函数读取数据时，换行符通常会留在输入缓冲区中。如果使用 `scanf` 函数获取字符，可能会出现意外的行为，因为 `scanf` 会读取到之前留在缓冲区中的换行符。

执行如下代码：

```
#include <stdio.h>
```

```
int main()
{
    // 定义两个 int 类型变量
    int num1, num2;
    // 定义两个 char 类型变量
    char ch1, ch2;

    // 获取用户输入，为变量进行初始化初始
    printf("请输入第一个数字: ");
    scanf("%d", &num1);

    printf("请输入第二个数字: ");
    scanf("%d", &num2);

    printf("请输入第一个字符: ");
    scanf("%c", &ch1);

    printf("请输入第二个字符: ");
    scanf("%c", &ch2);

    // 打印
    printf("num1=%d, num2=%d \n", num1, num2);
    printf("ch1=%c, ch2=%c \n", ch1, ch2);

    return 0;
}
```

运行之后，效果如下所示：

```
请输入第一个数字：12
请输入第二个数字：13
请输入第一个字符：请输入第二个字符： 
```

当程序执行到 `scanf` 时，如果输入缓冲区为空或者输入缓冲区内容无法与占位符匹配，程序会挂起暂停，等待用户输入。但是，如果输入缓冲区内容刚好能与占位符匹配，会直接从输入缓冲区中获取内容，而不会等待用户输入。具体过程分析如下：

缓冲区

代码

```
printf("请输入第一个数字: ");
scanf("%d", &num1);

printf("请输入第二个数字: ");
scanf("%d", &num2);

printf("请输入一个字符: ");
scanf("%c", &ch1);

printf("请输入第二个字符: ");
scanf("%c", &ch2);
```

终端

为避免缓冲区换行残留所引起的意外行为，我们可以在使用 `scanf()` 通过占位符 `%c` 获取字符之前，使用 `getchar()` 获取到缓冲区的空格，上面代码可以改为如下形式：

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // 定义两个 int 类型变量
```

```
    int num1, num2;
```

```
    // 定义两个 char 类型变量
```

```
    char ch1, ch2;
```

```
    // 获取用户输入，为变量进行初始化初始
```

```
    printf("请输入第一个数字: ");
```

```
    scanf("%d", &num1);
```

```
    printf("请输入第二个数字: ");
```

```
    scanf("%d", &num2);
```

```
    printf("请输入第一个字符: ");
```

```
    getchar(); // 从缓冲区中获取一个字符，用来获取上次输入的换行符
```

```
    scanf("%c", &ch1);
```

```
    printf("请输入第二个字符: ");
```

```
    getchar(); // 从缓冲区中获取一个字符，用来获取上次输入的换行符
```

```
    scanf("%c", &ch2);
```

```
// 打印
printf("num1=%d, num2=%d \n", num1, num2);
printf("ch1=%c, ch2=%c \n", ch1, ch2);

return 0;
}
```

17.2 VS Code 内置终端的中文编码问题



vscode输入中文的编码问题.pdf